Web アプリ開発 学習資料

08 Topcoat で練習アプリを作る（前編）— サーバー側の仕組み

06 と同じ依頼・承認アプリを、Rust＋Topcoat の実行ファイル 1 つで作る

08 Topcoat で練習アプリを作る（前編）: サーバー側の仕組み

この巻について

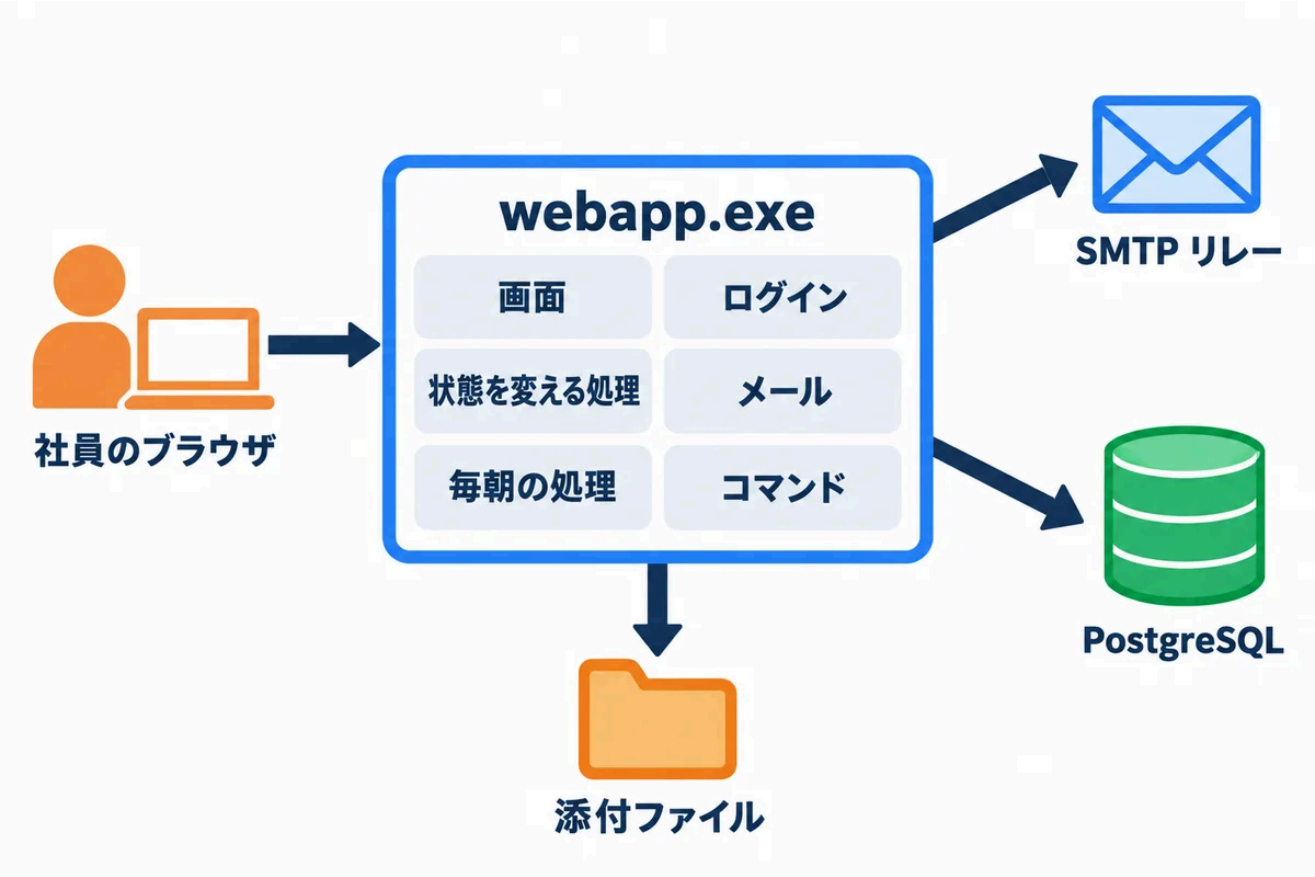
06 で Django＋React で作った「依頼・承認アプリ」を、もう 1 つの作り方 **Rust＋Topcoat** で作ります。できることは 06 と同じです（依頼を出す・メールの URL で承認する・差し戻す・出し直す・担当を決める・完了にする・毎朝の通知）。

- **前編（この巻）**: 設定・データベースの表・ログイン・状態を変える処理・メール・承認の URL・CSV の取り込み・毎朝の処理・テスト。
- **後編**: 画面（`view!`）・入力フォーム・図面・添付ファイル・HTTP の決まりの確かめ・サーバーへの持ち込み。
- **建てる先**: onprem の 04 のとおり、**実行ファイル `webapp.exe` 1 つ**（と、その隣の `assets` フォルダ）を、会社の Windows Server に WinSW で建てます。HTTP の 80 番で IIS が受け、名前で振り分けてアプリへ渡します。データベースはサーバーの中の PostgreSQL、メールは Microsoft 365 の SMTP リレー（25 番・STARTTLS）です。
- **前の巻**: 07（Rust と Topcoat の基礎）を先に読んでおくと、楽に読めます。06 前編と章の並びをそろえたので、見比べながら読むと「同じ仕事を、別の部品でどう書くか」が分かります。
- **試して確かめた範囲**: この巻のコード・出力・画面は、2026-11-02 に、Rust 1.99.1・Topcoat 0.11.0・sqlx 0.9.0・PostgreSQL 18.3 で**実際に動かした物**です。テストは 31 個（10 の巻で足した、安全のテスト 5 個を含む）、全部成功しています（13 章）。

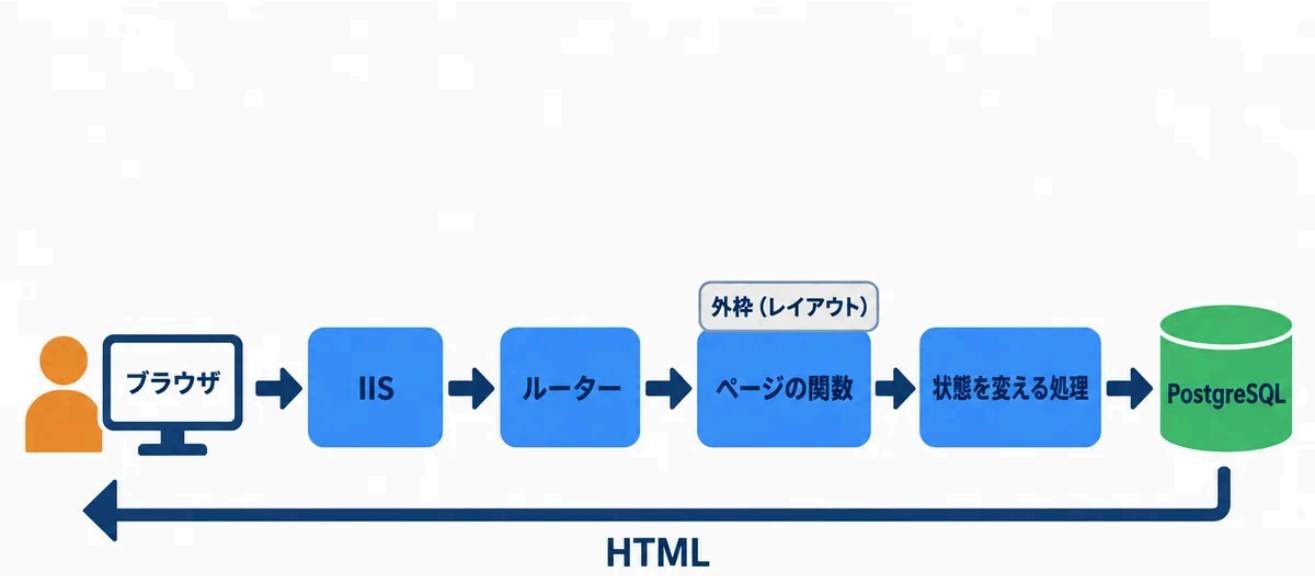
練習のコードは `samples/08_topcoat_app/webapp/` にあります。

ファイル	役目	06（Django）では
`src/config.rs`	設定（環境変数）を読む	`config/settings.py`
`migrations/0001_init.sql`	表を作る SQL	`models.py` と `migrations/`
`src/models.rs`	データの形・状態の名前	`models.py`
`src/auth.rs`・`src/http_cookie.rs`	ログイン・セッション	`django.contrib.auth`・`accounts/views.py`
`src/workflow.rs`	状態を変える処理（いちばん大事）	`workflow/services.py`
`src/mail.rs`	メール	`send_mail`・`transaction.on_commit`
`src/app/approve/token.rs`	承認の画面（メールの URL）	`approve_views.py`
`src/employees.rs`・`src/cli.rs`	CSV の取り込み・管理のコマンド	`management/commands/`
`src/jobs.rs`	毎朝 8:00 の通知	`send_daily_reminders.py`（タスク スケジューラ）
`tests/workflow.rs`	流れのテスト	`workflow/tests.py`
`src/app/`・`src/views.rs` など	画面（後編）	React（06 後編）

1. 全体の形



06 では、サーバー（Django）と画面（React）を別々に作り、React の成果物を Django から配りました。08 では、**画面の HTML もサーバーが作ります。**React も Node.js も要らず、できあがるのは `webapp.exe` 1 つです。



比べる所	06 Django＋React	08 Topcoat
画面	React がブラウザで組み立てる。サーバーとは JSON でやり取り	サーバーが HTML を作って返す（`view!`）。図面を描く所だけ小さな JavaScript
サーバーに置く物	Python・部品（wheelhouse）・React の成果物	`webapp.exe` と `assets` フォルダだけ
動かし方	waitress を WinSW で	`webapp.exe` を WinSW で
間違いに気付く時	動かしてから（テストで見つける）	多くはコンパイルの時（型が合わないと、exe ができない）
毎朝の通知	タスク スケジューラが `manage.ps1` を動かす	アプリの中で、1 分ごとに確かめる（11 章）
CSRF の守り	Django の CSRF トークン	Topcoat の OriginPolicy（`Origin` と `Host` を比べる。後編）

どちらも、「**状態を変える処理を 1 か所に集める**」「**保存が確定してからメールを送る**」「**同時に押されても 1 回だけ進める**」という考え方は同じです。

2. 開発の準備

▶ **準備:** 00 の 6.5（PostgreSQL）・6.6（Rust）と、07 の 1 章（`topcoat` の道具）。この巻のコマンドは、PowerShell で `webapp_learning\samples\08_topcoat_app\webapp` のフォルダにいるところから打ちます（資料のフォルダから `cd webapp_learning\samples\08_topcoat_app\webapp`）。

2.1 開発用のデータベース

06 と同じく、自分のパソコンの PostgreSQL に、練習用のユーザーとデータベースを作ります（PostgreSQL の入れ方は onprem の 02）。

コピー

```
-- psql で、管理者（postgres）として入って動かす。パスワードは自分で決める
CREATE ROLE webapp_topcoat_app LOGIN CREATEDB PASSWORD '（パスワード）';
CREATE DATABASE webapp_topcoat_dev OWNER webapp_topcoat_app ENCODING 'UTF8' TEMPLATE template0;
```

▶ **ここで初めて出てくるコード**

- **`PASSWORD`**（SQL）：`CREATE ROLE 名前 LOGIN CREATEDB PASSWORD '…'` の `PASSWORD` は、そのロール（データベースの利用者）がログインする時のパスワードを決める部分です。`LOGIN` はログインしてよい、`CREATEDB` はデータベースを作ってよい（テストで使う）という意味です。

- `CREATEDB`（データベースを作る権限）は、**開発のパソコンのテストのためだけ**に付けます。13 章のテストは、テストごとに空のデータベースを作るからです。
- サーバーのユーザー（`webapp_prod_app` など）には付けません（onprem の 04 の 5 章）。

2.2 部品（Cargo.toml）

コピー

```
[package]
name = "webapp"
version = "1.0.0"
edition = "2024"
rust-version = "1.99"
description = "依頼・承認アプリ（Rust＋Topcoat 版の練習）"
publish = false

# 版は = で固定する（Rust/ の巻・onprem の 04 と同じ考え方）
[dependencies]
anyhow = "=1.0.104"
argon2 = "=0.6.0"
csv = "=1.4.0"
dotenvy = "=0.15.7"
getrandom = "=0.4.3"
hex = "=0.4.3"
serde = { version = "=1.0.229", features = ["derive"] }
serde_json = "=1.0.154"
sha2 = "=0.11.1"
sqlx = { version = "=0.9.0", default-features = false, features = ["runtime-tokio", "postgres", "macros", "migrate", "time", "json"] }
time = { version = "=0.3.55", features = ["macros", "formatting", "parsing"] }
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros", "time", "fs"] }
topcoat = { version = "=0.11.0", default-features = false, features = [
    "asset", "cookie", "discover", "router", "serve", "session", "view", "mail", "mail-smtp", "multipart",
] }

[profile.dev]
debug = "line-tables-only"

[profile.dev.package."*"]
debug = false
```

部品	役目	06 では
`topcoat`	Web の土台（ルーター・画面・クッキー・セッション・メール・ファイルの受け取り）	Django・DRF
`sqlx`	PostgreSQL とのやり取り・マイグレーション	Django の ORM・psycopg
`tokio`	非同期（async）で動かす土台	waitress（スレッド）
`argon2`	パスワードのハッシュ	Django の `set_password`
`serde`・`serde_json`	JSON と Rust の形の行き来	DRF のシリアライザー
`time`	日付と時刻	`datetime`・`zoneinfo`
`sha2`・`hex`・`getrandom`	指紋（SHA-256）・16 進・乱数	`hashlib`・`secrets`
`csv`	CSV を読む	`csv`
`dotenvy`	開発のパソコンで `.env` を読む	`dev.ps1`
`anyhow`	コマンドのエラーを、まとめて扱う	（例外）

- 版は `=` で固定します（07 と同じ考え方）。`Cargo.lock` も Git に入れ、だれが作っても同じ部品になるようにします。
- `topcoat` は `default-features = false` にして、**使う機能だけ**を選んでいます。`mail-smtp` が SMTP リレーへの送信、`multipart` がファイルの受け取りです。

> **用語:** 機能（feature。部品の中の「使うか選べる部分」。選ばなければコンパイルされないので、exe が小さく、ビルドが速くなる）

2.3 設定のファイル（.env）

`.env.example` を `.env` という名前でコピーし、パスワードを直します。

コピー

```
# 開発のパソコン用の設定の見本。これを .env という名前でコピーして、値を直す（.env は Git に入れない）
# サーバーでは .env を使わず、onprem の 04 のサービスの XML（WinSW）に、同じ名前で書く

APP_ENV=development
HOST=127.0.0.1
PORT=3000
APP_BASE_URL=http://localhost:3000
# パスワードの所は、自分で決めた物にする
DATABASE_URL=postgres://webapp_dev:（パスワード）@127.0.0.1:5432/webapp_topcoat_dev
FILE_DIR=files
MAIL_FROM=noreply@example.com
# SMTP_HOST を書かなければ、メールは送らずに sent_mail フォルダへ .eml のファイルとして書く
# SMTP_HOST=example-com.mail.protection.outlook.com
# SMTP_PORT=25
# MAIL_REDIRECT_TO=it-team@example.com
```

- `.env` は **Git に入れません**（`.gitignore` に書いてある）。パスワードが入るからです。
- **サーバーでは `.env` を使いません。**同じ名前の値を、onprem の 04 のサービスの XML（WinSW）に書きます。名前をそろえてあるので、コードは変えずに済みます（3 章）。

2.4 動かす

コピー

```
topcoat asset bundle      # CSS と JavaScript を束ねる（target\debug\assets）
cargo run -- seed-demo    # 練習用の社員を作る
cargo run                 # サーバーとして動かす
```

実際の出力:

コピー

```
bundled assets into C:\…\08_topcoat_app\webapp\target\debug\assets
5 人を作りました。パスワードは全員 practice-pass-2026 です（練習用）。
起動します（Development）。メールの URL は http://localhost:3000 で始まります。
```

確かめること: ブラウザで `http://localhost:3000/` を開くと、ログインの画面に移る。
`http://localhost:3000/health` を開くと `ok` と出る。

- `cargo run` は、起動するときに**まだ流していないマイグレーション（表を作る SQL）を流します**（4 章）。表を作るために、別のコマンドを打つ必要はありません。
- `topcoat asset bundle` を忘れると、起動のときに「assets フォルダがありません」で止まります（15 章）。

3. 設定を読む（config.rs）

`src/config.rs`：

コピー

```rust
//! 設定を、環境変数から読む。名前は onprem の 04 のサービスの XML（WinSW）と同じ
//!
//! | 名前 | 意味 | 開発のパソコンで無いとき |
//! |---|---|---|
//! | `APP_ENV` | development / staging / production | development |
//! | `APP_BASE_URL` | メールに書く URL の頭 | http://localhost:3000 |
//! | `DATABASE_URL` | PostgreSQL の接続先 | （必ず要る） |
//! | `FILE_DIR` | 添付ファイルを置くフォルダ | ./files |
//! | `SMTP_HOST` / `SMTP_PORT` | メールの中継先（M365 の SMTP リレー）と、ポート | 無ければ ./sent_mail にファイルで書く |
//! | `SMTP_USE_TLS` | STARTTLS で暗号化するか（false は開発のパソコンだけ。09 の練習用の SMTP サーバー向け） | true |
//! | `MAIL_FROM` | 送り主のアドレス | noreply@example.com |
//! | `MAIL_REDIRECT_TO` | （検証だけ）全部のメールを、この宛先に送る | 無し |
//!
//! `HOST` と `PORT`（待ち受ける所）は、Topcoat の `start` が自分で読む。

use std::{env, fmt, path::PathBuf};

/// どこで動いているか
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AppEnv {
    Development,
    Staging,
    Production,
}

/// メールの出し方
#[derive(Debug, Clone)]
pub enum MailSettings {
    /// 開発のパソコン: 送らずに、.eml のファイルとしてフォルダに書く
    File(PathBuf),
    /// サーバー: SMTP リレーに渡す（starttls が true なら、STARTTLS で暗号化してから渡す）
    Smtp { host: String, port: u16, starttls: bool },
}

#[derive(Debug, Clone)]
pub struct Config {
    pub app_env: AppEnv,
    pub base_url: String,
    pub database_url: String,
    pub file_dir: PathBuf,
    pub mail: MailSettings,
    pub mail_from: String,
    pub mail_redirect_to: Option<String>,
}

/// 設定の間違い。起動のときに、すぐ止まって知らせる
#[derive(Debug)]
pub struct ConfigError(String);

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "設定の間違い: {}", self.0)
    }
}

impl std::error::Error for ConfigError {}

/// 空の値は「無い」と同じに扱う
fn var(name: &str) -> Option<String> {
    env::var(name).ok().map(|v| v.trim().to_string()).filter(|v| !v.is_empty())
}

impl Config {
    pub fn from_env() -> Result<Config, ConfigError> {
        let app_env = match var("APP_ENV").as_deref() {
            None | Some("development") => AppEnv::Development,
            Some("staging") => AppEnv::Staging,
            Some("production") => AppEnv::Production,
            Some(other) => return Err(ConfigError(format!("APP_ENV は development・staging・production のどれかです（今: {other}）"))),
        };
        let dev = app_env == AppEnv::Development;

        // サーバーでは、決まりの値に頼らない（書き忘れに、すぐ気付けるように）
        let required = |name: &str, dev_default: &str| -> Result<String, ConfigError> {
            match var(name) {
                Some(v) => Ok(v),
                None if dev => Ok(dev_default.to_string()),
                None => Err(ConfigError(format!("{name} がありません"))),
            }
        };

        let database_url = var("DATABASE_URL").ok_or_else(|| ConfigError("DATABASE_URL がありません".into()))?;
```

```
        let base_url = required("APP_BASE_URL", "http://localhost:3000")?.trim_end_matches('/').to_string();
        let file_dir = PathBuf::from(required("FILE_DIR", "files")?);
        let mail_from = required("MAIL_FROM", "noreply@example.com")?;

        let mail = match var("SMTP_HOST") {
            Some(host) => {
                let port = var("SMTP_PORT").unwrap_or_else(|| "25".into());
                let port = port.parse().map_err(|_| ConfigError(format!("SMTP_PORT が数ではありません（今: {port}）")))?;
                let starttls = match var("SMTP_USE_TLS").as_deref() {
                    None | Some("true") => true,
                    Some("false") => false,
                    Some(other) => return Err(ConfigError(format!("SMTP_USE_TLS は true か false です（今: {other}）"))),
                };
                // 暗号化しないで送るのは、開発のパソコンの中の練習だけ（会社のネットワークに平文で流さない）
                if !starttls && !dev {
                    return Err(ConfigError("SMTP_USE_TLS=false は、APP_ENV=development のときだけ使えます".into()));
                }
                MailSettings::Smtp { host, port, starttls }
            }
            None if dev => MailSettings::File(PathBuf::from("sent_mail")),
            None => return Err(ConfigError("SMTP_HOST がありません".into())),
        };

        let mail_redirect_to = var("MAIL_REDIRECT_TO");
        // 検証のサーバーから、本物の宛先にメールが届く事故を防ぐ
        if app_env == AppEnv::Staging && mail_redirect_to.is_none() {
            return Err(ConfigError("APP_ENV=staging のときは MAIL_REDIRECT_TO が要ります".into()));
        }

        Ok(Config { app_env, base_url, database_url, file_dir, mail, mail_from, mail_redirect_to })
    }
}
```

▶ ここで初めて出てくるコード

- `return`（関数）：関数を、その場で終えて値を返す命令。`return Err(…);` のように、途中で抜ける時に使います。Rust では、関数の最後の `;` の無い式が自動で戻り値になるので、最後の行では `return` を書かないのがふつうです。
- `.ok()`（エラー処理）：`Result` を `Option` に変えるメソッド。`Ok(値)` は `Some(値)` に、`Err(…)` は `None` になります（エラーの中身は捨てる）。`"42".parse().ok()` のように、「失敗したら無かったことにする」時に使います。
- `.as_deref()`（文字列）：`Option<String>` を、中身を借りて `Option<&str>` に変えるメソッド。`env::args().nth(1).as_deref() == Some("reset")` のように、「引数が `reset` かどうか」を文字列と比べる時に使います。
- `.ok_or_else()`（エラー処理）：`Option` を `Result` に変えるメソッド。`None` の時に、渡した関数でエラーを作ります（必要な時だけ作る `ok_or` の版）。
- `.unwrap_or_else()`（エラー処理）：`Option` や `Result` から中身を取り出し、無い（失敗した）時は、渡した関数を動かして代わりの値を作るメソッド。`unwrap_or` と違い、代わりの値を作る処理が、必要な時だけ動きます。
- `.parse()`（文字列）：文字列を、数などの別の型に変えるメソッド。`"42".parse::<i32>()` や、`let n: i32 = "42".parse().unwrap();` のように、何に変えるかを型で伝えます。変えられない文字（`"abc"` など）もあるので、結果は `Result` です。
- `.map_err()`（エラー処理）：`Result` のエラーの方だけを、別の物に変えるメソッド。`.map_err(|_| ConfigError::BadPort)` で、元のエラーを自分のエラーの型に変えます。成功の値はそのままです。
- `.is_none()`（Option）：`Option` が `None`（値が無い）なら `true` を返すメソッド。
- `env::var()`（プロセス）：環境変数（プログラムの外から渡す設定の値）を読む関数。`env::var("GREETING")`。設定されていないこともあるので、`Result` で返ります。`unwrap_or_else(|_| "既定値".to_string())` で、無い時の値を決めるのがふつうです。
- `PathBuf::from()`（ファイルとフォルダ）：文字列から、ファイルやフォルダの場所（`PathBuf`）を作る関数。`PathBuf::from("files")`。あとで `.join("名前")` で、下の場所をつなげられます。
- `PathBuf`（ファイルとフォルダ）：ファイルやフォルダの場所（パス）を、自分の物として持つ型（`String` のパス版）。`path.join("a")` で場所をつなげ、`.push("b")` で足せます。借りるだけの形は `&Path` です。
- `None`（Option）：`Option` の「値が無い」側。`Some(値)`（値がある）か `None`（無い）のどちらかになります。Rust には「null」が無い代わりに、無いかもしれない値は必ず `Option` で表し、`match` などで両方の場合を書きます。

- **サーバーでは、決まりの値に頼りません。** `APP_ENV` が `development` のときだけ、書いていない値に開発用の値を入れます。サーバーで書き忘れたら、起動のときにすぐ止まって知らせます。
- **検証のサーバーで、本物の宛先にメールが届く事故を防ぎます。** `APP_ENV=staging` なのに `MAIL_REDIRECT_TO` が無ければ、起動しません。

実際に試した出力（`APP_ENV=staging` で `MAIL_REDIRECT_TO` を書かずに起動）：

コピー

```
エラー: 設定の間違い: APP_ENV=staging のときは MAIL_REDIRECT_TO が要ります
```

環境変数	開発のパソコン（.env）	サーバー（onprem の 04 の XML）
`HOST`・`PORT`	127.0.0.1・3000	127.0.0.1・8080（本番）／8081（検証）
`APP_ENV`	development	production／staging
`APP_BASE_URL`	http://localhost:3000	http://webapp.example.local など
`DATABASE_URL`	自分のパソコンの PostgreSQL	サーバーの中の PostgreSQL
`FILE_DIR`	files	D:¥webapp¥prod¥files など
`SMTP_HOST`・`SMTP_PORT`	書かない（ファイルに書く）	M365 の SMTP リレー・25
`SMTP_USE_TLS`	書かない（09 の練習用の SMTP サーバーに送るときだけ false）	書かない（true。STARTTLS で暗号化）
`MAIL_FROM`	noreply@example.com	会社で決めた送り主
`MAIL_REDIRECT_TO`	書かない	検証だけ書く

4. 表（マイグレーション）とデータの形

4.1 表を作る SQL

06 では、`models.py` から Django がマイグレーションを作りました。sqlx では、**表を作る SQL を自分で書きます。** `migrations/` フォルダに番号を付けて置くと、アプリが起動のときに、まだ流していない物を番号の順に流します（`src/state.rs` の `migrate`）。

`migrations/0001_init.sql`：

コピー

```
-- 依頼・承認アプリの表（06 の Django 版の models.py と同じ中身を、SQL で書いた物）
-- アプリが起動したときに sqlx が流す。1 度流したファイルは、あとから書きかえない（直すときは 0002_... を足す）

-- 社員（ログインする人）
CREATE TABLE employees (
    id            BIGSERIAL PRIMARY KEY,
    employee_no   TEXT    NOT NULL UNIQUE,                -- 従業員番号（ログインに使う）
    name          TEXT    NOT NULL,
    dept          TEXT    NOT NULL DEFAULT '',
    job_title     TEXT    NOT NULL DEFAULT '',
    email         TEXT    NOT NULL,
    role          TEXT    NOT NULL DEFAULT 'member'
                  CHECK (role IN ('member', 'worker', 'admin')),
    password_hash TEXT,                                   -- NULL の人は、ログインできない
    is_active     BOOLEAN NOT NULL DEFAULT TRUE
);

-- ログイン中の記録。クッキーの文字列そのものではなく、その指紋（SHA-256）を持つ
CREATE TABLE sessions (
    token_hash  BYTEA       PRIMARY KEY,
    employee_id BIGINT      NOT NULL REFERENCES employees (id) ON DELETE CASCADE,
    expires_at  TIMESTAMPTZ NOT NULL
);

-- 依頼 1 件
CREATE TABLE requests (
    id              BIGSERIAL   PRIMARY KEY,
    title           TEXT        NOT NULL CHECK (char_length(title) BETWEEN 1 AND 100),
    detail          TEXT        NOT NULL DEFAULT '',
    due_date        DATE        NOT NULL,
    status          TEXT        NOT NULL DEFAULT 'circulating'
                    CHECK (status IN ('circulating', 'returned', 'pulled_back', 'approved',
                                      'in_progress', 'done', 'withdrawn')),
    requester_id    BIGINT      NOT NULL REFERENCES employees (id),
    assignee_id     BIGINT      REFERENCES employees (id),
    drawing         JSONB,                                -- 図面（無ければ NULL）
    return_comments TEXT        NOT NULL DEFAULT '',      -- 差し戻すたびに追記する
    round           SMALLINT    NOT NULL DEFAULT 1,       -- 出し直すたびに 1 増える
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX requests_requester_id ON requests (requester_id);

-- 回覧の 1 人分。承認の URL の指紋も、ここに持つ
CREATE TABLE steps (
    id               BIGSERIAL   PRIMARY KEY,
    request_id       BIGINT      NOT NULL REFERENCES requests (id) ON DELETE CASCADE,
    round            SMALLINT    NOT NULL,
    step_order       SMALLINT    NOT NULL,                -- 何番目か（order は SQL の言葉なので避ける）
    approver_id      BIGINT      NOT NULL REFERENCES employees (id),
    decision         TEXT        NOT NULL DEFAULT 'waiting'
                     CHECK (decision IN ('waiting', 'approved', 'returned', 'cancelled')),
    decided_at       TIMESTAMPTZ,
    token_hash       TEXT,                                -- URL の文字列の指紋（16 進）
    token_expires_at TIMESTAMPTZ,
    sent_at          TIMESTAMPTZ,
    UNIQUE (request_id, round, step_order)
);
CREATE INDEX steps_token_hash ON steps (token_hash);

-- 添付ファイル。中身はフォルダ（FILE_DIR）に置き、ここには名前と大きさだけを持つ
CREATE TABLE attachments (
    id            BIGSERIAL   PRIMARY KEY,
```

```
    request_id    BIGINT      NOT NULL REFERENCES requests (id) ON DELETE CASCADE,
    stored_name   TEXT        NOT NULL,                   -- フォルダに置いた名前（アプリが決める）
    original_name TEXT        NOT NULL,                   -- 送られてきたときの名前（表示とダウンロードに使う）
    content_type  TEXT        NOT NULL,
    size          INTEGER     NOT NULL,
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- だれが、いつ、何をしたかの記録（あとから変えない）
CREATE TABLE histories (
    id         BIGSERIAL   PRIMARY KEY,
    request_id BIGINT      NOT NULL REFERENCES requests (id) ON DELETE CASCADE,
    at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    actor_id   BIGINT      REFERENCES employees (id),      -- 毎朝の処理など、人でないときは NULL
    action     TEXT        NOT NULL,
    note       TEXT        NOT NULL DEFAULT ''
);

-- 毎朝の処理を、その日に済ませたかの記録（同じ日に 2 回送らないため）
CREATE TABLE job_runs (
    job         TEXT        NOT NULL,
    run_date    DATE        NOT NULL,
    started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (job, run_date)
);
```

▶ ここで初めて出てくるコード

- `TABLE`（SQL）：SQL で「表」を表す言葉。`CREATE TABLE`（表を作る）・`DROP TABLE`（表を消す）のように使います。
- `BIGSERIAL`（SQL）：PostgreSQL の、行を足すたびに 1, 2, 3… と自動で番号を振る列の型（大きな整数）。`id BIGSERIAL PRIMARY KEY` のように、行の番号に使います。
- `PRIMARY`（SQL）：`PRIMARY KEY`（主キー）の一部。主キーは、表の中で 1 行を必ず見分けられる列で、同じ値が 2 回入りません。ふつうは `id` 列にします。
- `KEY`（SQL）：`PRIMARY KEY`（主キー）・`FOREIGN KEY`（よその表とつなぐ列）などの「キー」。
- `TEXT`（SQL）：SQL の列の型の 1 つで、文字を入れる列。
- `NOT`（SQL）：SQL で「〜ではない」。`NOT NULL` は「この列は空（NULL）を許さない」という決まりです。
- `NULL`（SQL）：SQL で「値が無い」ことを表す特別な値。0 や空の文字列とは違います。
- `UNIQUE`（SQL）：「この列の値は、表の中で重ならない」という決まり。`email TEXT NOT NULL UNIQUE` は、同じメールアドレスを 2 回登録できなくします。
- `DEFAULT`（SQL）：列に値を入れなかった時に使う、決めておいた値（既定値）。`done INTEGER NOT NULL DEFAULT 0` は「指定が無ければ 0」。
- `CHECK`（SQL）：「この列に入れてよい値の条件」を決める決まり。`CHECK (length(title) BETWEEN 1 AND 80)` は、題名を 1〜80 文字に限ります。合わない値を入れようとすると、データベースが断ります。
- `IN`（SQL）：SQL で「このどれか」。`WHERE status IN ('queued', 'running')` は、状態がどちらかの行だけです。`CHECK (pinned IN (0, 1))` は、0 か 1 しか入れられない決まりです。
- `BOOLEAN`（SQL）：`TRUE`（はい）か `FALSE`（いいえ）を入れる列の型。`is_active BOOLEAN NOT NULL DEFAULT TRUE` は「使えるか。決まりは はい」。
- `TRUE`（SQL）：SQL の「はい」（真）の値。逆は `FALSE` です。
- `BYTEA`（SQL）：PostgreSQL の、バイトの並び（文字でないデータ）を入れる列の型。ここでは、承認の URL の文字列のハッシュ（SHA-256 の 32 バイト）を入れています。
- `BIGINT`（SQL）：大きな整数（約 ±900 京まで）を入れる列の型。`BIGSERIAL` の番号を指す列（`REFERENCES employees (id)`）は、同じ大きさの `BIGINT` にします。Rust では `i64` で受け取ります。
- `REFERENCES`（SQL）：「この列の値は、あの表の、あの列に必ずある値」という決まり（外部キー）。`user_id INTEGER REFERENCES users(id)` で、存在しないユーザーの番号を入れられなくなります。
- `ON`（SQL）：SQL の「〜の時」「〜で」。`ON DELETE CASCADE` は「消された時は」、`JOIN … ON a.id = b.user_id` は「この条件で横につなぐ」、`CREATE INDEX … ON 表 (列)` は「この表の、この列に」という意味です。
- `DELETE`（SQL）：`DELETE FROM 表 WHERE 条件` で、表の行を消す SQL の命令。`WHERE` を忘れると、全部の行が消えます。`ON DELETE CASCADE` の中では「消された時」という意味です。
- `CASCADE`（SQL）：`ON DELETE CASCADE` は、「つながった元の行（ユーザー）が消されたら、こちらの行（そのユーザーのノート）も一緒に消す」という決まりです。

- **`TIMESTAMPTZ`**（SQL）: PostgreSQL の、時差（タイムゾーン）付きの日時の型。中では世界標準時（UTC）で持ち、表示する時に日本時間などに変えます。Rust では `time::OffsetDateTime` で受け取ります。
- **`BETWEEN`**（SQL）:「〜から〜の間」。`CHECK (score BETWEEN 0 AND 100)` は「0 以上 100 以下だけ許す」という決まりです。
- **`AND`**（SQL）: SQL で「かつ」。`WHERE a = 1 AND b = 2` は両方に合う行だけ。`BETWEEN 0 AND 100` の一部としても使います。
- **`DATE`**（SQL）: 日付（年・月・日。時刻は無し）を入れる列の型。期限（`due_date`）などに使います。Rust では `time::Date` で受け取ります。
- **`JSONB`**（SQL）: PostgreSQL の、JSON を入れる列の型。中で扱いやすい形に変えて持つので、速く探せます。ここでは図面（四角や線の並び）を入れています。Rust では `sqlx::types::Json<型>` で受け取ります。
- **`SMALLINT`**（SQL）: 小さな整数（−32768〜32767）を入れる列の型。何回目か（`round`）・何番目か（`step_order`）などに使います。Rust では `i16` で受け取ります。
- **`INDEX`**（SQL）: `CREATE INDEX 名前 ON 表（列）` で、表に索引（本の索引のような、速く探すための仕組み）を作る SQL の命令。よく条件や並べかえに使う列に作ります。
- **`INTEGER`**（SQL）: SQL の列の型の 1 つで、整数を入れる列。

- **1 度流したファイルは、あとから書きかえません。** sqlx は、流したファイルの指紋を `_sqlx_migrations` の表に記録し、書きかえられていると起動を止めます。表を直すときは `0002_….sql` を足します。
- `CHECK` で、状態などに入ってよい値を、**データベースの側でも**決めておきます（アプリの間違いで変な値が入るのを防ぐ、もう 1 つの柵）。
- 06 の表と、ほぼ同じ形です。`order` は SQL の言葉（`ORDER BY`）なので、`step_order` にしました。
- `job_runs` は 08 で足した表です（11 章）。

用語: マイグレーション（表の形を変える手順を、ファイルにして順に流す仕組み。どのパソコン・サーバーでも、同じ形の表ができる）

4.2 データの形（models.rs）

`src/models.rs`:

コピー

```
//! データの形（表の 1 行）と、状態の名前。07 の status クレートを、データベースに合わせて広げた物

use time::{Date, OffsetDateTime, UtcOffset, macros::{format_description, offset}};

/// 日本の時刻（日本には夏時間が無いので、いつも +9 時間でよい）
pub const JST: UtcOffset = offset!(+9);

/// 日時を「2026-11-02 09:00」の形にする（日本の時刻で）
pub fn fmt_datetime(t: OffsetDateTime) -> String {
    t.to_offset(JST)
        .format(format_description!("[year]-[month]-[day] [hour]:[minute]"))
        .unwrap_or_default()
}

/// 日付を「2026-11-02」の形にする
pub fn fmt_date(d: Date) -> String {
    d.format(format_description!("[year]-[month]-[day]")).unwrap_or_default()
}

/// ファイルの大きさを「12 KB」「800 バイト」の形にする
pub fn fmt_size(bytes: i32) -> String {
    if bytes < 1024 { format!("{bytes} バイト") } else { format!("{} KB", bytes / 1024) }
}

/// 今日の日付（日本の時刻で）
pub fn today_jst() -> Date {
    OffsetDateTime::now_utc().to_offset(JST).date()
}

/// 文字（データベースの値）と enum を行き来させる、決まった書き方をまとめるマクロ
macro_rules! text_enum {
    ($name:ident { $($variant:ident => ($text:literal, $label:literal)),+ $(,)? }) => {
        #[derive(Debug, Clone, Copy, PartialEq, Eq)]
        pub enum $name { $($variant),+ }

        impl $name {
            /// データベースに入れる文字
            pub fn as_str(self) -> &'static str {
                match self { $($name::$variant => $text),+ }
            }
            /// 画面に出す名前
            pub fn label(self) -> &'static str {
                match self { $($name::$variant => $label),+ }
            }
        }

        // データベースから読んだ文字を enum にする（知らない文字なら、読み込みのエラー）
        impl TryFrom<String> for $name {
            type Error = String;
            fn try_from(s: String) -> Result<Self, String> {
                match s.as_str() {
                    $($text => Ok($name::$variant),)+
                    _ => Err(format!("{} に、知らない値があります: {s}", stringify!($name))),
                }
            }
        }
    };
}
```

```
text_enum!(Status {
    Circulating => ("circulating", "回覧中"),
    Returned => ("returned", "差し戻し中"),
    PulledBack => ("pulled_back", "引き戻し中"),
    Approved => ("approved", "回覧完了"),
    InProgress => ("in_progress", "対応中"),
    Done => ("done", "完了"),
    Withdrawn => ("withdrawn", "取り下げ"),
});

impl Status {
    /// 全部の状態（流れの順）
    pub const ALL: [Status; 7] = [
        Status::Circulating,
        Status::Returned,
        Status::PulledBack,
        Status::Approved,
        Status::InProgress,
        Status::Done,
        Status::Withdrawn,
    ];

    /// 01 の CSS の札の class（badge-circulating など）
    pub fn css_class(self) -> String {
        format!("badge badge-{}", self.as_str().replace('_', "-"))
    }
}

text_enum!(Decision {
    Waiting => ("waiting", "待ち"),
    Approved => ("approved", "承認"),
    Returned => ("returned", "差し戻し"),
    Cancelled => ("cancelled", "取り消し"),
});

text_enum!(Role {
    Member => ("member", "一般"),
    Worker => ("worker", "担当者"),
    Admin => ("admin", "管理者"),
});

impl Role {
    /// 担当を決めたり、全部の依頼を見たりできるか
    pub fn is_staff(self) -> bool {
        matches!(self, Role::Worker | Role::Admin)
    }
}

/// 社員 1 人
#[derive(Debug, Clone, sqlx::FromRow)]
pub struct Employee {
    pub id: i64,
    pub employee_no: String,
    pub name: String,
    pub dept: String,
    pub job_title: String,
    pub email: String,
    #[sqlx(try_from = "String")]
    pub role: Role,
}

/// 一覧の 1 行
```

```rust
#[derive(Debug, Clone, sqlx::FromRow)]
pub struct RequestRow {
    pub id: i64,
    pub title: String,
    pub due_date: Date,
    #[sqlx(try_from = "String")]
    pub status: Status,
    pub requester_name: String,
    /// 回覧中なら、今だれの承認を待っているか
    pub waiting_for: Option<String>,
    pub created_at: OffsetDateTime,
}

/// 詳しい画面に出す、依頼 1 件
#[derive(Debug, Clone, sqlx::FromRow)]
pub struct RequestDetail {
    pub id: i64,
    pub title: String,
    pub detail: String,
    pub due_date: Date,
    #[sqlx(try_from = "String")]
    pub status: Status,
    pub requester_id: i64,
    pub requester_name: String,
    pub requester_dept: String,
    pub assignee_id: Option<i64>,
    pub assignee_name: Option<String>,
    /// JSONB の列を、図面の struct として読む（Json で包む）
    pub drawing: Option<sqlx::types::Json<crate::drawing::Drawing>>,
    pub return_comments: String,
    pub round: i16,
    pub created_at: OffsetDateTime,
}

/// 回覧の 1 人分（詳しい画面の表）
#[derive(Debug, Clone, sqlx::FromRow)]
pub struct StepRow {
    pub id: i64,
    pub round: i16,
    pub step_order: i16,
    pub approver_id: i64,
    pub approver_name: String,
    pub approver_dept: String,
    #[sqlx(try_from = "String")]
    pub decision: Decision,
    pub decided_at: Option<OffsetDateTime>,
}

#[derive(Debug, Clone, sqlx::FromRow)]
pub struct AttachmentRow {
    pub id: i64,
    pub request_id: i64,
    pub stored_name: String,
    pub content_type: String,
    pub original_name: String,
    pub size: i32,
    pub uploaded_at: OffsetDateTime,
}

#[derive(Debug, Clone, sqlx::FromRow)]
pub struct HistoryRow {
    pub at: OffsetDateTime,
```

```
    pub actor_name: Option<String>,
    pub action: String,
    pub note: String,
}
```

▶ ここで初めて出てくるコード

- **`offset!`**（時間）：`time` の部品で、時差（`UtcOffset`）を書くマクロ。`offset!(+9)` は「世界標準時より 9 時間進んでいる」＝日本時間です。プログラムを作る時に確かめられるので、まちがった書き方はエラーになります。
- **`format_description!`**（時間）：`time` の部品で、日時を文字にする形（書式）を書くマクロ。`format_description!("[year]-[month]-[day] [hour]:[minute]")` は「2026-11-02 08:00」の形です。`.format(…)` に渡して使います。
- **`macro_rules!`**（型とメソッド）：自分でマクロ（同じ形のコードを、まとめて作り出す仕組み）を作る書き方。`macro_rules! 名前 { (受け取る形) => { 作るコード } }`。ここでは、似た enum とその変換を、まとめて作る `text_enum!` を作っています。`$name:ident` は「名前を 1 つ受け取る」、`$( … ),*` は「くり返し受け取る」という印です。
- **`stringify!`**（型とメソッド）：渡したコードを、そのまま文字列にするマクロ。`stringify!(Status)` は `"Status"`。エラーの文に、型の名前を入れる時に使っています。
- **`text_enum!`**（練習アプリで作る物（Rust））：この巻で `macro_rules!` を使って作るマクロ。`text_enum!(Status { Circulating => ("circulating", "回覧中"), … })` と書くと、enum と、データベースの文字との行き来（`as_str`）、画面に出す日本語（`label`）、文字から enum に戻す変換（`TryFrom<String>`）を、まとめて作ります。
- **`matches!`**（場合分け）：値が、ある形（パターン）に合うかを `true`・`false` で返すマクロ。`matches!(filter, Filter::Active | Filter::Done)` のように、`match` を 1 行で書けます。
- **`const`**（変数）：ずっと変わらない値（定数）に名前を付ける書き方。`const MAX_TITLE_CHARS: usize = 80;`。名前は大文字と `_` で書き、型（`usize` など）を必ず書きます。プログラムのどこからでも使えます。
- **`else`**（条件）：`if` の条件が合わなかった時の処理を書く所。`else if 条件 { … }` で、条件をいくつも続けられます。
- **`Self`**（型とメソッド）：`impl` の中で、「今、関数を付けている型そのもの」を表す名前。`fn new(…) -> Self` は「この型の値を返す」という意味で、型の名前を何度も書かずに済みます。
- **`#[sqlx]`**（データベース）：`sqlx` に、列の読み方を伝える印。`#[sqlx(try_from = "String")]` は「この列は、まず文字として読み、`TryFrom` で（ここでは enum に）変える」という意味です。
- **`.to_offset()`**（時間）：日時（`OffsetDateTime`）を、決めた時差の日時に変えるメソッド。`t.to_offset(JST)` は日本時間にします。指す瞬間は同じで、表し方だけが変わります。
- **`.format()`**（時間）：日時を、決めた形（`format_description!`）の文字列にするメソッド。結果は `Result` なので、`.unwrap_or_default()`（失敗したら空の文字）などで受け取ります。
- **`.unwrap_or_default()`**（エラー処理）：`Option` や `Result` から中身を取り出し、無い（失敗した）時は、その型の「空の値」（文字列なら `""`、数なら 0）を使うメソッド。
- **`.date()`**（時間）：日時（`OffsetDateTime`）から、日付（`Date`。年・月・日）だけを取り出すメソッド。

- **`.replace()`**（文字列）：文字列の中の、ある部分を全部置きかえた、新しい文字列を返すメソッド。`"a-b".replace('-', "/")` は `"a/b"` 。元の文字列は変わりません。
- **`OffsetDateTime::now_utc()`**（時間）：今の日時を、世界標準時（UTC）で返す関数。日本時間にする時は `.to_offset(JST)` をつなげます。
- **`Date`**（時間）：`time` の、日付（年・月・日）を表す型。データベースの `DATE` の列と行き来できます。
- **`OffsetDateTime`**（時間）：`time` の、時差付きの日時を表す型。データベースの `TIMESTAMPTZ` の列と行き来できます。
- **`UtcOffset`**（時間）：`time` の、時差（世界標準時から何時間ずれているか）を表す型。`offset!(+9)` で日本の時差を作れます。
- **`TryFrom`**（型とメソッド）：「ある型から作れるが、失敗することもある」変換の機能（トレイト）。`impl TryFrom<CreateNoteInput> for NewNote` を書くと、`NewNote::try_from(input)` で、入力を確かめながら変換でき、だめなら `Err` を返します。
- **`Self`**（型とメソッド）：`impl` の中で、「今、関数を付けている型そのもの」を表す名前（`-> Self`・`Self { … }` など）。
- **`ALL`**（SQL）：`UNION ALL` は、2 つの結果を、重なりも消さずにそのままつなげる書き方です。
- **`Decision`**（練習アプリで作る物（Rust））：承認する人 1 人ごとの答え（待ち・承認・差し戻し・取り消し）を表す enum。`text_enum!` で作ります。
- **`Waiting`**（練習アプリで作る物（Rust））：`Decision` の値の 1 つで、「まだ答えていない（待ち）」。
- **`Cancelled`**（練習アプリで作る物（Rust））：`Decision` の値の 1 つで、「依頼が取り下げられたので、答えなくてよくなった（取り消し）」。
- **`Role`**（練習アプリで作る物（Rust））：社員の役割（一般・担当者・管理者）を表す enum。`text_enum!` で作ります。担当者と管理者は、全部の依頼を見たり、担当を決めたりできます。
- **`Member`**（練習アプリで作る物（Rust））：`Role` の値の 1 つで、ふつうの社員（一般）。自分の依頼を出し、頼まれた承認をします。
- **`Worker`**（練習アプリで作る物（Rust））：`Role` の値の 1 つで、依頼を受けて作業する担当者。
- **`Admin`**（練習アプリで作る物（Rust））：`Role` の値の 1 つで、管理者。全部の依頼を見て、担当を決められます。
- **`FromRow`**（データベース）：`sqlx` の部品で、SQL の結果の 1 行を、struct に読み込めるようにする印。`#[derive(sqlx::FromRow)]` を付けると、列の名前と同じ名前の項目に、自動で入れてくれます。
- **`Json`**（データベース）：`sqlx` の `Json<型>`（`sqlx::types::Json`）は、`JSONB` の列を、Rust の struct として読み書きするための包みです。`Option<Json<Drawing>>` は「図面が無ければ `None`」。（Topcoat にも、答えを JSON で返すための `Json` があり、名前は同じでも別の物です。）
- **`Drawing`**（練習アプリで作る物（Rust））：依頼に付ける図面（四角や線の並び）を表す struct。後編で作り、`JSONB` の列に入れます。

- データベースには状態を文字（`'circulating'` など）で入れ、Rust の中では **enum** で扱います。`text_enum!` は、その行き来（`as_str`・`label`・`TryFrom<String>`）を 3 つの enum に同じように付けるための、小さなマクロです。
- `#[sqlx(try_from = "String")]` は、「この列は文字として読み、`TryFrom` で enum に変える」という印です。データベースに知らない文字が入っていたら、読み込みがエラーになります。
- **日本には夏時間が無い**ので、日本の時刻は「いつも +9 時間」で表せます（`JST`）。06 の Python では `tzdata` が要りましたが、ここでは要りません。

5. ログインとセッション



5.1 HTTP 用のクッキー（http_cookie.rs）

Topcoat 0.11 の決まりのクッキーには `Secure` が付き、**HTTP のページではログインが続きません。** onprem の 04 の 6.4 で確かめた `HttpCookieTokenStore` を、そのまま使います（`src/http_cookie.rs`。中身は onprem の 04 と同じなので、ここでは省きます）。

5.2 ログイン・ログアウト・今だれか（auth.rs）

`src/auth.rs`：

コピー

```
//! ログイン・ログアウトと、「今だれがログインしているか」
//!
//! - パスワードは、Argon2 で「元に戻せない形（ハッシュ）」にして保存する。そのままは保存しない。
//! - ログインすると、Topcoat が推測できない文字列（トークン）を作り、クッキーでブラウザに渡す。
//!   データベースには、その指紋（SHA-256）と期限だけを入れる（sessions の表）。
//! - 最後に使ってから 60 分で、ログインが切れる（使うたびに期限を 60 分先へ延ばす）。

use std::{sync::LazyLock, time::Duration};

use argon2::{
    Argon2,
    password_hash::{PasswordHasher, PasswordVerifier, phc::PasswordHash},
};
use time::OffsetDateTime;
use topcoat::{
    context::{Cx, memoize},
    router::{error::see_other, href},
    session::{self, SessionConfig},
};

use crate::{models::Employee, state::state};

/// 何も操作しないと、この時間でログインが切れる
pub const IDLE: Duration = Duration::from_secs(60 * 60);

/// ルーターに渡す、セッションの設定（HTTP 用のクッキー・60 分）
pub fn session_config() -> SessionConfig {
    SessionConfig::builder().token_store(crate::http_cookie::HttpCookieTokenStore).lifetime(IDLE).build()
}

// ---------- パスワード ----------

/// パスワードをハッシュにする。わざと重い計算なので、別のスレッドで動かす（ほかの人の画面を止めない）
pub async fn hash_password(password: String) -> String {
    tokio::task::spawn_blocking(move || hash_password_blocking(&password)).await.expect("ハッシュの計算が止まりました")
}

fn hash_password_blocking(password: &str) -> String {
    // 塩（salt）は、毎回ちがう乱数を Argon2 が自分で作り、結果の文字列の中に入れる
    Argon2::default().hash_password(password.as_bytes()).expect("ハッシュにできません").to_string()
}

/// パスワードが合っているか。合っていなければ false
pub async fn verify_password(password: String, hash: String) -> bool {
    tokio::task::spawn_blocking(move || {
        let Ok(parsed) = PasswordHash::new(&hash) else { return false };
        Argon2::default().verify_password(password.as_bytes(), &parsed).is_ok()
    })
    .await
    .unwrap_or(false)
}

/// 社員がいないときにも、同じ重さの確かめをするための、にせのハッシュ。
/// 「いない社員は早く断られる」と、時間の差で社員番号を探られてしまうので、それを防ぐ
static DUMMY_HASH: LazyLock<String> = LazyLock::new(|| hash_password_blocking("dummy-password-for-timing"));

// ---------- ログイン・ログアウト ----------

/// 社員番号とパスワードを確かめて、合っていればログインさせる。合っていなければ Ok(false)
pub async fn login(cx: &Cx, employee_no: &str, password: &str) -> topcoat::Result<bool> {
    let pool = &state(cx).pool;
    let row: Option<(i64, Option<String>)> =
        sqlx::query_as("SELECT id, password_hash FROM employees WHERE employee_no = $1 AND is_active")
            .bind(employee_no.trim())
            .fetch_optional(pool)
            .await?;

    let (id, hash) = match row {
        Some((id, Some(hash))) => (Some(id), hash),
        _ => (None, DUMMY_HASH.clone()), // いない・パスワードが無い人も、同じだけ時間をかける
    };
    let ok = verify_password(password.to_string(), hash).await;
    let Some(employee_id) = id.filter(|_| ok) else {
```

```
        return Ok(false);
    };

    // 新しいトークンを作って、クッキーで渡す（前のトークンは使わない。セッション固定の攻撃を防ぐ）
    let s = session::start(cx).await?;
    sqlx::query("INSERT INTO sessions (token_hash, employee_id, expires_at) VALUES ($1, $2, $3)")
        .bind(&s.token_hash[..])
        .bind(employee_id)
        .bind(OffsetDateTime::from(s.expires_at))
        .execute(pool)
        .await?;
    // ついでに、期限の切れた記録を掃除する
    sqlx::query("DELETE FROM sessions WHERE expires_at < now()").execute(pool).await?;
    Ok(true)
}

/// ログアウト。ブラウザのクッキーを消し、データベースの記録も消す
pub async fn logout(cx: &Cx) -> topcoat::Result<()> {
    if let Some(hash) = session::stop(cx).await? {
        sqlx::query("DELETE FROM sessions WHERE token_hash = $1").bind(&hash[..]).execute(&state(cx).pool).await?;
    }
    Ok(())
}

// ---------- 今だれがログインしているか ----------

/// 今ログインしている社員。いなければ None。
/// `#[memoize]` で、1 回のリクエストの中では、何回呼んでもデータベースを 1 回しか見ない
#[memoize(as_ref)]
pub async fn current_user(cx: &Cx) -> topcoat::Result<Option<Employee>> {
    let Some(hash) = session::token_hash(cx).await? else {
        return Ok(None);
    };
    let user = sqlx::query_as::<_, Employee>(
        "SELECT e.id, e.employee_no, e.name, e.dept, e.job_title, e.email, e.role
           FROM sessions s JOIN employees e ON e.id = s.employee_id
          WHERE s.token_hash = $1 AND s.expires_at > now() AND e.is_active",
    )
    .bind(&hash[..])
    .fetch_optional(&state(cx).pool)
    .await?;
    Ok(user)
}

/// ログインが要るページの、最初に呼ぶ。ログインしていなければ、ログインの画面へ移す。
/// ログインしていれば、期限を 60 分先へ延ばす（クッキーとデータベースの両方）
pub async fn require_user(cx: &Cx) -> topcoat::Result<Employee> {
    let Some(user) = current_user(cx).await?.clone() else {
        return Err(see_other(href!(crate::app::login::login_page).resolve(cx)).into());
    };
    if let Some(s) = session::refresh(cx).await? {
        sqlx::query("UPDATE sessions SET expires_at = $2 WHERE token_hash = $1")
            .bind(&s.token_hash[..])
            .bind(OffsetDateTime::from(s.expires_at))
            .execute(&state(cx).pool)
            .await?;
    }
    Ok(user)
}
```

▶ ここで初めて出てくるコード

- **`move`**（関数）：クロージャが使う外の変数を、借りるのではなく「持っていく」印。`thread::spawn(move || { … })` のように、別のスレッドに値を渡す時に必要です。
- **`await`**（非同期）：`async` の関数（Future）が終わるのを待って、結果を受け取る書き方。`sleep(…).await` のように、後ろに `.await` を付けます。待っている間は、ほかの仕事に順番をゆずります。`.await` を付けないと、その仕事は始まりません。
- **`#[memoize]`**（Topcoat）：Topcoat の印（属性）。付けた関数の結果を、**1 つの要求（リクエスト）の間だけ**覚えておきます。レイアウト・ページ・部品のどこから呼んでも、データベースを 1 回しか読まずに済みます。
- **`.token_store()`**（Topcoat）：セッションの札（トークン）を、どこに置くかを決めるメソッド。ここでは、HTTP のページでもログインが続くように、自分で用意した `HttpCookieTokenStore`（クッキーに置く係）を渡しています。
- **`.lifetime()`**（Topcoat）：セッションの長さ（何も操作しないと切れるまでの時間）を決めるメソッド。`.lifetime(IDLE)`（`IDLE` は 60 分）。
- **`.expect()`**（エラー処理）：`Option` や `Result` から中身を取り出すメソッド。中身が無い（失敗した）時は、かっこの中の文を表示して、その場でプログラムが止まります（パニック）。`unwrap()` と同じですが、止まった理由が分かる文を付けられるので、`unwrap()` よりこちらがおすすめです。
- **`.as_bytes()`**（文字列）：文字列を、中のバイト（コンピューターが持っている 0〜255 の数の並び）として見るメソッド。ネットやファイルに書く関数がバイトを求める時に使います。
- **`.is_ok()`**（エラー処理）：`Result` が `Ok`（成功）なら `true` を返すメソッド。中身は要らず、成功したかだけ知りたい時に使います。
- **`.bind()`**（データベース）：SQL の `$1`・`$2` … に入れる値を、順に渡すメソッド。値は SQL の文とは別に送られるので、どんな文字が入っていても SQL として読まれません（SQL インジェクションを防ぐ）。
- **`.fetch_optional()`**（データベース）：SQL を送って、結果を 0 行か 1 行で受け取るメソッド。行が無ければ `None` です（`Option` で返る）。
- **`.execute()`**（データベース）：値を入れたり変えたりする SQL の文を、1 つ実行するメソッド（libsql）。`conn.execute("UPDATE … WHERE id = ?1", params![id]).await?` のように、値は `params!` で渡します。変わった行の数が返ります。
- **`Duration::from_secs()`**（時間）：秒から、時間の長さ（`Duration`）を作る関数。`Duration::from_secs(10)` は 10 秒。
- **`SessionConfig::builder()`**（Topcoat）：Topcoat のセッションの設定を、少しずつ決めて組み立て始める関数。`.token_store(…)`・`.lifetime(…)` をつなげて、最後に `.build()` で出来上がりです。
- **`tokio::task::spawn_blocking()`**（非同期）：時間のかかる重い計算や、待ってしまう処理（パスワードのハッシュなど）を、非同期の仕事を回すスレッドとは別のスレッドで動かす関数。ほかの仕事を止めないために使います。
- **`Argon2::default()`**（認証）：パスワードを安全に保存するための計算（Argon2id）を、おすすめの設定で用意する書き方（`argon2` の部品）。この計算は、わざと重く作られていて、パスワードを総当たりで当てるのを難しくします。

- **`PasswordHash::new()`**（認証）: 保存してあるハッシュの文字列を読み取って、確かめに使える形にする関数。形が違えばエラーです。
- **`LazyLock::new()`**（スレッドと共有）: 初めて使われた時に 1 回だけ作る値を、作り方の関数と一緒に用意する関数。
- **`sqlx::query_as()`**（データベース）: `sqlx` で、結果の行を、決めた型（ `FromRow` の struct や、`(i64, String)` のような組）で受け取る SQL を用意する関数。
- **`session::start()`**（Topcoat）: Topcoat のセッションを新しく始める（ログインする）関数。ログインのたびに新しい札を作るので、前から知られている札を使わせない（セッション固定の攻撃を防ぐ）ことができます。
- **`sqlx::query()`**（データベース）: `sqlx` で、SQL の文を用意する関数。`sqlx::query("UPDATE … WHERE id = $1").bind(id).execute(pool).await?` のように、`$1`・`$2` に入れる値を `.bind()` で渡して、`.execute()`・`.fetch_…()` で送ります。
- **`OffsetDateTime::from()`**（時間）: ほかの形の日時（ `SystemTime` など）から、`OffsetDateTime` を作る関数。Topcoat のセッションの期限（ `SystemTime` ）を、データベースに入れる形に変えています。
- **`session::stop()`**（Topcoat）: Topcoat のセッションを終わらせる（ログアウトする）関数。消した札のハッシュを返すので、データベースの記録も消せます。
- **`session::token_hash()`**（Topcoat）: Topcoat の関数。今のリクエストのセッションの印（トークン）の、ハッシュを返します。データベースの `sessions` 表に、ハッシュで記録しておき、ログイン中かを調べるのに使います。
- **`session::refresh()`**（Topcoat）: 今のセッションの期限を、先へ延ばす関数。使うたびに呼ぶと、「何も操作しないと 60 分で切れる」動きになります。セッションが無ければ `None` です。
- **`LazyLock`**（スレッドと共有）: 初めて使われた時に 1 回だけ作られ、その後はずっと同じ値を使う入れ物。`static DUMMY: LazyLock<String> = LazyLock::new(|| { … });` のように、作るのに手間がかかる物を、プログラム全体で共有する時に使います。
- **`Duration`**（時間）: 時間の長さを表す型。`Duration::from_secs(5)` で 5 秒。`{:?}` で表示すると `5s` や `200ms` のように単位つきで出ます。
- **`Argon2`**（認証）: パスワードをハッシュにする方法の 1 つ（Argon2）を使うための型。`Argon2::default()` で、おすすめの設定のものを作ります。わざと時間とメモリを使うので、総当たりで破られにくくなります。
- **`PasswordHasher`**（認証）: パスワードからハッシュ（保存用の値）を作る機能（トレイト。`password_hash` の部品）。`argon2.hash_password(…)` を使うには、これを `use` します。
- **`PasswordVerifier`**（認証）: 入力されたパスワードが、保存してあるハッシュと合うかを確かめる機能（トレイト）。`argon2.verify_password(…)` を使うには、これを `use` します。
- **`PasswordHash`**（認証）: 保存してあるハッシュの文字列（ `$argon2id$v=19$…` ）を読み取った物の型。`PasswordHash::new(&保存した文字列)` で作り、確かめに使います。
- **`SessionConfig`**（Topcoat）: Topcoat のセッションの設定（クッキーの名前・期限など）の型。
- **`HttpCookieTokenStore`**（練習アプリで作る物（Rust））: セッションの札を、`Secure` の付かないクッキーに置く係（ `src/http_cookie.rs` ）。Topcoat の決まりのクッキーは HTTPS のページでしか送られないので、社内の HTTP のページでもログインが続くように、onprem の 04 で確かめた物をそのまま使います。

- `SELECT`（SQL）：表から行を取り出す SQL の命令。`SELECT 列, … FROM 表 WHERE 条件 ORDER BY 列` の形でよく使います。`SELECT *` は全部の列です。
- `FROM`（SQL）：`SELECT … FROM 表` の一部で、「どの表から」取り出すかを書きます。
- `JOIN`（SQL）：2 つの表を、条件で横につないで取り出す書き方。`FROM notes JOIN users ON users.id = notes.user_id` は、ノートと、その持ち主を 1 行に並べます。
- `WHERE`（SQL）：`SELECT … FROM 表 WHERE 条件` の、条件を書く所。`WHERE user_id = 1` は「持ち主が 1 番の行だけ」。`UPDATE`・`DELETE` で `WHERE` を忘れると、全部の行が変わる（消える）ので注意します。

決まり	どこで	理由
パスワードは Argon2 のハッシュで保存する	`hash_password`	データベースを見られても、パスワードは分からない
ハッシュの計算は `spawn_blocking` で	`hash_password`・`verify_password`	わざと重い計算なので、ほかの人のリクエストを止めない
いない社員でも、同じ重さの確かめをする	`DUMMY_HASH`	断られるまでの時間の差で、社員番号を探られない
ログインのたびに新しいトークン	`session::start`	前から知られているトークンを使わせない（セッション固定の攻撃）
データベースには、トークンの指紋だけ	`sessions.token_hash`	データベースを見られても、なりすませない
使うたびに、期限を 60 分先へ	`require_user` の `session::refresh`	何も操作しないと、60 分で切れる（06 と同じ）
ログアウトは POST だけ	`app/logout.rs`（後編）	リンクを踏ませるだけで、ログアウトさせられない

- `#[memoize(as_ref)]` を付けた `current_user` は、**1 回のリクエストの中では、何回呼んでもデータベースを 1 回しか見ません**（外枠のメニューと、ページの両方が呼ぶため）。
- 実際にログインしたときの返事（`curl` で確かめた。トークンの所は伏せた）：

コピー

```
HTTP/1.1 303 See Other
location: /
set-cookie: session=（トークン）; HttpOnly; SameSite=Lax; Path=/; Max-Age=3600
```

確かめること: `Secure` が付いていない（HTTP で使うため）。`HttpOnly`（JavaScript から読めない）・`SameSite=Lax`・`Max-Age=3600`（60 分）が付いている。

> **用語:** セッション（ログインしている間、「この人はだれか」を覚えておく仕組み）。トークン（推測できない、長い乱数の文字列）

6. 状態を変える処理（workflow.rs）

いちばん大事なファイルです。06 の `services.py` と同じ関数を、同じ順に並べました。

`src/workflow.rs`：

コピー

```
//! 依頼の状態を変える処理を、ここに集める（06 の Django 版の services.py と同じ動き）
//!
//! - 画面からは、ここの関数だけを呼ぶ。状態を直接書きかえない。
//! - どの関数も 1 つのトランザクションの中で動き、途中で失敗したら全部取り消す（`?` で抜けると、tx が捨てられて取り消しになる）。
//! - 同じ依頼を 2 人が同時に操作しても 1 回だけ進むように、行に鍵をかけてから（`FOR UPDATE`）読む。
//! - メールは「送る物の箱」に入れておき、`commit` が成功してから送る（mail.rs）。

use std::fmt;

use sha2::{Digest, Sha256};
use sqlx::{PgConnection, Postgres, Transaction, types::Json};
use time::{Date, OffsetDateTime};
use topcoat::context::Cx;

use crate::{
    config::Config,
    drawing::Drawing,
    mail::Outbox,
    models::{Decision, Employee, JST, Role, Status, fmt_date},
    state::state,
    upload::{self, SavedFile, UploadedFile},
};

/// 承認の URL が使える日数
pub const TOKEN_DAYS: i32 = 14;
/// 回覧者の人数の上限
pub const MAX_APPROVERS: usize = 3;

// ---------- エラー ----------

/// できない操作（Rule）と、思いがけない失敗（Internal）を分ける。
/// Rule の文は、そのまま画面に出してよい。Internal は画面には「エラー」とだけ出し、中身はログに書く
#[derive(Debug)]
pub enum WorkflowError {
    Rule(String),
    Internal(String),
}

impl fmt::Display for WorkflowError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            WorkflowError::Rule(m) => write!(f, "{m}"),
            WorkflowError::Internal(m) => write!(f, "内部のエラー: {m}"),
        }
    }
}

impl std::error::Error for WorkflowError {}

impl From<sqlx::Error> for WorkflowError {
    fn from(e: sqlx::Error) -> Self {
        WorkflowError::Internal(e.to_string())
    }
}

impl From<std::io::Error> for WorkflowError {
    fn from(e: std::io::Error) -> Self {
        WorkflowError::Internal(e.to_string())
    }
}

pub type WfResult<T> = Result<T, WorkflowError>;

fn rule<T>(message: impl Into<String>) -> WfResult<T> {
    Err(WorkflowError::Rule(message.into()))
}

// ---------- 入力 ----------

/// 依頼を出す・出し直すときに、画面から受け取る物（形はもう確かめてある）
#[derive(Debug, Clone)]
pub struct RequestInput {
    pub title: String,
    pub detail: String,
    pub due_date: Date,
    pub approver_ids: Vec<i64>,
    pub drawing: Option<Drawing>,
    pub files: Vec<UploadedFile>,
}

// ---------- 承認の URL ----------
```

```rust
/// URL に入れる、推測できない文字列（32 バイトの乱数を 16 進で 64 文字）
fn new_token() -> String {
    let mut bytes = [0u8; 32];
    getrandom::fill(&mut bytes).expect("乱数を作れません");
    hex::encode(bytes)
}

/// 文字列の指紋（SHA-256）。データベースには、これだけを入れる
pub fn hash_token(token: &str) -> String {
    hex::encode(Sha256::digest(token.as_bytes()))
}

/// URL で開いた回覧の 1 人分
#[derive(Debug, Clone, sqlx::FromRow)]
pub struct TokenStep {
    pub step_id: i64,
    pub request_id: i64,
    pub step_order: i16,
    #[sqlx(try_from = "String")]
    pub decision: Decision,
    pub token_expires_at: Option<OffsetDateTime>,
    pub approver_id: i64,
    pub approver_name: String,
    #[sqlx(try_from = "String")]
    pub request_status: Status,
}

/// URL の文字列を調べる。使えなければ、理由の文を Rule で返す。
/// `lock` が true なら、回覧と依頼の行に鍵をかける（承認・差し戻しのとき）
pub async fn check_token(conn: &mut PgConnection, token: &str, lock: bool) -> WfResult<TokenStep> {
    let sql = format!(
        "SELECT s.id AS step_id, s.request_id, s.step_order, s.decision, s.token_expires_at,
                s.approver_id, a.name AS approver_name, r.status AS request_status
           FROM steps s
           JOIN requests r ON r.id = s.request_id
           JOIN employees a ON a.id = s.approver_id
          WHERE s.token_hash = $1 {}",
        if lock { "FOR UPDATE OF s, r" } else { "" }
    );
    // sqlx 0.9 は、組み立てた SQL の文字を、そのままでは受け取らない（SQL インジェクションを防ぐため）。
    // ここで足すのは決まった文字（FOR UPDATE ...）だけで、外から来た値は $1 で渡しているので、
    // 「確かめた」という印の AssertSqlSafe で包む
    let step: Option<TokenStep> =
        sqlx::query_as(sqlx::AssertSqlSafe(sql)).bind(hash_token(token)).fetch_optional(&mut *conn).await?;
    let Some(step) = step else {
        return rule("この URL は使えません。");
    };
    match step.decision {
        Decision::Approved => return rule("この依頼は、承認済みです。"),
        Decision::Returned => return rule("この依頼は、差し戻し済みです。"),
        Decision::Cancelled if step.request_status == Status::PulledBack => return rule("依頼者により引き戻されました。"),
        Decision::Cancelled => return rule("この URL は、取り消されました。"),
        Decision::Waiting => {}
    }
    if step.token_expires_at.is_some_and(|t| t < OffsetDateTime::now_utc()) {
        return rule("この URL は、期限が切れています。管理者に再送を頼んでください。");
    }
    Ok(step)
}

// ---------- 小さな部品 ----------

type Tx = Transaction<'static, Postgres>;

/// 鍵をかけて読んだ依頼
#[derive(Debug, sqlx::FromRow)]
struct Locked {
    id: i64,
    title: String,
    #[sqlx(try_from = "String")]
    status: Status,
    requester_id: i64,
    assignee_id: Option<i64>,
    round: i16,
    return_comments: String,
}

async fn lock_request(tx: &mut Tx, id: i64) -> WfResult<Locked> {
    let r: Option<Locked> = sqlx::query_as(
        "SELECT id, title, status, requester_id, assignee_id, round, return_comments
           FROM requests WHERE id = $1 FOR UPDATE",
    )
    .bind(id)
    .fetch_optional(&mut **tx)
    .await?;
```

```
    match r {
        Some(r) => Ok(r),
        None => rule("依頼がありません。"),
    }
}

/// 自分の依頼だけを、鍵をかけて読む
async fn lock_own(tx: &mut Tx, user: &Employee, id: i64) -> WfResult<Locked> {
    let r = lock_request(tx, id).await?;
    if r.requester_id != user.id {
        return rule("自分の依頼ではありません。");
    }
    Ok(r)
}

/// 記録（だれが・何をした）を 1 行足す
async fn log(tx: &mut Tx, request_id: i64, actor_id: Option<i64>, action: &str, note: &str) -> WfResult<()> {
    sqlx::query("INSERT INTO histories (request_id, actor_id, action, note) VALUES ($1, $2, $3, $4)")
        .bind(request_id)
        .bind(actor_id)
        .bind(action)
        .bind(note)
        .execute(&mut **tx)
        .await?;
    Ok(())
}

/// 回覧者の決まり（1〜3 人・同じ人を 2 回は選べない・自分は選べない・いる社員だけ）
async fn check_approvers(tx: &mut Tx, requester_id: i64, ids: &[i64]) -> WfResult<()> {
    if ids.is_empty() || ids.len() > MAX_APPROVERS {
        return rule(format!("回覧者は 1〜{MAX_APPROVERS} 人にしてください。"));
    }
    let mut unique = ids.to_vec();
    unique.sort();
    unique.dedup();
    if unique.len() != ids.len() {
        return rule("同じ人を 2 回は選べません。");
    }
    if ids.contains(&requester_id) {
        return rule("自分自身は回覧者にできません。");
    }
    let found: i64 = sqlx::query_scalar("SELECT count(*) FROM employees WHERE id = ANY($1) AND is_active")
        .bind(ids)
        .fetch_one(&mut **tx)
        .await?;
    if found != ids.len() as i64 {
        return rule("回覧者に、いない社員が入っています。");
    }
    Ok(())
}

/// 承認のお願いのメールに書く物
#[derive(sqlx::FromRow)]
struct MailInfo {
    approver_name: String,
    approver_email: String,
    step_order: i16,
    title: String,
    requester_name: String,
    requester_dept: String,
    due_date: Date,
}

/// 新しい URL を作り（前の URL は使えなくなる）、承認のお願いのメールを箱に入れる
async fn send_approval_mail(tx: &mut Tx, outbox: &mut Outbox, config: &Config, step_id: i64) -> WfResult<()> {
    let token = new_token();
    sqlx::query(
        "UPDATE steps SET token_hash = $2, token_expires_at = now() + make_interval(days => $3), sent_at = now()
          WHERE id = $1",
    )
    .bind(step_id)
    .bind(hash_token(&token))
    .bind(TOKEN_DAYS)
    .execute(&mut **tx)
    .await?;

    let m: MailInfo = sqlx::query_as(
        "SELECT a.name AS approver_name, a.email AS approver_email, s.step_order,
                r.title, q.name AS requester_name, q.dept AS requester_dept, r.due_date
           FROM steps s
           JOIN employees a ON a.id = s.approver_id
           JOIN requests r ON r.id = s.request_id
           JOIN employees q ON q.id = r.requester_id
          WHERE s.id = $1",
    )
```

```
    .bind(step_id)
    .fetch_one(&mut **tx)
    .await?;

    let url = format!("{}/approve/{token}", config.base_url);
    let body = format!(
        "{} さん\n\n次の依頼の、承認をお願いします。（回覧者{}）\n\n件名: {}\n依頼者: {}（{}）\n希望の納期: {}\n\n\
         下の URL を開いて、「承認」か「差し戻し」を選んでください。\n{url}\n\n\
         ・この URL は、あなただけのものです。ほかの人に転送しないでください。\n\
         ・この URL は 1 回だけ使えます。{TOKEN_DAYS} 日で使えなくなります。\n",
        m.approver_name, m.step_order, m.title, m.requester_name, m.requester_dept, fmt_date(m.due_date),
    );
    outbox.add(&m.approver_email, &format!("承認のお願い: {}", m.title), body);
    Ok(())
}

/// 回覧を始める（回覧者を並べて、1 人目に承認のお願い）
async fn start_round(
    tx: &mut Tx,
    outbox: &mut Outbox,
    config: &Config,
    request_id: i64,
    round: i16,
    approver_ids: &[i64],
) -> WfResult<()> {
    let mut first = None;
    for (i, approver_id) in approver_ids.iter().enumerate() {
        let step_id: i64 = sqlx::query_scalar(
            "INSERT INTO steps (request_id, round, step_order, approver_id) VALUES ($1, $2, $3, $4) RETURNING id",
        )
        .bind(request_id)
        .bind(round)
        .bind(i as i16 + 1)
        .bind(approver_id)
        .fetch_one(&mut **tx)
        .await?;
        first.get_or_insert(step_id);
    }
    if let Some(step_id) = first {
        send_approval_mail(tx, outbox, config, step_id).await?;
    }
    Ok(())
}

/// 添付ファイルをフォルダに書き、表に 1 行ずつ足す。書いたファイルは `saved` に入れる（失敗したときに消すため）
async fn add_files(tx: &mut Tx, config: &Config, request_id: i64, files: &[UploadedFile], saved: &mut Vec<SavedFile>) -> WfResult<()> {
    for file in files {
        let f = upload::save(&config.file_dir, file).await?;
        saved.push(f.clone());
        sqlx::query(
            "INSERT INTO attachments (request_id, stored_name, original_name, content_type, size) VALUES ($1, $2, $3, $4, $5)",
        )
        .bind(request_id)
        .bind(&f.stored_name)
        .bind(&f.original_name)
        .bind(f.content_type)
        .bind(f.size as i32)
        .execute(&mut **tx)
        .await?;
    }
    Ok(())
}

/// 保存が確定したら送る。失敗したら、書いたファイルを消す（データベースは tx を捨てて取り消し）
async fn finish<T>(cx: &Cx, result: WfResult<(T, Tx, Outbox)>, saved: &[SavedFile]) -> WfResult<T> {
    let committed = match result {
        Ok((value, tx, outbox)) => tx.commit().await.map(|_| (value, outbox)).map_err(WorkflowError::from),
        Err(e) => Err(e),
    };
    match committed {
        Ok((value, outbox)) => {
            outbox.send_all(cx).await; // ここで初めて送る
            Ok(value)
        }
        Err(e) => {
            upload::remove(&state(cx).config.file_dir, saved).await;
            Err(e)
        }
    }
}

// ---------- 状態を変える操作 ----------

/// 依頼を出す → 回覧中。1 人目に承認のお願いのメール
pub async fn submit(cx: &Cx, requester: &Employee, input: RequestInput) -> WfResult<i64> {
```

```
    let st = state(cx);
    let mut saved = Vec::new();
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        check_approvers(&mut tx, requester.id, &input.approver_ids).await?;
        let id: i64 = sqlx::query_scalar(
            "INSERT INTO requests (title, detail, due_date, requester_id, drawing) VALUES ($1, $2, $3, $4, $5) RETURNING id",
        )
        .bind(&input.title)
        .bind(&input.detail)
        .bind(input.due_date)
        .bind(requester.id)
        .bind(input.drawing.as_ref().map(Json))
        .fetch_one(&mut *tx)
        .await?;
        start_round(&mut tx, &mut outbox, &st.config, id, 1, &input.approver_ids).await?;
        add_files(&mut tx, &st.config, id, &input.files, &mut saved).await?;
        log(&mut tx, id, Some(requester.id), "依頼を出した", "").await?;
        Ok((id, tx, outbox))
    }
    .await;
    finish(cx, result, &saved).await
}

/// 承認 → 次の人へ。最後の人なら回覧完了（依頼者と、担当の部署へメール）。依頼の件名を返す
pub async fn approve(cx: &Cx, token: &str) -> WfResult<String> {
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        let step = check_token(&mut tx, token, true).await?; // 鍵をかけて読む。同時に押されても 1 回だけ進む
        let r = lock_request(&mut tx, step.request_id).await?;
        // 指紋は消さずに残す。同じ URL をもう一度開いた人に「承認済みです」と知らせるため
        sqlx::query("UPDATE steps SET decision = 'approved', decided_at = now() WHERE id = $1")
            .bind(step.step_id)
            .execute(&mut *tx)
            .await?;
        log(&mut tx, r.id, Some(step.approver_id), &format!("承認（回覧者{}）", step.step_order), "").await?;

        let next: Option<i64> = sqlx::query_scalar(
            "SELECT id FROM steps WHERE request_id = $1 AND round = $2 AND decision = 'waiting' ORDER BY step_order LIMIT 1",
        )
        .bind(r.id)
        .bind(r.round)
        .fetch_optional(&mut *tx)
        .await?;

        if let Some(next_id) = next {
            send_approval_mail(&mut tx, &mut outbox, &st.config, next_id).await?;
        } else {
            sqlx::query("UPDATE requests SET status = 'approved', updated_at = now() WHERE id = $1")
                .bind(r.id)
                .execute(&mut *tx)
                .await?;
            let (name, email): (String, String) = sqlx::query_as("SELECT name, email FROM employees WHERE id = $1")
                .bind(r.requester_id)
                .fetch_one(&mut *tx)
                .await?;
            outbox.add(
                &email,
                &format!("回覧完了: {}", r.title),
                format!(
                    "{name} さん\n\n依頼「{}」は、全員の承認が終わりました（回覧完了）。\n\
                     担当の部署と、打ち合わせの日を決めてください。\n",
                    r.title
                ),
            );
            let workers: Vec<String> = sqlx::query_scalar("SELECT email FROM employees WHERE role = 'worker' AND is_active")
                .fetch_all(&mut *tx)
                .await?;
            for to in workers {
                outbox.add(
                    &to,
                    &format!("新しい依頼: {}", r.title),
                    format!("新しい依頼が届きました。\n\n件名: {}\n\nアプリの「依頼の一覧」から、担当を決めてください。\n", r.title),
                );
            }
        }
        Ok((r.title, tx, outbox))
    }
    .await;
    finish(cx, result, &[]).await
}
```

```rust
/// 差し戻し → 差し戻し中。コメントは【日時 回覧者n 氏名】を付けて追記する。依頼の件名を返す
pub async fn return_request(cx: &Cx, token: &str, comment: &str) -> WfResult<String> {
    let comment = comment.trim();
    if comment.is_empty() {
        return rule("差し戻すときは、理由を書いてください。");
    }
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        let step = check_token(&mut tx, token, true).await?;
        let r = lock_request(&mut tx, step.request_id).await?;
        sqlx::query("UPDATE steps SET decision = 'returned', decided_at = now() WHERE id = $1")
            .bind(step.step_id)
            .execute(&mut *tx)
            .await?;
        // まだ回ってきていない人の分は、取り消し（その人の URL は使えなくなる）
        sqlx::query("UPDATE steps SET decision = 'cancelled' WHERE request_id = $1 AND round = $2 AND decision = 'waiting'")
            .bind(r.id)
            .bind(r.round)
            .execute(&mut *tx)
            .await?;

        let now = OffsetDateTime::now_utc().to_offset(JST);
        let line = format!(
            "【{:02}/{:02} {:02}:{:02} 回覧者{} {}】{comment}",
            u8::from(now.month()), now.day(), now.hour(), now.minute(), step.step_order, step.approver_name
        );
        let comments = if r.return_comments.is_empty() { line.clone() } else { format!("{}\n{line}", r.return_comments) };
        sqlx::query("UPDATE requests SET status = 'returned', return_comments = $2, updated_at = now() WHERE id = $1")
            .bind(r.id)
            .bind(&comments)
            .execute(&mut *tx)
            .await?;
        log(&mut tx, r.id, Some(step.approver_id), &format!("差し戻し（回覧者{}）", step.step_order), comment).await?;

        let (name, email): (String, String) = sqlx::query_as("SELECT name, email FROM employees WHERE id = $1")
            .bind(r.requester_id)
            .fetch_one(&mut *tx)
            .await?;
        outbox.add(
            &email,
            &format!("差し戻し: {}", r.title),
            format!(
                "{name} さん\n\n依頼「{}」が、差し戻されました。\n\n{line}\n\n\
                 アプリで内容を直して、出し直してください。\n",
                r.title
            ),
        );
        Ok((r.title, tx, outbox))
    }
    .await;
    finish(cx, result, &[]).await
}

/// 引き戻し（回覧中だけ）→ 引き戻し中。承認待ちの人の URL は使えなくなる
pub async fn pull_back(cx: &Cx, user: &Employee, request_id: i64) -> WfResult<()> {
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let r = lock_own(&mut tx, user, request_id).await?;
        if r.status != Status::Circulating {
            return rule(format!("「{}」の依頼は引き戻せません。", r.status.label()));
        }
        sqlx::query("UPDATE steps SET decision = 'cancelled' WHERE request_id = $1 AND round = $2 AND decision = 'waiting'")
            .bind(r.id)
            .bind(r.round)
            .execute(&mut *tx)
            .await?;
        set_status(&mut tx, r.id, Status::PulledBack).await?;
        log(&mut tx, r.id, Some(user.id), "引き戻した", "").await?;
        Ok(((), tx, Outbox::new()))
    }
    .await;
    finish(cx, result, &[]).await
}

/// 直して出し直す → 回覧中。最初の回覧者からやり直す（回覧の回数を 1 増やす）
pub async fn resubmit(cx: &Cx, user: &Employee, request_id: i64, input: RequestInput) -> WfResult<()> {
    let st = state(cx);
    let mut saved = Vec::new();
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        let r = lock_own(&mut tx, user, request_id).await?;
```

```
        if !matches!(r.status, Status::Returned | Status::PulledBack) {
            return rule(format!("「{}」の依頼は出し直せません。", r.status.label()));
        }
        check_approvers(&mut tx, user.id, &input.approver_ids).await?;
        let round = r.round + 1;
        sqlx::query(
            "UPDATE requests SET title = $2, detail = $3, due_date = $4, drawing = $5, round = $6,
                    status = 'circulating', updated_at = now()
              WHERE id = $1",
        )
        .bind(r.id)
        .bind(&input.title)
        .bind(&input.detail)
        .bind(input.due_date)
        .bind(input.drawing.as_ref().map(Json))
        .bind(round)
        .execute(&mut *tx)
        .await?;
        start_round(&mut tx, &mut outbox, &st.config, r.id, round, &input.approver_ids).await?;
        add_files(&mut tx, &st.config, r.id, &input.files, &mut saved).await?;
        log(&mut tx, r.id, Some(user.id), &format!("出し直した（{round} 回目）"), "").await?;
        Ok(((), tx, outbox))
    }
    .await;
    finish(cx, result, &saved).await
}

/// 取り下げ（差し戻し中・引き戻し中だけ）
pub async fn withdraw(cx: &Cx, user: &Employee, request_id: i64) -> WfResult<()> {
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let r = lock_own(&mut tx, user, request_id).await?;
        if !matches!(r.status, Status::Returned | Status::PulledBack) {
            return rule(format!("「{}」の依頼は取り下げられません。", r.status.label()));
        }
        set_status(&mut tx, r.id, Status::Withdrawn).await?;
        log(&mut tx, r.id, Some(user.id), "取り下げた", "").await?;
        Ok(((), tx, Outbox::new()))
    }
    .await;
    finish(cx, result, &[]).await
}

/// 担当を決める（回覧完了 → 対応中）
pub async fn assign(cx: &Cx, worker: &Employee, request_id: i64) -> WfResult<()> {
    if !worker.role.is_staff() {
        return rule("担当者だけができます。");
    }
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let r = lock_request(&mut tx, request_id).await?;
        if r.status != Status::Approved {
            return rule(format!("「{}」の依頼は、担当を決められません。", r.status.label()));
        }
        sqlx::query("UPDATE requests SET assignee_id = $2, status = 'in_progress', updated_at = now() WHERE id = $1")
            .bind(r.id)
            .bind(worker.id)
            .execute(&mut *tx)
            .await?;
        log(&mut tx, r.id, Some(worker.id), "担当になった", "").await?;
        Ok(((), tx, Outbox::new()))
    }
    .await;
    finish(cx, result, &[]).await
}

/// 引き渡した（対応中 → 完了）。依頼者にメール
pub async fn complete(cx: &Cx, worker: &Employee, request_id: i64) -> WfResult<()> {
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        let r = lock_request(&mut tx, request_id).await?;
        if r.status != Status::InProgress || r.assignee_id != Some(worker.id) {
            return rule("担当している、対応中の依頼だけを完了にできます。");
        }
        set_status(&mut tx, r.id, Status::Done).await?;
        log(&mut tx, r.id, Some(worker.id), "完了にした", "").await?;
        let (name, email): (String, String) = sqlx::query_as("SELECT name, email FROM employees WHERE id = $1")
            .bind(r.requester_id)
            .fetch_one(&mut *tx)
            .await?;
        outbox.add(
```

```
            &email,
            &format!("完了: {}", r.title),
            format!("{name} さん\n\n依頼「{}」の対応が終わりました（完了）。\n", r.title),
        );
        Ok(((), tx, outbox))
    }
    .await;
    finish(cx, result, &[]).await
}

/// 承認のお願いを送り直す（前の URL は使えなくなる）。管理者の画面と、毎朝の処理で使う。
/// `actor` は、送り直した人（毎朝の処理なら None）
pub async fn resend(cx: &Cx, actor: Option<&Employee>, step_id: i64) -> WfResult<()> {
    let st = state(cx);
    let result = async {
        let mut tx = st.pool.begin().await?;
        let mut outbox = Outbox::new();
        let row: Option<(i64, i16, String, String)> = sqlx::query_as(
            "SELECT s.request_id, s.step_order, s.decision, r.status
               FROM steps s JOIN requests r ON r.id = s.request_id
              WHERE s.id = $1 FOR UPDATE OF s, r",
        )
        .bind(step_id)
        .fetch_optional(&mut *tx)
        .await?;
        let Some((request_id, order, decision, status)) = row else {
            return rule("回覧がありません。");
        };
        if decision != Decision::Waiting.as_str() || status != Status::Circulating.as_str() {
            return rule("承認を待っている回覧だけを送り直せます。");
        }
        send_approval_mail(&mut tx, &mut outbox, &st.config, step_id).await?;
        log(&mut tx, request_id, actor.map(|a| a.id), &format!("承認のお願いを送り直した（回覧者{order}）"), "").await?;
        Ok(((), tx, outbox))
    }
    .await;
    finish(cx, result, &[]).await
}

async fn set_status(tx: &mut Tx, request_id: i64, status: Status) -> WfResult<()> {
    sqlx::query("UPDATE requests SET status = $2, updated_at = now() WHERE id = $1")
        .bind(request_id)
        .bind(status.as_str())
        .execute(&mut **tx)
        .await?;
    Ok(())
}

/// 社員の役割を変える（サーバーの上のコマンドで使う）
pub async fn grant_role(pool: &sqlx::PgPool, employee_no: &str, role: Role) -> WfResult<()> {
    let n = sqlx::query("UPDATE employees SET role = $2 WHERE employee_no = $1")
        .bind(employee_no)
        .bind(role.as_str())
        .execute(pool)
        .await?
        .rows_affected();
    if n == 0 {
        return rule(format!("従業員番号 {employee_no} の社員がいません。"));
    }
    Ok(())
}
```

▶ ここで初めて出てくるコード

- **`as`**（数）：型を変える書き方。`x as f64` は整数を小数に、`300 as u8` は 300 を u8 に変えます。入りきらない分は**黙って切り捨てられる**ので（300 → 44）、範囲が分からない時は `try_from` を使います。
- **`.is_some_and()`**（Option）：`Option` に値があり、しかもその値が条件に合う時だけ `true` を返すメソッド。`cookie.is_some_and(|c| c == "admin=yes")`。
- **`.len()`**（Vec（並び））：中身の数（長さ）を返すメソッド。`Vec` なら入っている個数、`String`・`&str` ならバイト数です。日本語は 1 文字が 3 バイトなので、文字の数を数える時は `.chars().count()` を使います。
- **`.to_vec()`**（Vec（並び））：並び（スライス）の中身を写して、新しい `Vec` を作るメソッド。
- **`.sort()`**（Vec（並び））：`Vec` の中身を、小さい順（文字なら辞書の順）に並べかえるメソッド。新しい `Vec` は作らず、その `Vec` 自身を並べかえるので、`mut` が要ります。
- **`.dedup()`**（Vec（並び））：`Vec` の中で、隣どうしで同じ値が続く所を 1 つにまとめるメソッド。先に `sort` しておくと、全部の重複が消えます。
- **`.contains()`**（文字列）：含まれていれば `true` を返すメソッド。文字列なら `"report.csv".contains("2026")`、範囲なら `(1..=80).contains(&length)`（1〜80 に入っているか）、`Vec` なら `v.contains(&x)` です。
- **`.fetch_one()`**（データベース）：SQL を送って、結果を 1 行受け取るメソッド。行が無いとエラーになるので、必ず 1 行ある時（`INSERT … RETURNING id` など）に使います。
- **`.enumerate()`**（イテレーター）：並びを 1 つずつ取り出す時に、「何番目か」の番号を一緒に付けるメソッド。`for (i, x) in list.iter().enumerate()` で、`i` に 0, 1, 2…、`x` に中身が入ります。
- **`.get_or_insert()`**（Option）：`Option` が `None` の時だけ、渡した値を入れるメソッド（もう `Some` なら何もしない）。`first.get_or_insert(step_id)` は「最初の 1 つだけを覚えておく」書き方です。
- **`.commit()`**（データベース）：トランザクションの中の変更を、まとめて確定するメソッド。`tx.commit().await?`。`commit` しないまま `tx` が無くなると、変更は全部取り消されます（ロールバック）。
- **`.send_all()`**（練習アプリで作る物（Rust））：`Outbox` にためたメールを、まとめて送るメソッド。トランザクションを `commit` したあとに呼びます。
- **`.begin()`**（データベース）：トランザクション（いくつもの SQL を、全部するか全部しないかの 1 まとまりにする仕組み）を始めるメソッド。`let mut tx = pool.begin().await?;`。
- **`.as_ref()`**（所有権と借用）：中身を借りた形に変えるメソッド。`Option<String>` を `Option<&String>` にするなど、中身を移さずに覗きたい時に使います。
- **`.fetch_all()`**（データベース）：SQL を送って、結果の行を全部 `Vec` で受け取るメソッド。
- **`.month()`**（時間）：日時から「月」を取り出すメソッド。`Month::November` のような enum で返るので、数にする時は `u8::from(now.month())` とします。
- **`.day()`**（時間）：日時から「日」（1〜31）を取り出すメソッド。
- **`.hour()`**（時間）：日時から「時」（0〜23）を取り出すメソッド。
- **`.minute()`**（時間）：日時から「分」（0〜59）を取り出すメソッド。

- `.rows_affected()`（データベース）：`.execute()` の結果から、変わった行の数を返すメソッド。0 なら「条件に合う行が無かった（もう誰かが変えた）」と分かります。
- `getrandom::fill()`（認証）：OS が用意する、予想できない乱数で、バイトの並びを埋める関数。`getrandom::fill(&mut bytes)`。承認の URL の文字列（トークン）のように、当てられてはいけない値を作るのに使います。
- `hex::encode()`（文字列）：バイトの並びを、`0` 〜 `9`・`a` 〜 `f` の文字（16 進数）に変える関数。1 バイトが 2 文字になります。
- `Sha256::digest()`（認証）：データから、SHA-256 というハッシュ（32 バイトの指紋）を計算する関数（`sha2` の部品）。
- `sqlx::query_scalar()`（データベース）：`sqlx` で、結果が 1 つの値だけ（`count(*)` など）の SQL を用意する関数。`let found: i64 = sqlx::query_scalar("SELECT count(*) …")…` のように、値をそのまま受け取れます。
- `upload::save()`（練習アプリで作る物（Rust））：送られてきた添付ファイルを、ファイルを置くフォルダに保存する関数。後編で作ります。
- `upload::remove()`（練習アプリで作る物（Rust））：保存した添付ファイルを消す関数。データベースへの記録に失敗した時に、保存したファイルを片づけるのに使います。後編で作ります。
- `Outbox::new()`（練習アプリで作る物（Rust））：空の `Outbox`（送るメールの箱）を作る関数。
- `u8::from()`（数）：ほかの型の値を `u8`（0〜255 の数）に変える関数。`u8::from(now.month())` は、`Month::November` を 11 にします（必ず入りきる変換なので、`try_from` ではなく `from`）。
- `Digest`（認証）：ハッシュの計算（`digest`・`update`・`finalize`）を使えるようにする機能（トレイト。`sha2` などの部品）。
- `Sha256`（認証）：SHA-256 というハッシュの計算をする型。データの「指紋」を作るのに使います（パスワードの保存には、わざと重い Argon2 を使います）。
- `PgConnection`（データベース）：`sqlx` の、PostgreSQL への接続 1 本の型。トランザクション（`Transaction`）の中の接続も、`&mut *tx` でこの型として渡せます。
- `Postgres`（データベース）：`sqlx` で、「相手のデータベースは PostgreSQL」を表す型。`Transaction<'_, Postgres>` のように、型の中で使います。
- `Transaction`（データベース）：`sqlx` の、トランザクションの型。`pool.begin()` で始まり、`.commit()` で確定します。
- `Outbox`（練習アプリで作る物（Rust））：送るメールを、すぐには送らずに、ためておく箱（`src/mail.rs`）。データベースの変更を `commit` で確定してから、`.send_all()` でまとめて送ります。確定に失敗したのにメールだけ届く、ということを防げます。
- `SavedFile`（練習アプリで作る物（Rust））：保存した添付ファイルの情報（保存した名前・元の名前・大きさなど）をまとめた struct。後編で作ります。
- `UploadedFile`（練習アプリで作る物（Rust））：フォームから送られてきた添付ファイル（名前・種類・中身のバイト）をまとめた struct。後編で作ります。
- `From`（型とメソッド）：「ある型から作れる」機能（トレイト）。`impl From<RepoError> for AppError` を書くと、`RepoError` を `AppError` に変えられます。`?` は、エラーを返す時にこの変換を自動で使います。

- **`Into`**（型とメソッド）:「ある型に変えられる」機能（トレイト。`From` の逆向き）。`body: impl Into<String>` は「`String` に変えられる物なら何でも（`&str` でも `String` でも）受け取る」という引数で、中で `.into()` を呼んで変えます。
- **`AS`**（SQL）: SQL で、列や表に別の名前を付ける書き方。`SELECT COUNT(*) AS notes` は、結果の列の名前を `notes` にします。`FROM users AS u` は、表を `u` と短く呼べるようにします。
- **`AssertSqlSafe`**（データベース）: `sqlx` に「この組み立てた SQL の文は、決まった文字だけでできていると確かめた」と伝える包み。`sqlx::query(sqlx::AssertSqlSafe(sql))`。外から来た値は、必ず `$1` と `.bind()` で別に渡します（そうしないと SQL インジェクションの穴になる）。
- **`FOR`**（SQL）: `SELECT … FOR UPDATE` は、読んだ行に鍵をかける書き方です。同じ行を、別の人が同時に変えようとしても、こちらのトランザクションが終わるまで待たせるので、「2 人が同時に承認して 2 回進む」ことを防げます。
- **`UPDATE`**（SQL）: `UPDATE 表 SET 列 = 値 WHERE 条件` で、行の中身を書きかえる SQL の命令。`WHERE` を忘れると、全部の行が変わります。
- **`SET`**（SQL）: `UPDATE` の中で、「どの列を、どの値にするか」を書く所。
- **`OF`**（SQL）: `FOR UPDATE OF s, r` は、つないだ表（`JOIN`）のうち、鍵をかける表を名前（ここでは `s` と `r`）で選ぶ書き方です。
- **`PgPool`**（データベース）: `sqlx` の、PostgreSQL への接続の集まり（プール）の型。何本かの接続を用意しておき、頼まれるたびに貸し出します。複製（`.clone()`）しても、同じ集まりを指します。

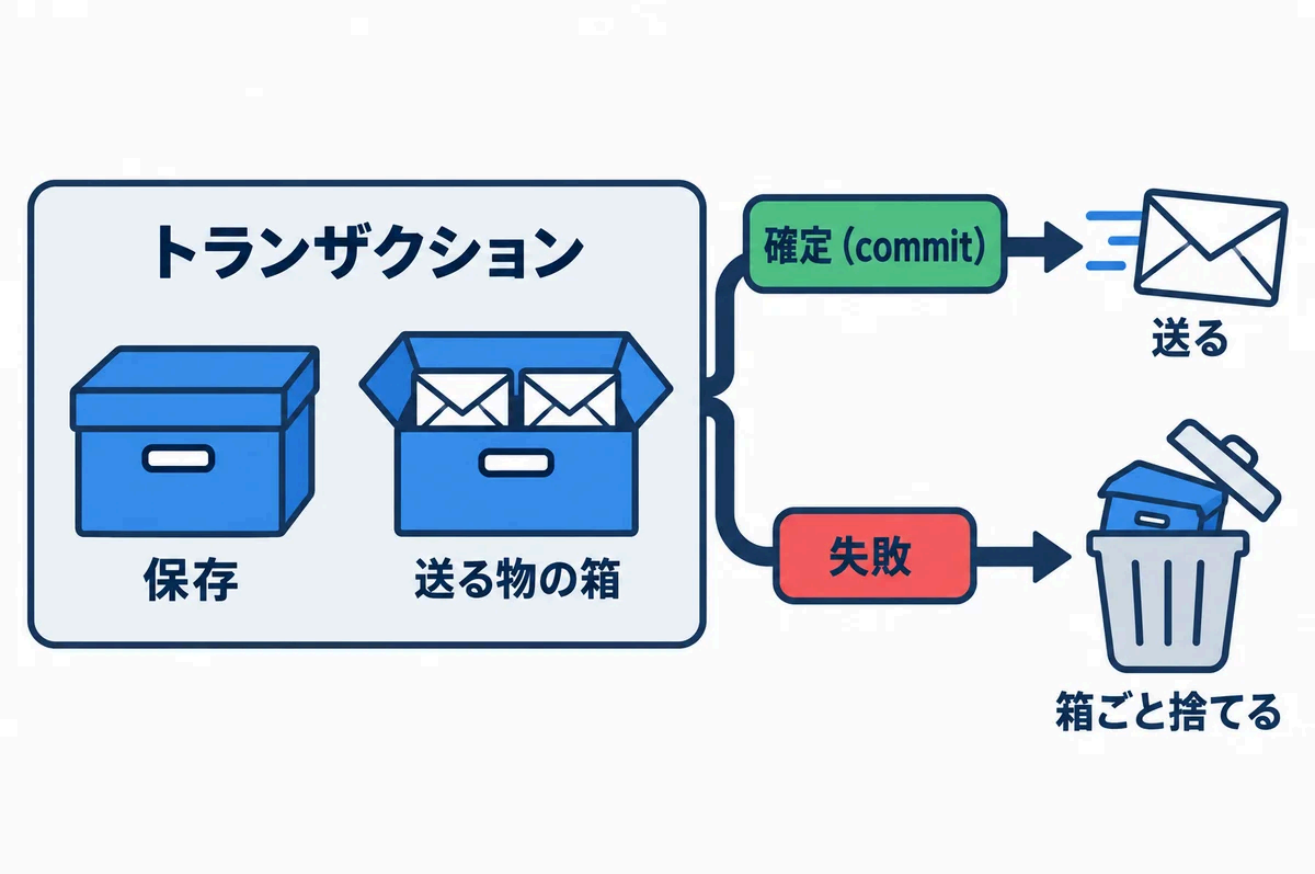
6.1 全部やるか、全部やめるか（トランザクション）

コピー

```
let mut tx = st.pool.begin().await?;   // ここから
// …… 何回か書きかえる。途中の ? でエラーになったら、tx は捨てられる
tx.commit().await?;                    // ここまでを、まとめて確定
```

- Django の `@transaction.atomic` に当たります。
- Rust では、**`commit` しないまま `tx` が捨てられると、自動で取り消し**（ROLLBACK）になります。途中の `?` でエラーを返すだけで、取り消しが起きます。取り消しを書き忘れる心配がありません。

6.2 保存が確定してから、メールを送る（送る物の箱）



`src/mail.rs`：

コピー

```
//! メールを送る。送り方（ファイル・SMTP リレー）は設定で選び、送る側のコードは変えない
//!
//! 大事な決まり:
//! 1. **データベースへの保存が確定してから送る。** 処理の途中では「送る物の箱（Outbox）」に入れておき、
//!    `commit` が成功したあとで `send_all` する。途中で失敗して取り消したら、箱ごと捨てる（送らない）。
//! 2. **検証のサーバーでは、本物の宛先に送らない。** `MAIL_REDIRECT_TO` があれば、全部そこへ送る。

use topcoat::{
    context::Cx,
    mail::{FileTransport, MailConfig, SmtpTransport, mail, send},
};

use crate::{
    config::{Config, MailSettings},
    state::state,
};

/// 件名の頭に付ける印（メールの一覧で、このアプリからのメールだと分かるように）
const SUBJECT_PREFIX: &str = "【依頼・承認】";

/// 設定から、Topcoat のメールの設定を作る（起動のときに 1 回）
pub fn mail_config(settings: &MailSettings) -> Result<MailConfig, topcoat::mail::SmtpError> {
    let builder = MailConfig::builder();
    let builder = match settings {
        // 開発のパソコン: 送らずに、フォルダに .eml のファイルとして書く（メールソフトで開ける）
        MailSettings::File(dir) => builder.transport(FileTransport::new(dir)),
        // サーバー: M365 の SMTP リレーへ。STARTTLS で暗号化してから渡す（ポートは 25）。
        // 相手が STARTTLS を使えなければ、送らずにエラーにする（暗号化しないまま送らない）
        MailSettings::Smtp { host, port, starttls: true } => {
            builder.transport(SmtpTransport::starttls(host)?.port(*port).build())
        }
        // 開発のパソコンだけ: 練習用の SMTP サーバー（09 の smtp_sink.py）へ、暗号化せずに渡す
        MailSettings::Smtp { host, port, starttls: false } => builder.transport(SmtpTransport::unencrypted(host).port(*port).build()),
    };
    Ok(builder.build())
}

/// 送る前のメール 1 通
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Outgoing {
    pub to: String,
    pub subject: String,
    pub body: String,
}

impl Outgoing {
    /// 件名の頭に【依頼・承認】を付けて作る
    pub fn new(to: &str, subject: &str, body: String) -> Outgoing {
        Outgoing { to: to.to_string(), subject: format!("{SUBJECT_PREFIX}{subject}"), body }
    }
}

/// 送る物の箱。処理の途中で入れておき、保存が確定してから、まとめて送る
#[derive(Debug, Default)]
pub struct Outbox {
    items: Vec<Outgoing>,
}

impl Outbox {
    pub fn new() -> Outbox {
        Outbox::default()
    }

    pub fn add(&mut self, to: &str, subject: &str, body: String) {
        self.items.push(Outgoing::new(to, subject, body));
    }

    pub fn items(&self) -> &[Outgoing] {
        &self.items
    }

    /// まとめて送る。1 通が失敗しても、ほかは送る（失敗はログに書く）
    ///
    /// 保存はもう確定しているので、ここでエラーを返して画面をエラーにはしない。
    /// 届かなかった承認のお願いは、管理者の「送り直す」か、毎朝の処理で送り直せる。
    pub async fn send_all(self, cx: &Cx) {
        for item in self.items {
            let (to, subject) = (item.to.clone(), item.subject.clone());
            if let Err(e) = send_now(cx, item).await {
                eprintln!("メールを送れませんでした（宛先: {to}・件名: {subject}）: {e}");
            }
        }
```

```
    }
}

/// 1 通を、今すぐ送る（検証のサーバーなら宛先を差し替えてから）。失敗したらエラーを返す
pub async fn send_now(cx: &Cx, item: Outgoing) -> topcoat::Result<()> {
    let config = &state(cx).config;
    let item = redirect(config, item);
    send_one(cx, &config.mail_from, &item).await
}

/// 検証のサーバーでは、宛先を差し替える。本来の宛先は本文の先頭に書く
pub fn redirect(config: &Config, item: Outgoing) -> Outgoing {
    match &config.mail_redirect_to {
        None => item,
        Some(to) => Outgoing {
            to: to.clone(),
            subject: format!("【検証】{}", item.subject),
            body: format!("（検証のメールです。本来の宛先: {}）\n\n{}", item.to, item.body),
        },
    }
}

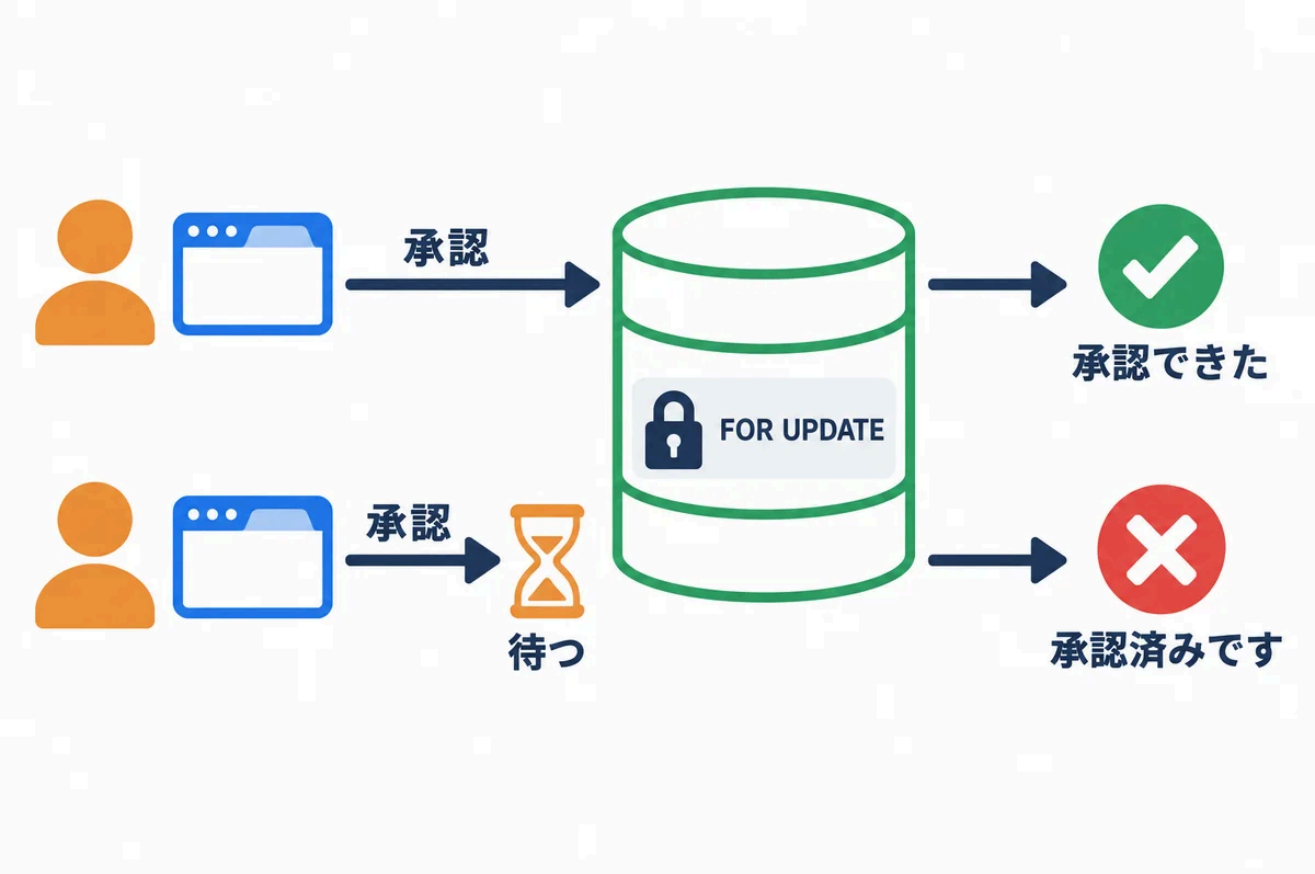
async fn send_one(cx: &Cx, from: &str, item: &Outgoing) -> topcoat::Result<()> {
    let mail = mail! {
        from: from,
        to: item.to.as_str(),
        subject: item.subject.as_str(),
        text: item.body.as_str(),
    }?;
    send(cx, mail).await?;
    Ok(())
}
```

▶ ここで初めて出てくるコード

- **`eprintln!`**（画面に出す）：`println!` のエラー版。「標準エラー出力」というエラー用の出口に書きます。画面ではふつうの出力と同じに見えますが、`> ファイル` で結果をファイルに書き出した時も、エラーは画面に残ります。失敗の知らせを出す時に使います。
- **`mail!`**（メール）：Topcoat の部品で、送るメールを組み立てるマクロ。`mail! { from: …, to: …, subject: …, text: … }`。宛先の形がまちがっているとエラーを返すので、`?` を付けます。
- **`.transport()`**（メール）：メールを実際に送る係（トランスポート）を決めるメソッド。ファイルに書き出す `FileTransport`・SMTP で送る `SmtpTransport`・メモリーにためる `MemoryTransport` などを渡します。
- **`.port()`**（メール）：つなぐ先のポート番号（SMTP なら 25・587 など）を決めるメソッド。
- **`MailConfig::builder()`**（メール）：`MailConfig` を組み立て始める関数。`.transport(送る係)` をつなげて、`.build()` で出来上がりです。
- **`FileTransport::new()`**（メール）：`FileTransport` を、書き出すフォルダを渡して作る関数。
- **`SmtpTransport::starttls()`**（メール）：STARTTLS（つないでから暗号化に切りかえる方法）で送る `SmtpTransport` を作り始める関数。`.port(587)` などをつなげて、`.build()` で出来上がりです。
- **`SmtpTransport::unencrypted()`**（メール）：暗号化しないで送る `SmtpTransport` を作り始める関数。社内の、外へ出ない中継サーバー（リレー）に渡す時などに使います。
- **`FileTransport`**（メール）：メールを送らずに、フォルダの中のファイルに書き出す係。開発中に、送られたはずのメールの中身を確かめるのに使います。
- **`MailConfig`**（メール）：Topcoat の、メールの送り方（どこへ・どうやって送るか）の設定の型。
- **`SmtpTransport`**（メール）：SMTP（メールを送る決まり）で、メールサーバーに渡して送る係。
- **`SmtpError`**（メール）：Topcoat の、SMTP の設定や送信で起きたエラーの型。
- **`Default`**（型とメソッド）：「初期値（空の値）を作れる」機能（トレイト）。`#[derive(Default)]` を付けると、数は 0、文字列は `""`、`bool` は `false` の初期値を作れます。

- 処理の途中では、メールを**箱**（`Outbox`）**に入れるだけ**です。`finish` が `commit` に成功してから、`send_all` で送ります。
- 途中で失敗したら、箱ごと捨てます。「承認できなかったのに、次の人に承認のお願いが届いた」が起きません。06 の `transaction.on_commit` と同じ考え方を、**目に見える形**で書いた物です。
- 送るのに失敗しても、保存はもう確定しているので、画面はエラーにしません（ログに書く）。届かなかった承認のお願いは、管理者の「送り直す」か、毎朝の処理で送り直せます。
- 検証のサーバーでは、`redirect` が宛先を `MAIL_REDIRECT_TO` に差し替え、件名に `【検証】`、本文の先頭に本来の宛先を書きます（onprem の 04 の 10.4 の設計例）。

6.3 同時に押されても、1 回だけ進める（FOR UPDATE）



コピー

```
SELECT … FROM steps s JOIN requests r … WHERE s.token_hash = $1 FOR UPDATE OF s, r
```

- `FOR UPDATE` は、読んだ行に**鍵をかけます**。同じ行を `FOR UPDATE` で読もうとした、ほかのトランザクションは、鍵が外れる（`commit` する）まで待ちます。
- 待っていた方は、鍵が外れてから読むので、「もう承認済み」と分かり、断られます。
- 13 章のテスト「同時に 2 回押しても 1 回だけ進む」で、`tokio::join!` を使い、**本当に 2 つ同時に**承認を走らせて確かめています。

6.4 できない操作と、思いがけない失敗を分ける（WorkflowError）

種類	例	画面では
`Rule`（できない操作）	「この依頼は、承認済みです。」「自分の依頼ではありません。」	文をそのまま出す（400 や 410）
`Internal`（思いがけない失敗）	データベースにつながらない	「エラー」とだけ出し、中身はログに書く（500）

Python では、`WorkflowError` だけを `except` で受け、ほかの例外は Django に任せました。Rust では、2 つを**enum の 2 つの形**にして、`match` で分けます。分け忘れると、コンパイルが通りません。

6.5 sqlx 0.9 は、組み立てた SQL の文字を、そのままでは受け取らない

`check_token` だけは、`lock` が true のときに `FOR UPDATE OF s, r` を足すため、SQL の文字を `format!` で組み立てています。sqlx 0.9 では、組み立てた文字をそのまま渡すと、**コンパイルのエラー**になります（実際の文）：

コピー

```
error[E0277]: dynamic SQL strings should be audited for possible injections
```

- SQL インジェクション（外から来た文字が、SQL の一部として動いてしまう攻撃）を防ぐための決まりです。
- ここで足すのは決まった文字だけで、外から来た値（URL の文字列）は `$1` で別に渡しています。それを確かめたうえで、`sqlx::AssertSqlSafe(sql)` で包んで「確かめた」と印を付けます。
- **外から来た値を `format!` で SQL に入れるのは、絶対にしません。**値は必ず `.bind()` で渡します（10 で詳しく）。

7. 1 回限りの承認の URL

7.1 承認のお願いのメール

`send_approval_mail` が、32 バイトの乱数を 16 進の 64 文字にして URL に入れ、**データベースには、その指紋（SHA-256）だけ**を保存します（06 と同じ考え方）。

開発のパソコンでは、メールは送らずに `sent_mail\` に `.eml` のファイルとして書かれます。実際のファイルの先頭（本文は、日本語のために base64 という形で書かれている。Message-ID の所は伏せた）：

コピー

```
From: noreply@example.com
To: 012346@example.com
Subject: =?utf-8?b?44CQ5L6d6aC844O75om/6KqN44CR5om/6KqN44Gu44GK6aGY44GE?=
 =?utf-8?b?OiDmsrvlhbfjga7oo73kvZw=?=
Message-ID: <GB3qVILkB0IpxIdSenYBR4ON5TXTyh8AGZht@（パソコンの名前）>
MIME-Version: 1.0
Date: Sun, 01 Nov 2026 16:05:04 +0000
Content-Type: text/plain; charset=utf-8
Content-Transfer-Encoding: base64

5L2Q6JekIOiKseWtkCDjgZXjgpMNCg0K5qyh44Gu5L6d6aC844Gu44CB5om/6KqN44KS44GK6aGY
44GE44GX44G+44GZ44CC77yI5Zue6Kan6ICFMe+8iQ0KDQrku7blkI06IOayu+WFt+OBruijveS9
nA0K5L6d6aC86ICFOiDlsbHnlLAg5aSq6YOO77yI6Kit6KiI6YOo77yJDQrluIzmnJvjga7ntI3m
nJ86IDIwMjYtMTEtMzANCg0K5LiL44GuIFVSTCDjgpLplovjgYTjgabjgIHjgIzmib/oqo3jgI3j
…（続く）
```

`.eml` はメールソフト（Outlook など）で開けます。読み解いた中身:

コピー

```
To: 012346@example.com
Subject: 【依頼・承認】承認のお願い: 治具の製作

佐藤 花子 さん

次の依頼の、承認をお願いします。（回覧者1）

件名: 治具の製作
依頼者: 山田 太郎（設計部）
希望の納期: 2026-11-30

下の URL を開いて、「承認」か「差し戻し」を選んでください。
http://localhost:3000/approve/fe90f09a983f61d2fc0b2b806da0af1444421e2931f4fede8ecfe52a40b075e7

・この URL は、あなただけのものです。ほかの人に転送しないでください。
・この URL は 1 回だけ使えます。14 日で使えなくなります。
```

7.2 承認の画面（app/approve/token.rs）

`src/app/approve/token.rs`：

コピー

```
//! /approve/{token} … メールの URL から開く、承認の画面。ログインは要らない
//!
//! URL の中の推測できない文字列が、「この回覧者本人だ」という印になる（06 の approve_views.py と同じ考え方）。
//! データベースには文字列そのものではなく指紋だけがあるので、データベースを見られても URL は作れない。

use serde::Deserialize;
use topcoat::{
    Result,
    context::Cx,
    router::{
        StatusCode,
        content::Form,
        error::{RouterErrorExt, not_found},
        href, module_param, page, path_param, response::Response, route,
    },
    view::{View, view},
};

use crate::{
    files,
    queries::{self, FullRequest},
    state::state,
    views::request_table,
    workflow::{self, TokenStep, WorkflowError},
};

module_param!(pub token: String, error = not_found);
// 添付ファイルの番号（/approve/{token}/files/{file_id}）
path_param!(pub file_id: i64, error = not_found);

/// URL の文字列を調べる（鍵はかけない。見るだけ）
async fn check(cx: &Cx) -> Result<std::result::Result<TokenStep, String>> {
    let token = path_param::<Token>(cx)?;
    let mut conn = state(cx).pool.acquire().await?;
    match workflow::check_token(&mut conn, token, false).await {
        Ok(step) => Ok(Ok(step)),
        Err(WorkflowError::Rule(m)) => Ok(Err(m)),
        Err(e) => Err(e.into()),
    }
}

#[page]
async fn approve_page(cx: &Cx) -> Result<impl View> {
    let token = path_param::<Token>(cx)?.clone();
    let page: std::result::Result<(TokenStep, FullRequest), String> = match check(cx).await? {
        Ok(step) => {
            let full = queries::find_request(&state(cx).pool, step.request_id).await?.ok_or_not_found()?;
            Ok((step, full))
        }
        Err(m) => Err(m),
    };
    // 使えない URL なら 410 Gone（もう無い）
    let status = if page.is_ok() { StatusCode::OK } else { StatusCode::GONE };
    Ok(view! {
        (status)
        match &page {
            Ok((step, full)) => {
                <h1>"承認のお願い（回覧者" (step.step_order) "）"</h1>
                <p>(&step.approver_name) " さん、次の依頼の承認をお願いします。"</p>
                request_table(r: &full.request)
                if !full.attachments.is_empty() {
                    <h2>"添付ファイル"</h2>
                    <ul>
                        for a in &full.attachments {
                            <li><a href=(href!(approve_file, Token(token.clone()), FileId(a.id)))>(&a.original_name)</a></li>
                        }
                    </ul>
                }
                <h2>"どちらかを選んでください"</h2>
                <form method="post" class="actions left">
                    <button type="submit" name="decision" value="approve" class="btn-primary">"承認する"</button>
                </form>
                <form method="post" class="form-grid">
                    <label for="comment">"差し戻す理由"</label>
                    <textarea id="comment" name="comment" rows="3" required=""></textarea>
                    <span></span>
                    <div class="actions">
                        <button type="submit" name="decision" value="return" class="btn-secondary">"差し戻す"</button>
                    </div>
                </form>
            },
            Err(m) => {
                <h1>"承認の画面"</h1>
```

```
                <p class="errors">(m)</p>
            },
        }
    })
}

/// ボタンから届く値
#[derive(Deserialize)]
struct DecisionForm {
    decision: String,
    comment: Option<String>,
}

#[page(POST)]
async fn approve_submit(cx: &Cx, Form(input): Form<DecisionForm>) -> Result<impl View> {
    let token = path_param::<Token>(cx)?.clone();
    let result = if input.decision == "approve" {
        workflow::approve(cx, &token).await.map(|t| format!("「{t}」を承認しました。"))
    } else {
        workflow::return_request(cx, &token, input.comment.as_deref().unwrap_or("")).await.map(|t| format!("「{t}」を差し戻しました。"))
    };
    let (status, class, message) = match result {
        Ok(m) => (StatusCode::OK, "message", m),
        Err(WorkflowError::Rule(m)) => (StatusCode::BAD_REQUEST, "errors", m),
        Err(e) => return Err(e.into()),
    };
    Ok(view! {
        (status)
        <h1>"承認の画面"</h1>
        <p class=(class) role="status">(message)</p>
        <p>"この画面は、閉じてかまいません。"</p>
    })
}

/// 承認の画面から、添付ファイルを開く（URL が使える間だけ・その依頼のファイルだけ）
#[route(GET "./files/{file_id}")]
async fn approve_file(cx: &Cx) -> Result<Response> {
    let Ok(step) = check(cx).await? else {
        return Err(not_found().into());
    };
    let file_id = *path_param::<FileId>(cx)?;
    let st = state(cx);
    let a = queries::find_attachment(&st.pool, file_id).await?.ok_or_not_found()?;
    if a.request_id != step.request_id {
        return Err(not_found().into());
    }
    files::download(&st.config.file_dir, &a).await
}
```

▶ ここで初めて出てくるコード

- **`module_param!`**（Topcoat）: Topcoat のマクロ。モジュールの区切りを「URL の変わる部品」にします。`mod id` の中に `module_param!(id: u64, error = not_found);` と書くと、URL が `/notes/{id}` になり、数として読めない時は 404 になります。
- **`path_param!`**（Topcoat）: URL の中の値（`/attachments/{file_id}` の `file_id` など）を受け取る型を作るマクロ。`path_param!(pub file_id: i64, error = not_found);` は、`FileId` という型を作り、数でない時は「見つかりません」にします。`href!(…, FileId(5))` で URL を作る時にも使います。
- **`#[route]`**（Topcoat）: Topcoat の印（属性）。画面（HTML）ではなく、JSON やただの文字などを返す「ルート」にします。`#[route(GET)]` のように、メソッドを必ず書きます。人が見る画面は `#[page]`、プログラムが読む答えは `#[route]` と使い分けます。
- **`.acquire()`**（データベース）: 接続の集まり（プール）から、接続を 1 本借りるメソッド。トランザクションを使わずに、同じ接続で続けて SQL を送りたい時に使います。
- **`.ok_or_not_found()`**（Topcoat）: Topcoat のメソッド。`Option` が `None`（無い）の時に、404 Not Found のエラーにします。`… .ok_or_not_found()?` と書くと、無い時はそこで 404 を返します（`RouterErrorExt` を `use` して使う）。
- **`queries::find_request()`**（練習アプリで作る物（Rust））: 依頼を 1 件、承認の流れや添付も含めて読む関数。無ければ `None` です。
- **`queries::find_attachment()`**（練習アプリで作る物（Rust））: 添付ファイルの記録を、番号で 1 件読む関数。無ければ `None` です。
- **`files::download()`**（練習アプリで作る物（Rust））: 添付ファイルを読んで、ダウンロードさせる答え（レスポンス）を作る関数。後編で作ります。
- **`StatusCode`**（HTTP）: HTTP の状態コード（`200 OK`・`404 Not Found` など）を表す型。`StatusCode::OK`・`StatusCode::NOT_FOUND` のように、名前で書けます。
- **`RouterErrorExt`**（Topcoat）: `.ok_or_not_found()` などを使えるようにする機能（トレイト）。
- **`Response`**（HTTP）: 答え（レスポンス）の型。Topcoat では `topcoat::router::response::Response`。ふつうのページは `view!` で作りますが、ファイルのダウンロードのように、見出しや本文を自分で決めたい時に直接作ります。
- **`FullRequest`**（練習アプリで作る物（Rust））: 依頼 1 件の全部（依頼そのもの・承認の流れ・添付）をまとめた struct。
- **`Token`**（Topcoat）: `topcoat::session::Token` は、セッションの印（でたらめな長い文字列）の型。ブラウザーのクッキーに入れ、サーバーには、そのハッシュだけを保存します。
- **`OK`**（HTTP）: `StatusCode::OK` は、HTTP の答えの番号 200（成功）です。
- **`GONE`**（HTTP）: 状態コード 410（Gone: もう無い）。`StatusCode::GONE`。ここでは、承認の URL の期限が切れた・もう使えない時に返しています（404 の「初めから無い」と区別する）。
- **`FileId`**（Topcoat）: `path_param!(pub file_id: i64 …)` が作る型。URL の中の、添付ファイルの番号を表します。`FileId(a.id)` で URL を作り、`*path_param::<FileId>(cx)?` で受け取ります。

- ログインは要りません。URL の中の文字列が「この回覧者本人だ」という印になります。
- 開くだけ（GET）のときは鍵をかけず、ボタンを押した（POST）ときに `approve` の中で鍵をかけます。
- 使えない URL を開いたら、**410 Gone**（もう無い）を返します。
- 添付ファイルも、URL が使える間だけ、その依頼のファイルだけを開けます（`approve_file`）。

実際の画面（佐藤さんが、メールの URL を開いた所）：

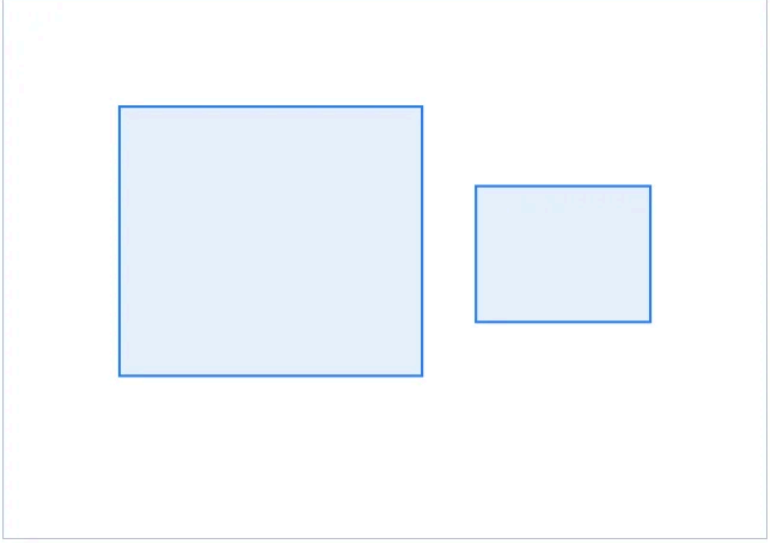
依頼・承認アプリ

承認のお願い（回覧者1）

佐藤 花子 さん、次の依頼の承認をお願いします。

件名	治具の製作
依頼者	山田 太郎（設計部）
希望の納期	2026-11-30
状態	回覧中
内容	検査用の治具を作ってください。 寸法は図面のとおりです。

図面



添付ファイル

- 現場の写真.png

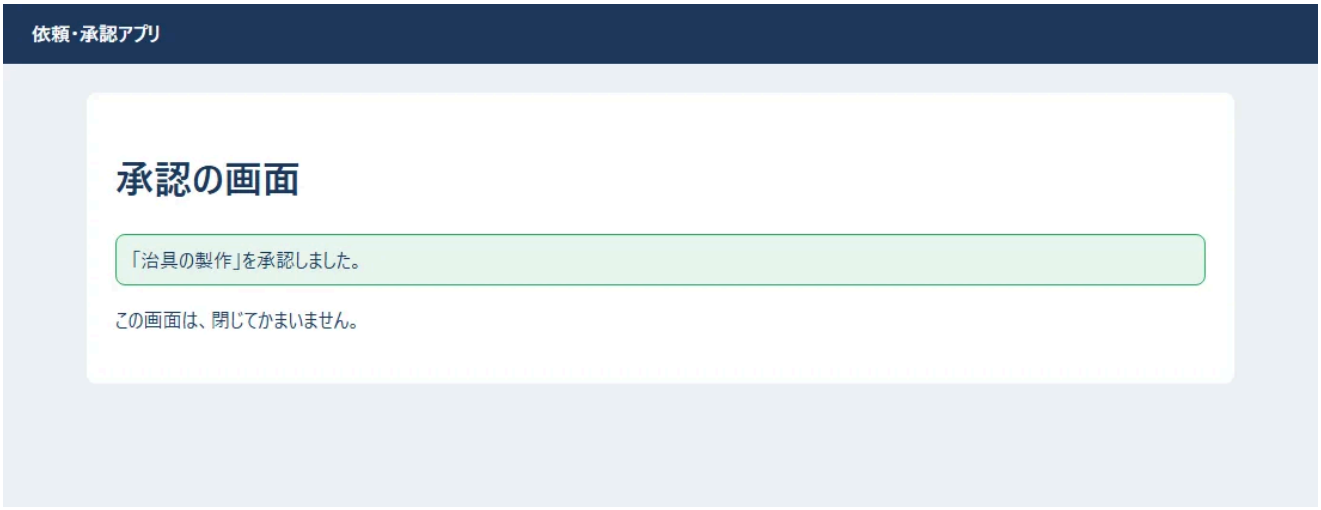
どちらかを選んでください

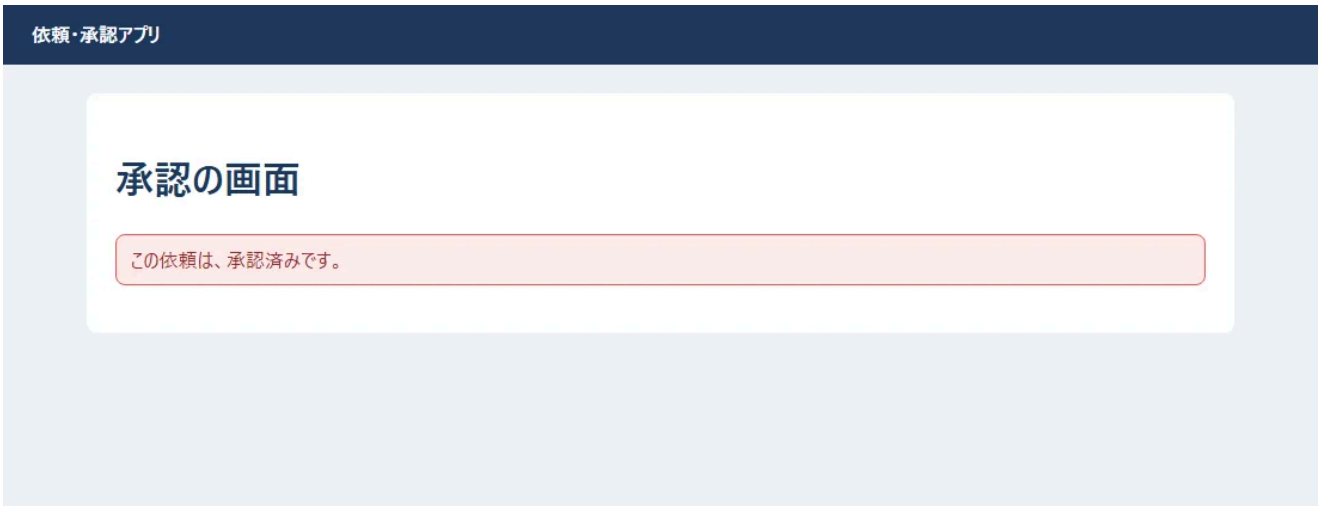
承認する

差し戻す理由

差し戻す

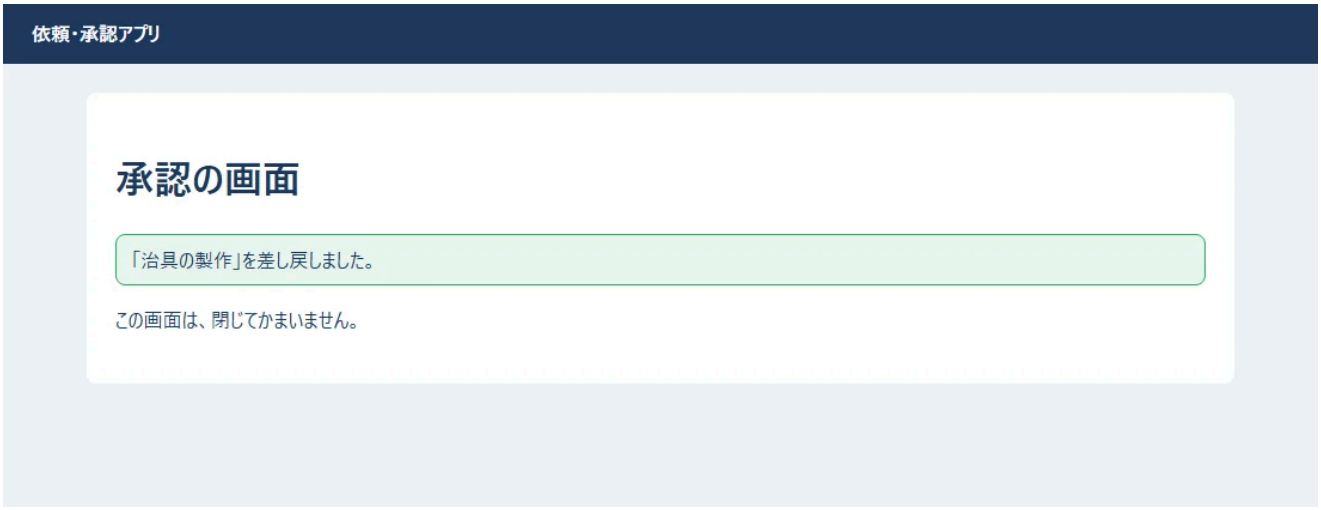
「承認する」を押した所と、同じ URL をもう一度開いた所:





確かめること: 2 回目は「この依頼は、承認済みです。」と出る（`curl` で確かめると、状態コードは 410）。

2 人目（鈴木さん）が、理由を書いて差し戻した所:



8. 社員の CSV と、管理のコマンド

8.1 1 つの exe が、サーバーとコマンドを兼ねる（cli.rs）

`src/cli.rs`：

コピー

```
//! exe のコマンド。1 つの webapp.exe が、サーバーと、管理用のコマンドを兼ねる
//!
//! ```text
//! webapp.exe                                  サーバーとして動く（serve と同じ）
//! webapp.exe import-employees 社員.csv [--dry-run]   社員の CSV を取り込む
//! webapp.exe grant-role --employee-no 090001 --role admin   役割を変える（member・worker・admin）
//! webapp.exe set-password --employee-no 012345  パスワードを決める（標準入力から 1 行読む）
//! webapp.exe send-daily-reminders               毎朝の通知を、今すぐ 1 回動かす
//! webapp.exe send-test-mail --to 自分のアドレス   テストのメールを 1 通送る（メールの道を確かめる。09）
//! webapp.exe seed-demo                          練習用の社員を作る（開発のパソコンだけ）
//! ```

use std::{io::BufRead, sync::Arc};

use anyhow::{Context, bail};
use topcoat::context::{AppContext, Cx};

use crate::{
    app, auth,
    config::{AppEnv, Config},
    employees, jobs,
    mail::mail_config,
    models::Role,
    state::{AppState, connect, migrate},
    workflow,
};

// 行末の \ は「次の行へ続く（改行と、次の行の頭の空白は入らない）」という印
const USAGE: &str = "使い方: webapp [serve | import-employees <CSV> [--dry-run] \
                     | grant-role --employee-no <番号> --role <member|worker|admin> \
                     | set-password --employee-no <番号> | send-daily-reminders \
                     | send-test-mail --to <アドレス> | seed-demo]";

pub async fn run(args: Vec<String>) -> anyhow::Result<()> {
    // 開発のパソコンでは、.env のファイルから環境変数を読む（サーバーには .env を置かず、サービスの XML に書く）
    let _ = dotenvy::dotenv();
    let config = Config::from_env()?;
    let command = args.first().map(String::as_str).unwrap_or("serve");
    let rest = args.get(1..).unwrap_or_default();

    match command {
        "serve" => serve(config).await,
        "import-employees" => import_employees(config, rest).await,
        "grant-role" => {
            let no = option(rest, "--employee-no")?;
            let role = Role::try_from(option(rest, "--role")?).map_err(anyhow::Error::msg)?;
            let state = open(config).await?;
            workflow::grant_role(&state.pool, &no, role).await?;
            println!("{no} の役割を「{}」にしました。", role.label());
            Ok(())
        }
        "set-password" => set_password(config, rest).await,
        "send-daily-reminders" => {
            let state = open(config).await?;
            let cx = Cx::new(background_context(&state)?);
            let r = jobs::daily_reminders(&cx).await?;
            println!("送り直した承認のお願い: {} 件 / 納期が近い依頼: {} 件", r.resent, r.due_soon);
            Ok(())
        }
        "send-test-mail" => send_test_mail(config, rest).await,
        "seed-demo" => seed_demo(config).await,
        "help" | "--help" | "-h" => {
            println!("{USAGE}");
            Ok(())
        }
        other => bail!("知らないコマンドです: {other}\n{USAGE}"),
    }
}

/// `--name 値` の形の値を取り出す
fn option(args: &[String], name: &str) -> anyhow::Result<String> {
    let i = args.iter().position(|a| a == name).with_context(|| format!("{name} がありません\n{USAGE}"))?;
    args.get(i + 1).cloned().with_context(|| format!("{name} の値がありません"))
}

/// データベースにつなぎ、表を新しくする（まだ流していないマイグレーションを流す）
async fn open(config: Config) -> anyhow::Result<AppState> {
    let pool = connect(&config.database_url).await.context("データベースにつなげません（DATABASE_URL を確かめる）")?;
    migrate(&pool).await.context("マイグレーションに失敗しました")?;
```

```
    Ok(AppState { pool, config: Arc::new(config) })
}

/// 画面のリクエストの外（毎朝の処理・コマンド）で使う、アプリの共有の物の入れ物
fn background_context(state: &AppState) -> anyhow::Result<Arc<AppContext>> {
    let mut app = AppContext::new();
    app.insert(state.clone());
    app.insert(mail_config(&state.config.mail)?);
    Ok(Arc::new(app))
}

async fn serve(config: Config) -> anyhow::Result<()> {
    let state = open(config).await?;
    let mail = mail_config(&state.config.mail)?;
    jobs::spawn_daily(background_context(&state)?);
    println!("起動します（{:?}）。メールの URL は {} で始まります。", state.config.app_env, state.config.base_url);
    // HOST と PORT（待ち受ける所）は、Topcoat が環境変数から読む。無ければ 127.0.0.1:3000
    topcoat::start(app::router(state, mail)).await?;
    Ok(())
}

async fn import_employees(config: Config, args: &[String]) -> anyhow::Result<()> {
    let path = args.iter().find(|a| !a.starts_with("--")).with_context(|| format!("CSV のファイルを指定してください\n{USAGE}"))?;
    let dry_run = args.iter().any(|a| a == "--dry-run");
    let bytes = std::fs::read(path).with_context(|| format!("{path} を読めません"))?;
    let rows = match employees::read_csv(&bytes) {
        Ok(rows) => rows,
        Err(errors) => {
            for e in &errors {
                eprintln!("× {e}");
            }
            bail!("{} 件の間違いがあるので、取り込みませんでした。", errors.len());
        }
    };
    if dry_run {
        println!("間違いはありません（{} 件）。--dry-run なので、取り込んでいません。", rows.len());
        return Ok(());
    }
    let state = open(config).await?;
    let (created, updated) = employees::import(&state.pool, &rows).await?;
    println!("取り込みました（新しく {created} 件・書きかえ {updated} 件）。");
    Ok(())
}

/// テストのメールを 1 通送る。データベースは使わないので、つながなくても動く
async fn send_test_mail(config: Config, args: &[String]) -> anyhow::Result<()> {
    let to = option(args, "--to")?;
    let where_to = match &config.mail {
        crate::config::MailSettings::File(dir) => format!("ファイル（{}）", dir.display()),
        crate::config::MailSettings::Smtp { host, port, starttls } => {
            format!("{host}:{port}（STARTTLS: {}）", if *starttls { "使う" } else { "使わない" })
        }
    };
    let now = time::OffsetDateTime::now_utc().to_offset(crate::models::JST);
    let body = format!(
        "依頼・承認アプリからの、送信テストです。\n\n送った時刻: {}\nAPP_ENV: {:?}\n渡した先: {where_to}\n",
        crate::models::fmt_datetime(now),
        config.app_env
    );
    // connect_lazy は、本当に使うまでデータベースにつながない
    let pool = sqlx::PgPool::connect_lazy(&config.database_url)?;
    let state = AppState { pool, config: Arc::new(config) };
    let cx = Cx::new(background_context(&state)?);
    if let Err(e) = crate::mail::send_now(&cx, crate::mail::Outgoing::new(&to, "送信テスト", body)).await {
        // 原因の連なり（「つなげない」←「証明書を確かめられない」など）を、全部書き出す
        let mut message = e.to_string();
        let mut source = e.source();
        while let Some(s) = source {
            message.push_str(&format!("\n  原因: {s}"));
            source = s.source();
        }
        bail!("送れませんでした（渡した先: {where_to}）: {message}");
    }
    println!("渡しました（宛先: {to}・渡した先: {where_to}）。届いたかを、受け取る側で確かめてください。");
    Ok(())
}

async fn set_password(config: Config, args: &[String]) -> anyhow::Result<()> {
    let no = option(args, "--employee-no")?;
    eprintln!("{no} の新しいパスワードを入れて、Enter を押してください（12 文字以上）:");
    let mut line = String::new();
    std::io::stdin().lock().read_line(&mut line)?;
```

```
    let password = line.trim_end_matches(['\r', '\n']).to_string();
    if password.chars().count() < 12 {
        bail!("パスワードは 12 文字以上にしてください。");
    }
    let state = open(config).await?;
    let hash = auth::hash_password(password).await;
    let n = sqlx::query("UPDATE employees SET password_hash = $2 WHERE employee_no = $1")
        .bind(&no)
        .bind(hash)
        .execute(&state.pool)
        .await?
        .rows_affected();
    if n == 0 {
        bail!("従業員番号 {no} の社員がいません。");
    }
    println!("{no} のパスワードを決めました。");
    Ok(())
}

/// 練習用の社員（06 の seed_demo と同じ 5 人）
pub const DEMO_PEOPLE: [(&str, &str, &str, &str, Role); 5] = [
    ("012345", "山田 太郎", "設計部", "主任", Role::Member),
    ("012346", "佐藤 花子", "設計部", "課長", Role::Member),
    ("012347", "鈴木 一郎", "設計部", "部長", Role::Member),
    ("020001", "高橋 三郎", "設備課", "係長", Role::Worker),
    ("090001", "管理 次郎", "情報システム部", "", Role::Admin),
];
pub const DEMO_PASSWORD: &str = "practice-pass-2026"; // 練習用。本物のパスワードには使わない

async fn seed_demo(config: Config) -> anyhow::Result<()> {
    if config.app_env != AppEnv::Development {
        bail!("APP_ENV が development のとき（開発のパソコン）だけ使えます。");
    }
    let state = open(config).await?;
    for (no, name, dept, title, role) in DEMO_PEOPLE {
        // ハッシュは 1 人ずつ作る。塩（salt）が毎回違うので、同じパスワードでも、ハッシュは人ごとに違う
        // （1 回作って使い回すと、全員のハッシュが同じになり、「同じパスワードの人」が見えてしまう）
        let hash = auth::hash_password(DEMO_PASSWORD.to_string()).await;
        sqlx::query(
            "INSERT INTO employees (employee_no, name, dept, job_title, email, role, password_hash)
             VALUES ($1, $2, $3, $4, $5, $6, $7)
             ON CONFLICT (employee_no) DO UPDATE
                SET name = EXCLUDED.name, dept = EXCLUDED.dept, job_title = EXCLUDED.job_title,
                    email = EXCLUDED.email, role = EXCLUDED.role, password_hash = EXCLUDED.password_hash, is_active = TRUE",
        )
        .bind(no)
        .bind(name)
        .bind(dept)
        .bind(title)
        .bind(format!("{no}@example.com"))
        .bind(role.as_str())
        .bind(&hash)
        .execute(&state.pool)
        .await?;
    }
    println!("{} 人を作りました。パスワードは全員 {DEMO_PASSWORD} です（練習用）。", DEMO_PEOPLE.len());
    Ok(())
}
```

▶ ここで初めて出てくるコード

- **`bail!`**（エラー処理）：`anyhow` のマクロ。その場でエラーを作って、関数から返します。`bail!("ノートが 1 件もありません");` は `return Err(anyhow!("…"));` の短い書き方です。
- **`while`**（くり返し）：条件が `true` の間、くり返す書き方。`while 条件 { … }`。条件が最初から `false` なら、1 回も動きません。
- **`.first()`**（Vec（並び））：並びの最初の 1 つを `Option` で返すメソッド（空なら `None`）。最後の 1 つは `.last()` です。
- **`.get()`**（Vec（並び））：番号やキーで中身を取り出すメソッド。`v.get(10)` は、範囲の外なら止まらずに `None` を返します（`v[10]` はプログラムが止まる）。文字列の `s.get(0..3)` は、文字の区切りでない位置なら `None` です。`HashMap` の `map.get("キー")` も、無ければ `None` です。
- **`.position()`**（イテレーター）：並びの中で、条件に合う最初の物が「何番目か」を `Option` で返すメソッド。`notes.iter().position(|n| n.id == id)`。無ければ `None` です。
- **`.with_context()`**（エラー処理）：`anyhow` のメソッド。失敗した時のエラーに、「何をしていて失敗したか」の説明を書き足します。`.with_context(|| format!("{path} を読めませんでした"))?`。ログを見た人が、原因をすぐ分かるようになります。
- **`.cloned()`**（Option）：借りている値を、`.clone()` で複製した値にするメソッド。`Option<&String>` を `Option<String>` に、イテレーターなら借りた物の並びを複製の並びにします。
- **`.context()`**（エラー処理）：`anyhow` の部品で、エラーに「何をしていた時のエラーか」の説明を足すメソッド。`fs::read_to_string(path).context("設定ファイルを読めません")?`。説明を `format!` で作る時は、必要な時だけ作る `.with_context(|| …)` を使います。
- **`.starts_with()`**（文字列）：その文字（文字列）で始まっていれば `true` を返すメソッド。`"report.csv".starts_with("report")` は `true`。終わりを調べるのは `.ends_with()`、含むかは `.contains()` です。
- **`.any()`**（イテレーター）：並びの中に、条件に合う物が 1 つでもあれば `true` を返すメソッド。全部が合うかは `.all()` で調べます。
- **`.display()`**（ファイルとフォルダ）：ファイルやフォルダの場所（`Path`・`PathBuf`）を、`println!` で表示できる形にするメソッド。場所は、そのままでは `{}` で出せないので、`path.display()` を通して出します。
- **`.source()`**（エラー処理）：エラーの「元になったエラー」を返すメソッド。`while let Some(e) = err.source()` のようにたどると、原因をさかのぼれます。
- **`.push_str()`**（文字列）：`String` の後ろに文字列を足すメソッド。`s.push_str("さん");`。`s` は書きかえるので、`let mut s` のように `mut` が要ります。
- **`.read_line()`**（入出力）：1 行だけ読んで、文字列に書き足すメソッド。読んだ行の最後には改行が付いたままなので、`.trim_end()` などで取ってから使います。
- **`.trim_end_matches()`**（文字列）：文字列の後ろから、決めた文字（文字列）をくり返し取り除いたものを返すメソッド。`name.trim_end_matches(".rs")` は、`.rs` を取ります。

- **`.chars()`**（文字列）：文字列を、1 文字ずつ（ `char` ）取り出せる形にするメソッド。`s.chars().count()` で文字の数、`s.chars().take(60).collect::<String>()` で先頭の 60 文字。日本語も 1 文字ずつ正しく扱えます（ `len()` はバイト数）。
- **`.count()`**（イテレーター）：イテレーター（1 つずつ取り出せる物）の中身を最後まで数えて、その数を返すメソッド。`env::args().skip(1).count()` は「渡された引数の数」、`s.chars().count()` は「文字の数」です。
- **`dotenvy::dotenv()`**（設定）： `.env` ファイルに書いた「名前=値」を、環境変数として読み込む関数（ `dotenvy` の部品）。開発のパソコンで、設定をファイルにまとめておく時に使います。 `.env` は Git に入れません。
- **`Cx::new()`**（Topcoat）： `Cx`（Topcoat の、共有の物を取り出す入れ物）を、画面のリクエストの外で作る関数。毎朝の処理やコマンドで、メールの設定などを使うために作ります。
- **`jobs::daily_reminders()`**（練習アプリで作る物（Rust））：毎朝の処理（期限の近い依頼や、止まっている承認の知らせ）を 1 回する関数。
- **`Arc::new()`**（スレッドと共有）：値を、いくつものスレッドで一緒に持てる `Arc` に入れる関数。`Arc::new(Mutex::new(0))` は「鍵付きの 0 を、みんなで持つ」。
- **`AppContext::new()`**（Topcoat）：空の `AppContext` を作る関数。 `.insert(物)` で入れていきます。
- **`jobs::spawn_daily()`**（練習アプリで作る物（Rust））：毎朝の処理を、サーバーの裏で動かし続ける関数。
- **`topcoat::start()`**（Topcoat）：組み立てたルーターを受け取って、Web サーバーを動かし始める関数。`topcoat::start(router).await` は、止めるまでリクエストを待ち続けます。
- **`std::fs::read()`**（ファイルとフォルダ）：ファイルの中身を全部、バイトの並び（ `Vec<u8>` ）として読む関数。文字として読む時は `fs::read_to_string` です。
- **`employees::read_csv()`**（練習アプリで作る物（Rust））：社員の CSV を読んで、1 行ずつ確かめる関数。まちがいがあれば、何行目の何がおかしいかを返します。
- **`employees::import()`**（練習アプリで作る物（Rust））：読んだ社員を、データベースに入れる（あれば変える）関数。作った数と変えた数を返します。
- **`time::OffsetDateTime::now_utc()`**（時間）：今の日時を、世界標準時（UTC）で返す関数（ `OffsetDateTime::now_utc()` と同じ）。
- **`sqlx::PgPool::connect_lazy()`**（データベース）：PostgreSQL へのプールを、まだつながずに作る関数。最初に使う時に、はじめてつなぎます。
- **`String::new()`**（文字列）：空の `String` を作る関数。あとで中身を足していく時（ `read_line(&mut s)` で 1 行読み込む など）に使います。
- **`std::io::stdin()`**（入出力）：キーボードからの入力（標準入力）を表す物を返す関数。`std::io::stdin().lock().read_line(&mut line)?` で、1 行読めます。
- **`BufRead`**（入出力）：「1 行ずつ読める物」が持っている機能（トレイト）。`read_line` や `lines` を使うには、`use std::io::BufRead;` が要ります。
- **`Arc`**（スレッドと共有）：いくつものスレッドから、1 つの値を一緒に持てるようにする型（ `Rc` のスレッド版）。書きかえもしたい時は、`Arc<Mutex<T>>` の形にします。

- **`Context`**（エラー処理）：`anyhow::Context` は、`with_context`・`context` メソッドを使えるようにする機能（トレイト）。`use anyhow::Context;` と書いておくと、`Result` や `Option` に説明を付けられます。
- **`AppContext`**（Topcoat）：アプリで共有する物（データベースや設定など）を、型ごとに入れておく入れ物。
- **`AppState`**（練習アプリで作る物（Rust））：アプリ全体で共有する物（データベースのプールと設定）をまとめた struct。`.app_context(…)` でルーターに渡し、どのページからも `state(cx)` で使えるようにします。
- **`INSERT`**（SQL）：`INSERT INTO 表 (列, …) VALUES (値, …)` で、表に新しい行を足す SQL の命令。
- **`INTO`**（SQL）：`INSERT INTO 表` の一部で、「この表へ」入れる、という意味。
- **`VALUES`**（SQL）：`INSERT INTO 表 (列, …) VALUES (値, …)` の、入れる値を書く所。
- **`CONFLICT`**（HTTP）：`StatusCode::CONFLICT` は、HTTP の答えの番号 409（「ぶつかった」）。同じメールアドレスで登録しようとした時などに返します。
- **`DO`**（SQL）：`INSERT … ON CONFLICT (列) DO UPDATE SET …` は、「同じ値の行がもうあれば、足す代わりに、その行を変える」という書き方です（アップサート）。何もしない時は `DO NOTHING` です。
- **`EXCLUDED`**（SQL）：`ON CONFLICT … DO UPDATE` の中で、「足そうとした（はじかれた）方の行」を指す名前。`SET name = EXCLUDED.name` は「今の行の名前を、新しく入れようとした名前に変える」です。

- 06 の `manage.py import_employees` などに当たる物を、**`webapp.exe` のサブコマンド**にしました。サーバーに持ち込む物が増えません。
- サーバーでは、onprem の 04 の 7.7 のとおり、その窓の中だけ `DATABASE_URL` などを教えてから動かします。
- `set-password` は、パスワードを**コマンドの引数ではなく、標準入力から**読みます。引数に書くと、PowerShell の履歴に残るからです。
- `send-test-mail --to 自分のアドレス` は、テストのメールを 1 通送ります（onprem の 04 の 10.3 の「アプリから送る」。使い方は 09 の巻）。

8.2 CSV を読んで確かめる（employees.rs）

`src/employees.rs`：

コピー

```
//! 社員の CSV を取り込む（06 の import_employees と同じ決まり）
//!
//! - 1 行目の見出しは「従業員番号,氏名,所属,役職,メールアドレス」
//! - 1 行でも間違いがあれば、1 件も取り込まない（全部か、何もしないか）
//! - 同じ従業員番号の社員がいれば、氏名などを書きかえる。いなければ作る（パスワードは未設定）
//! - CSV は Excel の「CSV UTF-8（コンマ区切り）」で保存した物（先頭の BOM は読み飛ばす）

use sqlx::PgPool;

pub const COLUMNS: [&str; 5] = ["従業員番号", "氏名", "所属", "役職", "メールアドレス"];

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct EmployeeRow {
    pub employee_no: String,
    pub name: String,
    pub dept: String,
    pub job_title: String,
    pub email: String,
}

/// CSV を読んで、確かめる。間違いがあれば、行の番号付きの文を全部返す
pub fn read_csv(bytes: &[u8]) -> Result<Vec<EmployeeRow>, Vec<String>> {
    let bytes = bytes.strip_prefix(b"\xef\xbb\xbf").unwrap_or(bytes); // BOM を読み飛ばす
    if std::str::from_utf8(bytes).is_err() {
        return Err(vec!["文字コードが UTF-8 ではありません。Excel で「CSV UTF-8」として保存し直してください。".into()]);
    }
    let mut reader = csv::ReaderBuilder::new().has_headers(true).from_reader(bytes);
    let headers: Vec<String> = match reader.headers() {
        Ok(h) => h.iter().map(|s| s.trim().to_string()).collect(),
        Err(e) => return Err(vec![format!("1 行目を読めません: {e}")]),
    };
    if headers != COLUMNS {
        return Err(vec![format!("1 行目の見出しは {} にしてください（今: {}）", COLUMNS.join(","), headers.join(","))]);
    }

    let mut rows = Vec::new();
    let mut errors = Vec::new();
    let mut seen = std::collections::HashSet::new();
    for (i, record) in reader.records().enumerate() {
        let n = i + 2; // 見出しが 1 行目なので、データは 2 行目から
        let record = match record {
            Ok(r) => r,
            Err(e) => {
                errors.push(format!("{n} 行目: 読めません（{e}）"));
                continue;
            }
        };
        let field = |k: usize| record.get(k).unwrap_or("").trim().to_string();
        let row = EmployeeRow { employee_no: field(0), name: field(1), dept: field(2), job_title: field(3), email: field(4) };

        if !(row.employee_no.len() == 6 && row.employee_no.bytes().all(|b| b.is_ascii_digit())) {
            errors.push(format!("{n} 行目: 従業員番号は 6 桁の数字にしてください（\"{}\"）", row.employee_no));
        } else if !seen.insert(row.employee_no.clone()) {
            errors.push(format!("{n} 行目: 従業員番号 {} が 2 回出てきます", row.employee_no));
        } else if row.name.is_empty() {
            errors.push(format!("{n} 行目: 氏名が空です"));
        } else if !looks_like_email(&row.email) {
            errors.push(format!("{n} 行目: メールアドレスの形が違います（\"{}\"）", row.email));
        } else {
            rows.push(row);
        }
    }
    if errors.is_empty() { Ok(rows) } else { Err(errors) }
}

/// メールアドレスの、おおまかな形（@ が 1 つ・前後が空でない・後ろに . がある）
fn looks_like_email(s: &str) -> bool {
    match s.split_once('@') {
        Some((local, domain)) => {
            !local.is_empty() && !domain.contains('@') && domain.contains('.') && !domain.starts_with('.') && !domain.ends_with('.')
        }
        None => false,
    }
}

/// 取り込む（1 つのトランザクションで）。新しく作った数と、書きかえた数を返す
pub async fn import(pool: &PgPool, rows: &[EmployeeRow]) -> sqlx::Result<(usize, usize)> {
    let mut tx = pool.begin().await?;
    let (mut created, mut updated) = (0, 0);
    for r in rows {
```

```
        // xmax = 0 なら新しく入った行（PostgreSQL の決まり。作ったか書きかえたかを見分ける）
        let inserted: bool = sqlx::query_scalar(
            "INSERT INTO employees (employee_no, name, dept, job_title, email) VALUES ($1, $2, $3, $4, $5)
             ON CONFLICT (employee_no) DO UPDATE
                SET name = EXCLUDED.name, dept = EXCLUDED.dept, job_title = EXCLUDED.job_title, email = EXCLUDED.email
             RETURNING (xmax = 0)",
        )
        .bind(&r.employee_no)
        .bind(&r.name)
        .bind(&r.dept)
        .bind(&r.job_title)
        .bind(&r.email)
        .fetch_one(&mut *tx)
        .await?;
        if inserted { created += 1 } else { updated += 1 }
    }
    tx.commit().await?;
    Ok((created, updated))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn 正しいcsvを読める() {
        let rows = read_csv(include_bytes!("../sample_data/employees.csv")).unwrap();
        assert_eq!(rows.len(), 3);
        assert_eq!(rows[2].name, "山田 太郎");
    }

    #[test]
    fn 間違いのあるcsvは全部の間違いを返す() {
        let errors = read_csv(include_bytes!("../sample_data/employees_bad.csv")).unwrap_err();
        assert_eq!(
            errors,
            vec![
                "3 行目: 従業員番号は 6 桁の数字にしてください（\"30004\"）".to_string(),
                "4 行目: メールアドレスの形が違います（\"kato.example.com\"）".to_string(),
            ]
        );
    }

    #[test]
    fn shift_jisは断る() {
        // 「従」を Shift_JIS で書いた 2 バイト
        let errors = read_csv(b"\x8f\x5d,a\n").unwrap_err();
        assert!(errors[0].contains("UTF-8 ではありません"));
    }
}
```

▶ ここで初めて出てくるコード

- **`include_bytes!`**（ファイルとフォルダ）：ファイルの中身を、プログラムを作る時に、バイトの並びとして中に埋め込むマクロ。`include_bytes!("../sample_data/employees.csv")`。テストで、決まったファイルを使う時に便利です。
- **`continue`**（くり返し）：くり返しの「今回の残り」を飛ばして、次の回へ進む命令。`for` や `while` の中で、条件に合わない物を飛ばす時に使います。
- **`.strip_prefix()`**（文字列）：文字列が、決めた文字で始まっていれば、それを取り除いた残りを `Some(残り)` で返すメソッド。始まっていなければ `None` です。
- **`.has_headers()`**（ファイルとフォルダ）：CSV の 1 行目が見出し（列の名前）かどうかを決めるメソッド。`true` なら、1 行目は中身として数えません。
- **`.from_reader()`**（ファイルとフォルダ）：決めた読み方で、渡したデータ（バイトの並びやファイル）から CSV を読む係を作るメソッド。
- **`.headers()`**（HTTP）：答え（レスポンス）の見出し（ヘッダー）の一覧を返すメソッド。`.headers().get("set-cookie")` のように、名前で 1 つ取り出せます。
- **`.join()`**（ファイルとフォルダ）：場所（パス）に、フォルダやファイルの名前をつなげた新しい場所を作るメソッド。`dir.join("notes.txt")` は、`dir` の中の `notes.txt` を表します。区切りの `\` は自動で入ります。（文字列の配列に使う `.join(", ")` は、「区切り文字でつなげて 1 つの文字列にする」別の意味です。）
- **`.records()`**（ファイルとフォルダ）：CSV の行を、1 行ずつ取り出せる形にするメソッド。`for (i, record) in reader.records().enumerate()` で、何行目かと中身を同時に使えます。
- **`.bytes()`**（文字列）：文字列を、1 バイトずつ取り出せる形にするメソッド。`s.bytes().all(|b| b.is_ascii_digit())` で「全部、半角の数字か」を調べます。
- **`.all()`**（イテレーター）：イテレーターの中身が、全部条件に合うかを `true` / `false` で返すメソッド。`v.iter().all(|n| *n > 0)` は「全部 0 より大きいか」。1 つでも合うかは `.any()` です。
- **`.is_ascii_digit()`**（文字）：その文字（バイト）が、半角の数字（`0` 〜 `9`）かを `true` / `false` で返すメソッド。
- **`.split_once()`**（文字列）：文字列を、最初に見つかった区切りの所で 2 つに分けるメソッド。`"title=買い物".split_once('=')` は `Some(("title", "買い物"))`。区切りが無い時は `None` です。
- **`.ends_with()`**（文字列）：その文字（文字列）で終わっていれば `true` を返すメソッド。`name.ends_with(".db")`。
- **`csv::ReaderBuilder::new()`**（ファイルとフォルダ）：`csv` の部品で、CSV の読み方を組み立て始める関数。`.has_headers(true)`（1 行目は見出し）などを決めて、`.from_reader(バイト)` で読む係を作ります。
- **`std::collections::HashSet::new()`**（HashMap（表））：空の `HashSet`（同じ物を 2 回入れない入れ物）を作る関数。`.insert(値)` は、初めてなら `true`、もうあれば `false` を返すので、「同じ社員番号が 2 回出てきた」ことを見つけられます。
- **`ReaderBuilder`**（ファイルとフォルダ）：`csv` の、CSV の読み方を組み立てる型。

- **`HashSet`**（HashMap（表））：同じ値は 1 つしか入らない集まり（値だけの `HashMap`）。重複を消す・「もう見たか」を覚える時に使います。`set.insert(x)` は、新しく入った時だけ `true` を返します。
- **`RETURNING`**（SQL）：`INSERT … RETURNING id` で、入れた（変えた）行の値（作られた番号など）を、その場で返してもらう書き方。

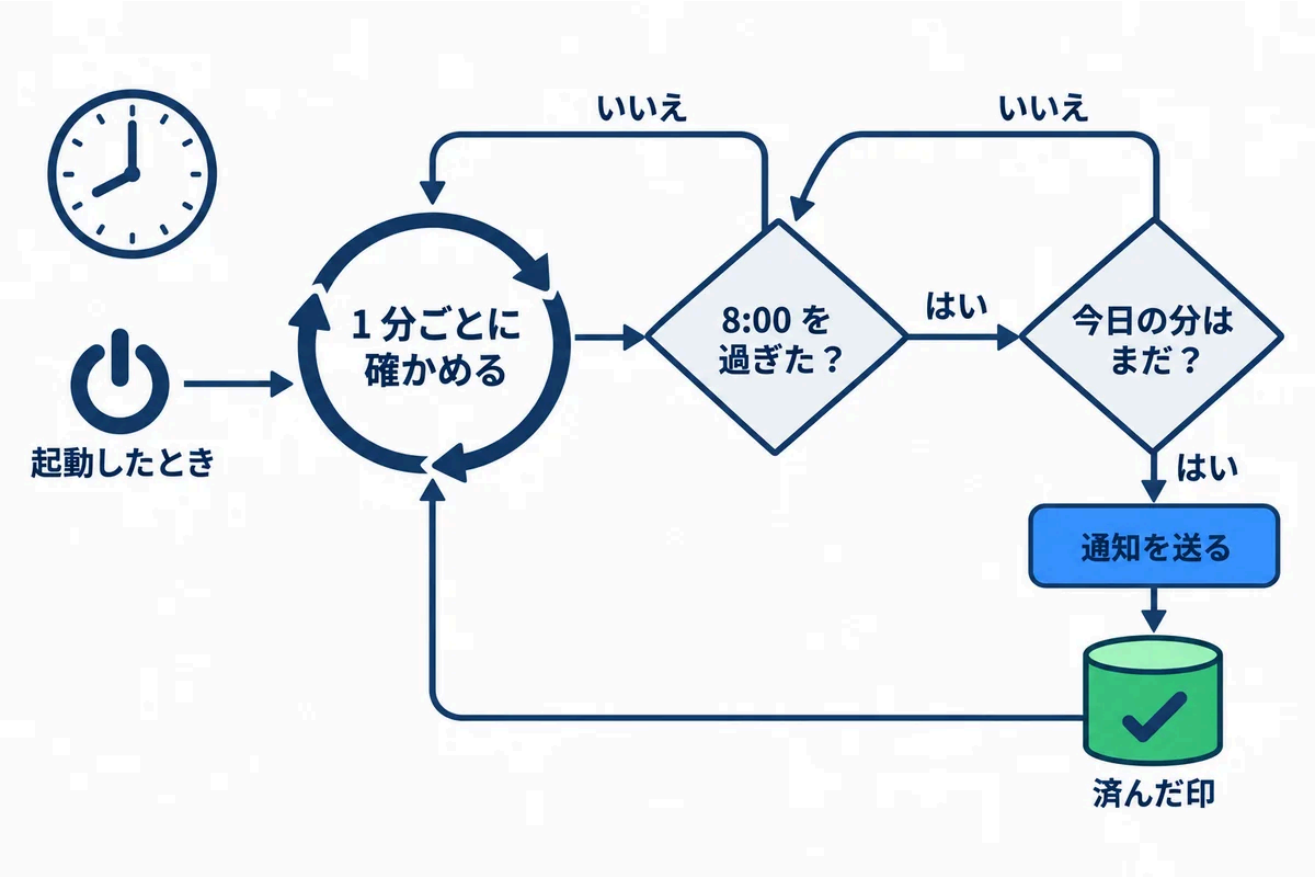
- 06 と同じく「1 行でも間違いがあれば、1 件も取り込まない」です。読む・確かめる（`read_csv`）と、取り込む（`import`）を分け、取り込みは 1 つのトランザクションで行います。
- Excel の「CSV UTF-8」は、先頭に BOM（目に見えない 3 バイト）が付きます。読み飛ばしてから読みます。Shift_JIS の CSV は、はっきりした文で断ります。
- `ON CONFLICT … DO UPDATE` は、「同じ従業員番号があれば書きかえ、無ければ足す」を、1 つの SQL でする書き方です（06 の `update_or_create`）。

8.3 実際に動かした結果

コピー

```
$ webapp.exe help
使い方: webapp [serve | import-employees <CSV> [--dry-run] | grant-role --employee-no <番号> --role
<member|worker|admin> | set-password --employee-no <番号> | send-daily-reminders | send-test-mail --to <ア
ドレス> | seed-demo]
(exit 0)
$ webapp.exe import-employees sample_data/employees_bad.csv
× 3 行目: 従業員番号は 6 桁の数字にしてください（"30004"）
× 4 行目: メールアドレスの形が違います（"kato.example.com"）
エラー: 2 件の間違いがあるので、取り込みませんでした。
(exit 1)
$ webapp.exe import-employees sample_data/employees.csv --dry-run
間違いはありません（3 件）。--dry-run なので、取り込んでいません。
(exit 0)
$ webapp.exe import-employees sample_data/employees.csv
取り込みました（新しく 2 件・書きかえ 1 件）。
(exit 0)
$ webapp.exe grant-role --employee-no 030002 --role worker
030002 の役割を「担当者」にしました。
(exit 0)
$ webapp.exe grant-role --employee-no 999999 --role admin
エラー: 従業員番号 999999 の社員がいません。
(exit 1)
$ webapp.exe send-daily-reminders
送り直した承認のお願い: 0 件 / 納期が近い依頼: 1 件
(exit 0)
$ webapp.exe seed-demo（APP_ENV=production）
エラー: APP_ENV が development のとき（開発のパソコン）だけ使えます。
(exit 1)
$ webapp.exe（APP_ENV=staging で MAIL_REDIRECT_TO なし）
エラー: 設定の間違い: APP_ENV=staging のときは MAIL_REDIRECT_TO が要ります
(exit 1)
```

9. 毎朝 8:00 の通知（jobs.rs）



`src/jobs.rs`：

コピー

```
//! 毎朝 8:00 の通知（onprem の 04 の 12.1）
//!
//! 1. 承認のお願いを送ってから 3 日たっても止まっている回覧 → 承認者に送り直す（前の URL は使えなくなる）
//! 2. 希望の納期まで 3 日以内なのに、まだ終わっていない依頼 → 依頼者と担当者に知らせる
//!
//! Django 版は、Windows のタスク スケジューラから動かした。Topcoat 版は **アプリの中で** 動かす:
//! 1 分ごとに「8:00 を過ぎたか・今日の分は済んだか」を見る。8:00 にアプリが止まっていても、
//! 起動したあとの最初の確かめで、今日の分を動かす（`job_runs` の表で、同じ日に 2 回は動かない）。

use std::{sync::Arc, time::Duration};

use time::{OffsetDateTime, Time};
use topcoat::context::{AppContext, Cx};

use crate::{
    mail::Outbox,
    models::{JST, Status, fmt_date, today_jst},
    state::state,
    workflow::{self, WfResult, WorkflowError},
};

const JOB: &str = "daily-reminders";
pub const STALE_DAYS: i32 = 3;
pub const DUE_SOON_DAYS: i64 = 3;
/// 何時に動かすか（日本の時刻）
pub const RUN_AT: Time = time::macros::time!(8:00);

/// 納期が近い依頼の 1 行
#[derive(sqlx::FromRow)]
struct DueSoon {
    title: String,
    due_date: time::Date,
    #[sqlx(try_from = "String")]
    status: Status,
    requester_email: String,
    assignee_email: Option<String>,
    assignee_name: Option<String>,
}

#[derive(Debug, PartialEq, Eq)]
pub struct Report {
    pub resent: usize,
    pub due_soon: usize,
}

/// 通知を 1 回分おこなう（コマンドの send-daily-reminders からも呼ぶ）
pub async fn daily_reminders(cx: &Cx) -> WfResult<Report> {
    let pool = &state(cx).pool;

    // 1. 止まっている承認
    let stale: Vec<i64> = sqlx::query_scalar(
        "SELECT s.id FROM steps s JOIN requests r ON r.id = s.request_id
          WHERE s.decision = 'waiting' AND r.status = 'circulating'
            AND s.sent_at < now() - make_interval(days => $1)
          ORDER BY s.id",
    )
    .bind(STALE_DAYS)
    .fetch_all(pool)
    .await?;
    let mut resent = 0;
    for step_id in stale {
        match workflow::resend(cx, None, step_id).await {
            Ok(()) => resent += 1,
            // 調べてから送るまでの間に、承認されたなど。飛ばして次へ
            Err(WorkflowError::Rule(m)) => eprintln!("送り直しを飛ばしました（回覧 {step_id}）: {m}"),
            Err(e) => return Err(e),
        }
    }

    // 2. 納期が近い依頼
    let limit = today_jst() + time::Duration::days(DUE_SOON_DAYS);
    let rows: Vec<DueSoon> = sqlx::query_as(
        "SELECT r.title, r.due_date, r.status, q.email AS requester_email, w.email AS assignee_email, w.name AS assignee_name
           FROM requests r
           JOIN employees q ON q.id = r.requester_id
           LEFT JOIN employees w ON w.id = r.assignee_id
          WHERE r.status IN ('approved', 'in_progress') AND r.due_date <= $1
          ORDER BY r.due_date, r.id",
    )
    .bind(limit)
    .fetch_all(pool)
```

```
    .await?;
    let mut outbox = Outbox::new();
    for r in &rows {
        let assignee = r.assignee_name.as_ref().map(|n| format!("（担当: {n}）")).unwrap_or_default();
        let body =
            format!("依頼「{}」の希望の納期は {} です。\n今の状態: {}{assignee}\n", r.title, fmt_date(r.due_date), r.status.label());
        let mut to = vec![r.requester_email.clone()];
        if let Some(a) = r.assignee_email.as_ref().filter(|a| **a != r.requester_email) {
            to.push(a.clone()); // 依頼者と担当者が同じ人なら、1 通だけ
        }
        for t in to {
            outbox.add(&t, &format!("納期が近づいています: {}", r.title), body.clone());
        }
    }
    outbox.send_all(cx).await;
    Ok(Report { resent, due_soon: rows.len() })
}

/// 8:00 を過ぎていて、今日の分がまだなら動かす。動かしたら Some
pub async fn run_daily_if_due(cx: &Cx, now: OffsetDateTime) -> WfResult<Option<Report>> {
    let now = now.to_offset(JST);
    if now.time() < RUN_AT {
        return Ok(None);
    }
    let pool = &state(cx).pool;
    // 「今日の分は自分がやる」と、先に 1 行入れる。もう入っていれば（済んでいれば）何もしない
    let claimed = sqlx::query("INSERT INTO job_runs (job, run_date) VALUES ($1, $2) ON CONFLICT DO NOTHING")
        .bind(JOB)
        .bind(now.date())
        .execute(pool)
        .await?
        .rows_affected()
        == 1;
    if !claimed {
        return Ok(None);
    }
    match daily_reminders(cx).await {
        Ok(report) => Ok(Some(report)),
        Err(e) => {
            // 失敗したら、印を消して、次の確かめ（1 分後）でやり直す
            sqlx::query("DELETE FROM job_runs WHERE job = $1 AND run_date = $2").bind(JOB).bind(now.date()).execute(pool).await?;
            Err(e)
        }
    }
}

/// サーバーの中で、1 分ごとに確かめ続ける（serve のときに 1 回だけ呼ぶ）
pub fn spawn_daily(app: Arc<AppContext>) {
    tokio::spawn(async move {
        let cx = Cx::new(app); // 画面のリクエストの外で、メールの設定などを使うための Cx
        loop {
            match run_daily_if_due(&cx, OffsetDateTime::now_utc()).await {
                Ok(Some(r)) => println!("毎朝の通知: 送り直した承認のお願い {} 件・納期が近い依頼 {} 件", r.resent, r.due_soon),
                Ok(None) => {}
                Err(e) => eprintln!("毎朝の通知に失敗しました（1 分後にやり直します）: {e}"),
            }
            tokio::time::sleep(Duration::from_secs(60)).await;
        }
    });
}
```

▶ ここで初めて出てくるコード

- **`time!`**（時間）：`time` の部品で、時刻（`Time`）を書くマクロ。`time!(8:00)` は 8 時 0 分。まちがった時刻（`25:00` など）は、プログラムを作る時にエラーになります。
- **`loop`**（くり返し）：終わりのないくり返し。`loop { … }` の中で `break` を書いた所で抜けます。サーバーが「次の接続を待つ」のをずっと続ける時などに使います。
- **`.time()`**（時間）：日時（`OffsetDateTime`）から、時刻（`Time`。時・分・秒）だけを取り出すメソッド。`now.time() < RUN_AT` は「今の時刻が、決めた時刻（8:00）より前か」です。
- **`time::Duration::days()`**（時間）：`time` の、日数ぶんの長さ（`Duration`）を作る関数。`今日 + time::Duration::days(3)` は「3 日後」です。
- **`tokio::spawn()`**（非同期）：非同期の仕事を、別のタスクとしてすぐ動かし始める関数。`tokio::spawn(async move { … })`。返ってくる物（`JoinHandle`）を `.await` すると、結果を受け取れます。
- **`tokio::time::sleep()`**（非同期）：決めた時間だけ待つ、非同期の関数。`tokio::time::sleep(Duration::from_millis(300)).await`。待っている間、ほかの仕事は止まりません（`std::thread::sleep` はスレッドごと止めてしまうので、`async` の中では使いません）。
- **`Time`**（時間）：`time` の、時刻（時・分・秒）を表す型。`time!(8:00)` で「8 時 0 分」を作れます。
- **`ORDER`**（SQL）：`ORDER BY 列` で、取り出した行を並べる書き方。`DESC` を付けると大きい順（新しい順）です。
- **`BY`**（SQL）：`ORDER BY`（並べる）・`GROUP BY`（まとめる）の一部で、「〜で」という意味です。
- **`LEFT`**（SQL）：`LEFT JOIN` は、左の表の行は、相手が無くても全部残す JOIN。「ノートが 0 件の人も一覧に出す」時に使います。

- 06 は、タスク スケジューラが毎朝 `manage.ps1` を動かしました。08 は onprem の 04 の 12.1 のとおり、**アプリの中で動かします**。
- 「8:00 ちょうど」を待つのではなく、**1 分ごとに「8:00 を過ぎたか・今日の分は済んだか」を見ます**。こうすると、8:00 にアプリが止まっていても、起動したあとの最初の確かめで、今日の分が動きます。
- 「今日の分は自分がやる」と `job_runs` に先に 1 行入れ（`ON CONFLICT DO NOTHING`）、入れられたときだけ動かします。同じ日に 2 回は送りません。失敗したら印を消し、1 分後にやり直します（図では分かりやすく「送ってから印」と描きましたが、実際は**先に印を入れ、失敗したら消す**順です。サーバーを 2 つ動かしても、片方だけが送るようにするため）。
- 手で 1 回動かしたいときは、`webapp.exe send-daily-reminders` です（8.3 の出力）。

10. 画面から呼ぶ、読むだけの SQL（queries.rs）

`src/queries.rs`：

コピー

```
//! 画面に出すために読むだけの SQL（状態は変えない）
//!
//! 「だれが、どの依頼を見られるか」の決まりも、ここに置く：
//! 担当者・管理者は全部。一般の人は、自分が出した依頼と、自分が回覧者に入っている依頼だけ。

use sqlx::PgPool;

use crate::models::{AttachmentRow, Employee, HistoryRow, RequestDetail, RequestRow, StepRow};

/// 一覧。`mine_only` なら、自分が出した依頼だけ
pub async fn list_requests(pool: &PgPool, user: &Employee, mine_only: bool) -> sqlx::Result<Vec<RequestRow>> {
    sqlx::query_as(
        "SELECT r.id, r.title, r.due_date, r.status, q.name AS requester_name,
                (SELECT a.name FROM steps s JOIN employees a ON a.id = s.approver_id
                  WHERE s.request_id = r.id AND s.round = r.round AND s.decision = 'waiting'
                  ORDER BY s.step_order LIMIT 1) AS waiting_for,
                r.created_at
           FROM requests r
           JOIN employees q ON q.id = r.requester_id
          WHERE ($1 OR r.requester_id = $2
                 OR EXISTS (SELECT 1 FROM steps s WHERE s.request_id = r.id AND s.approver_id = $2))
            AND (NOT $3 OR r.requester_id = $2)
          ORDER BY r.created_at DESC
          LIMIT 200",
    )
    .bind(user.role.is_staff())
    .bind(user.id)
    .bind(mine_only)
    .fetch_all(pool)
    .await
}

/// 依頼 1 件と、その回覧・添付・記録
pub struct FullRequest {
    pub request: RequestDetail,
    pub steps: Vec<StepRow>,
    pub attachments: Vec<AttachmentRow>,
    pub histories: Vec<HistoryRow>,
}

impl FullRequest {
    /// この人が見てよいか
    pub fn visible_to(&self, user: &Employee) -> bool {
        user.role.is_staff() || self.request.requester_id == user.id || self.steps.iter().any(|s| s.approver_id == user.id)
    }

    /// 今の回の回覧だけ
    pub fn current_steps(&self) -> impl Iterator<Item = &StepRow> {
        self.steps.iter().filter(|s| s.round == self.request.round)
    }
}

pub async fn find_request(pool: &PgPool, id: i64) -> sqlx::Result<Option<FullRequest>> {
    let request: Option<RequestDetail> = sqlx::query_as(
        "SELECT r.id, r.title, r.detail, r.due_date, r.status,
                r.requester_id, q.name AS requester_name, q.dept AS requester_dept,
                r.assignee_id, w.name AS assignee_name,
                r.drawing, r.return_comments, r.round, r.created_at
           FROM requests r
           JOIN employees q ON q.id = r.requester_id
           LEFT JOIN employees w ON w.id = r.assignee_id
          WHERE r.id = $1",
    )
    .bind(id)
    .fetch_optional(pool)
    .await?;
    let Some(request) = request else { return Ok(None) };

    let steps = sqlx::query_as(
        "SELECT s.id, s.round, s.step_order, s.approver_id, a.name AS approver_name, a.dept AS approver_dept,
                s.decision, s.decided_at
           FROM steps s JOIN employees a ON a.id = s.approver_id
          WHERE s.request_id = $1
          ORDER BY s.round DESC, s.step_order",
    )
```

```
    .bind(id)
    .fetch_all(pool)
    .await?;
    let attachments = attachments_of(pool, id).await?;
    let histories = sqlx::query_as(
        "SELECT h.at, e.name AS actor_name, h.action, h.note
           FROM histories h LEFT JOIN employees e ON e.id = h.actor_id
          WHERE h.request_id = $1
          ORDER BY h.at, h.id",
    )
    .bind(id)
    .fetch_all(pool)
    .await?;
    Ok(Some(FullRequest { request, steps, attachments, histories }))
}

pub async fn attachments_of(pool: &PgPool, request_id: i64) -> sqlx::Result<Vec<AttachmentRow>> {
    sqlx::query_as(
        "SELECT id, request_id, stored_name, content_type, original_name, size, uploaded_at
           FROM attachments WHERE request_id = $1 ORDER BY id",
    )
    .bind(request_id)
    .fetch_all(pool)
    .await
}

pub async fn find_attachment(pool: &PgPool, id: i64) -> sqlx::Result<Option<AttachmentRow>> {
    sqlx::query_as(
        "SELECT id, request_id, stored_name, content_type, original_name, size, uploaded_at
           FROM attachments WHERE id = $1",
    )
    .bind(id)
    .fetch_optional(pool)
    .await
}

/// 回覧者に選べる人（自分以外の、いる社員）
pub async fn approver_choices(pool: &PgPool, user: &Employee) -> sqlx::Result<Vec<Employee>> {
    sqlx::query_as(
        "SELECT id, employee_no, name, dept, job_title, email, role
           FROM employees WHERE is_active AND id <> $1
          ORDER BY dept, employee_no",
    )
    .bind(user.id)
    .fetch_all(pool)
    .await
}

/// 集計: 状態ごとの件数と、担当者ごとの完了の件数
pub struct Stats {
    pub by_status: Vec<(String, i64)>,
    pub done_by_assignee: Vec<(String, i64)>,
}

pub async fn stats(pool: &PgPool) -> sqlx::Result<Stats> {
    let by_status = sqlx::query_as("SELECT status, count(*) FROM requests GROUP BY status").fetch_all(pool).await?;
    let done_by_assignee = sqlx::query_as(
        "SELECT w.name, count(*) FROM requests r JOIN employees w ON w.id = r.assignee_id
          WHERE r.status = 'done' GROUP BY w.name ORDER BY count(*) DESC, w.name",
    )
    .fetch_all(pool)
    .await?;
    Ok(Stats { by_status, done_by_assignee })
}
```

▶ ここで初めて出てくるコード

- **`LIMIT`**（SQL）：`LIMIT 件数` で、取り出す行の数の上限を決める書き方。`ORDER BY … LIMIT 20` は「並べて、先頭の 20 件だけ」。
- **`OR`**（SQL）：SQL で「または」。`WHERE a = 1 OR b = 2` は、どちらかに合う行です。`AND` と混ぜる時は、`( )` で囲んで順番をはっきりさせます。
- **`EXISTS`**（SQL）：SQL で「ある」。`IF NOT EXISTS` は「無ければ」という意味です。
- **`DESC`**（SQL）：`ORDER BY 列 DESC` で、大きい順（新しい順）に並べます（小さい順は `ASC`）。
- **`Iterator`**（イテレーター）：「1 つずつ取り出せる物」（イテレーター）の機能（トレイト）。`map`・`filter`・`sum` などのメソッドは、全部この機能に付いています。`impl Iterator<Item = &Note>` は「`&Note` を 1 つずつ出す何か」を返す、という戻り値の書き方です。
- **`Item`**（イテレーター）：イテレーターが 1 つずつ取り出す物の型（関連型）。`IntoIterator<Item = T>` は「`T` を 1 つずつ取り出せる物なら何でもよい」という意味です。
- **`GROUP`**（SQL）：`GROUP BY 列` で、同じ値の行をまとめて、まとめごとに数える・足す時の書き方。`SELECT user_id, COUNT(*) FROM notes GROUP BY user_id` は、人ごとのノートの数です。

- **値は必ず `$1`・`$2` と `.bind()` で渡します。**SQL の文と値が別々にデータベースへ届くので、値に `'` や `;` が入っていても、SQL として動くことはありません（SQL インジェクションを防ぐ）。
- 「だれが、どの依頼を見られるか」の決まりも、ここに置きました（担当者・管理者は全部。一般の人は、自分が出した依頼と、自分が回覧者に入っている依頼だけ）。06 の `_visible_requests` と同じです。

11. ここまでの部品を 1 つにまとめる（lib.rs・main.rs・state.rs）

`src/lib.rs`：

コピー

```
//! 依頼・承認アプリ（Rust＋Topcoat 版の練習）。06 の Django＋React 版と、同じことをする
//!
//! | ファイル | 役目 |
//! |---|---|
//! | config.rs | 設定（環境変数）を読む |
//! | state.rs | データベースの接続と、アプリ全体で 1 つの物 |
//! | models.rs | データの形・状態の名前・日時の表し方 |
//! | auth.rs / http_cookie.rs | ログイン・ログアウト・セッション |
//! | workflow.rs | 依頼の状態を変える処理（いちばん大事） |
//! | queries.rs | 画面に出すために読むだけの SQL |
//! | mail.rs | メール（保存が確定してから送る・検証では宛先を差し替える） |
//! | upload.rs / files.rs | 添付ファイルの確かめ・保存・ダウンロード |
//! | drawing.rs | 図面の JSON |
//! | employees.rs | 社員の CSV の取り込み |
//! | jobs.rs | 毎朝 8:00 の通知 |
//! | request_form.rs / views.rs | 画面の部品 |
//! | app.rs / app/ | 画面（URL ごと） |
//! | cli.rs | exe のコマンド |

pub mod app;
pub mod auth;
pub mod cli;
pub mod config;
pub mod drawing;
pub mod employees;
pub mod files;
pub mod http_cookie;
pub mod jobs;
pub mod mail;
pub mod models;
pub mod queries;
pub mod request_form;
pub mod state;
pub mod upload;
pub mod views;
pub mod workflow;
```

`src/main.rs`:

コピー

```
//! webapp.exe の入り口。中身は lib.rs の側にある（テストからも使えるように）

use std::process::ExitCode;

#[tokio::main]
async fn main() -> ExitCode {
    let args: Vec<String> = std::env::args().skip(1).collect();
    match webapp::cli::run(args).await {
        Ok(()) => ExitCode::SUCCESS,
        Err(e) => {
            // {:#} で、原因の連なり（「つなげません: 〜」）まで出す
            eprintln!("エラー: {e:#}");
            ExitCode::FAILURE
        }
    }
}
```

▶ ここで初めて出てくるコード

- **`#[tokio]`**（非同期）: `#[tokio::main]` は、`main` を `async` にして、tokio（非同期の仕事を回す仕組み）の上で動かす印（属性）。`#[tokio::test]` は、テストを `async` で書けるようにする印です。
- **`.skip()`**（イテレーター）: 並びの先頭から、決めた数だけ飛ばすメソッド。`env::args().skip(1)` は、0番目（プログラム自身）を飛ばします。
- **`std::env::args()`**（プロセス）: プログラムに渡された引数を取り出す関数（`env::args()` と同じ）。
- **`ExitCode`**（プロセス）: `main` が返す、終了コードの型。`fn main() -> ExitCode` にして、`ExitCode::SUCCESS`（0）か `ExitCode::FAILURE`（1）を返します。`process::exit` と違い、片づけをしてから終われます。
- **`SUCCESS`**（プロセス）: `ExitCode::SUCCESS` は、終了コード 0（成功）です。
- **`FAILURE`**（プロセス）: `ExitCode::FAILURE` は、終了コード 1（失敗）です。

`src/state.rs`:

コピー

```
//! アプリ全体で 1 つだけ持つ物（データベースの接続のたまり場と、設定）
//!
//! ルーターに `.app_context(AppState { .. })` で渡し、どのページからも `state(cx)` で取り出す。

use std::sync::Arc;

use sqlx::{PgPool, postgres::PgPoolOptions};
use topcoat::context::{Cx, app_context};

use crate::config::Config;

#[derive(Clone)]
pub struct AppState {
    /// 接続のたまり場。使うたびに、空いている接続を 1 つ借りて、返す
    pub pool: PgPool,
    pub config: Arc<Config>,
}

/// どのページ・どの関数からでも、アプリの共有の物を取り出す
pub fn state(cx: &Cx) -> &AppState {
    app_context::<AppState>(cx)
}

/// PostgreSQL につなぐ。つながらなければ、5 秒であきらめてエラーにする
pub async fn connect(database_url: &str) -> Result<PgPool, sqlx::Error> {
    PgPoolOptions::new()
        .max_connections(10)
        .acquire_timeout(std::time::Duration::from_secs(5))
        .connect(database_url)
        .await
}

/// migrations フォルダの SQL のうち、まだ流していない物を、番号の順に流す
pub async fn migrate(pool: &PgPool) -> Result<(), sqlx::migrate::MigrateError> {
    sqlx::migrate!("./migrations").run(pool).await
}
```

▶ ここで初めて出てくるコード

- **`migrate!`**（データベース）：`sqlx` の部品で、`migrations` フォルダの SQL のファイル（テーブルを作る・変える手順）を、プログラムの中に取り込むマクロ。`sqlx::migrate!("./migrations").run(pool).await` で、まだ流していない物を順に流します。
- **`.max_connections()`**（データベース）：プールに用意する接続の、いちばん多い数を決めるメソッド。
- **`.acquire_timeout()`**（データベース）：接続を借りる時に、空くのを待つ長さの上限を決めるメソッド。これを過ぎるとエラーにします（ずっと待ち続けない）。
- **`PgPoolOptions::new()`**（データベース）：`PgPoolOptions` を、決まりの設定で作る関数。`.max_connections(10)`・`.acquire_timeout(…)` をつなげて決め、最後に `.connect(URL)` でプールを作ります。
- **`std::time::Duration::from_secs()`**（時間）：秒数ぶんの長さ（`Duration`）を作る関数。`Duration::from_secs(5)` は 5 秒です。
- **`PgPoolOptions`**（データベース）：`sqlx` の、PostgreSQL の接続の集まり（プール）の設定を組み立てる型。
- **`MigrateError`**（データベース）：`sqlx` の、マイグレーションに失敗した時のエラーの型。

- 中身を `lib.rs` の側に置き、`main.rs` は入り口だけにしました。こうすると、`tests/` のテストから、中の関数を直接呼べます。
- `AppState`（接続のたまり場と設定）は、ルーターに 1 回渡せば、どのページからも `state(cx)` で取り出せます（07 の `app_context` と同じ）。

12. よくある失敗（前編）

起きたこと	原因	直し方
起動すると「DATABASE_URL がありません」	`.env` が無い・名前が違う	`.env.example` を `.env` にコピーする
起動すると「データベースにつなげません」	PostgreSQL が止まっている・パスワードが違う	2.1 のユーザーとパスワードを確かめる
起動すると「マイグレーションに失敗しました」	流したあとの `0001_init.sql` を書きかえた	書きかえを戻し、`0002_….sql` を足す
`error[E0277]: dynamic SQL strings should be audited …`	組み立てた SQL の文字を、そのまま渡した	値は `.bind()` で渡す。決まった文字だけを足したなら `AssertSqlSafe` で包む（6.5）
CSV が「UTF-8 ではありません」で断られる	Excel で「CSV（コンマ区切り）」で保存した（Shift_JIS）	「CSV UTF-8（コンマ区切り）」で保存し直す
承認のお願いが届かない（開発のパソコン）	開発のパソコンでは送らない	`sent_mail\` の `.eml` を開く
検証のサーバーが起動しない	`MAIL_REDIRECT_TO` が無い	XML に書く（3 章）
`cargo test` が `failed to connect to setup test database` と `permission denied to create database` で止まる	テストのユーザーに `CREATEDB` が無い	2.1 の `CREATEDB`（開発のパソコンだけ）

13. 自動のテスト

`tests/workflow.rs`：

コピー

```
//! 依頼の流れのテスト（06 の Django 版の tests.py と同じ決まりを、Rust 版でも確かめる）
//!
//! `#[sqlx::test]` は、テストごとに空のデータベースを作り、migrations を流してから、`pool` を渡してくれる。
//! 動かし方: DATABASE_URL に、データベースを作ってよいユーザーの接続先を入れて `cargo test`

use std::{path::PathBuf, sync::Arc};

use sqlx::PgPool;
use time::macros::{date, datetime};
use topcoat::{
    context::{Cx, CxTestBuilder},
    mail::{MailConfig, MemoryTransport, TextBody},
};
use webapp::{
    config::{AppEnv, Config, MailSettings},
    jobs,
    models::{Employee, Status},
    state::AppState,
    upload::UploadedFile,
    workflow::{self, RequestInput, WorkflowError},
};

/// テストの舞台: 社員 4 人と、送ったメールをためる入れ物
struct World {
    cx: Cx,
    sent: MemoryTransport,
    pool: PgPool,
    file_dir: PathBuf,
    requester: Employee,
    a1: Employee,
    a2: Employee,
    worker: Employee,
}

async fn add_employee(pool: &PgPool, no: &str, name: &str, email: &str, role: &str) -> Employee {
    sqlx::query_as(
        "INSERT INTO employees (employee_no, name, dept, email, role) VALUES ($1, $2, '設計部', $3, $4)
         RETURNING id, employee_no, name, dept, job_title, email, role",
    )
    .bind(no)
    .bind(name)
    .bind(email)
    .bind(role)
    .fetch_one(pool)
    .await
    .unwrap()
}

async fn world(pool: PgPool, redirect: Option<&str>) -> World {
    let requester = add_employee(&pool, "000001", "依頼 一郎", "req@example.com", "member").await;
    let a1 = add_employee(&pool, "000002", "承認 一子", "a1@example.com", "member").await;
    let a2 = add_employee(&pool, "000003", "承認 二郎", "a2@example.com", "member").await;
    let worker = add_employee(&pool, "000004", "担当 三郎", "w@example.com", "worker").await;

    let mut random = [0u8; 8];
    getrandom::fill(&mut random).unwrap();
    let file_dir = std::env::temp_dir().join(format!("webapp-test-{}", hex::encode(random)));
    let config = Config {
        app_env: AppEnv::Development,
        base_url: "http://webapp.example.local".into(),
        database_url: String::new(),
        file_dir: file_dir.clone(),
        mail: MailSettings::File("unused".into()),
        mail_from: "noreply@example.com".into(),
        mail_redirect_to: redirect.map(Into::into),
    };
    // メールは送らずに、メモリーにためる（あとで中身を確かめる）
    let sent = MemoryTransport::new();
    let cx = CxTestBuilder::new()
        .app_context(AppState { pool: pool.clone(), config: Arc::new(config) })
        .app_context(MailConfig::builder().transport(sent.clone()).build())
        .build();
    World { cx, sent, pool, file_dir, requester, a1, a2, worker }
}

/// 送ったメールの（宛先, 件名, 本文）
fn mails(w: &World) -> Vec<(String, String, String)> {
    w.sent
        .sent()
        .iter()
```

```
        .map(|m| {
            let body = match m.text() {
                TextBody::Text(t) => t.clone(),
                _ => String::new(),
            };
            (m.to()[0].address().to_string(), m.subject().to_string(), body)
        })
        .collect()
}

/// メールの本文から、承認の URL の文字列を取り出す
fn token_in(body: &str) -> String {
    body.split("/approve/").nth(1).expect("URL がありません").lines().next().unwrap().trim().to_string()
}

fn input(approvers: &[&Employee]) -> RequestInput {
    RequestInput {
        title: "治具の製作".into(),
        detail: "検査用".into(),
        due_date: date!(2026 - 11 - 30),
        approver_ids: approvers.iter().map(|a| a.id).collect(),
        drawing: None,
        files: Vec::new(),
    }
}

async fn submit(w: &World) -> i64 {
    workflow::submit(&w.cx, &w.requester, input(&[&w.a1, &w.a2])).await.unwrap()
}

async fn status(w: &World, id: i64) -> Status {
    let s: String = sqlx::query_scalar("SELECT status FROM requests WHERE id = $1").bind(id).fetch_one(&w.pool).await.unwrap();
    Status::try_from(s).unwrap()
}

fn rule_message<T: std::fmt::Debug>(r: Result<T, WorkflowError>) -> String {
    match r {
        Err(WorkflowError::Rule(m)) => m,
        other => panic!("Rule のエラーのはずが {other:?}"),
    }
}

#[sqlx::test]
async fn 出すと1人目にメールが届き回覧中になる(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    assert_eq!(status(&w, id).await, Status::Circulating);
    let m = mails(&w);
    assert_eq!(m.len(), 1);
    assert_eq!(m[0].0, "a1@example.com");
    assert_eq!(m[0].1, "【依頼・承認】承認のお願い: 治具の製作");
    assert!(m[0].2.contains("http://webapp.example.local/approve/"));
}

#[sqlx::test]
async fn 全員が承認すると回覧完了になり担当者に知らせる(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    workflow::approve(&w.cx, &token_in(&mails(&w)[0].2)).await.unwrap();
    assert_eq!(mails(&w)[1].0, "a2@example.com"); // 2 人目へ
    workflow::approve(&w.cx, &token_in(&mails(&w)[1].2)).await.unwrap();
    assert_eq!(status(&w, id).await, Status::Approved);
    let mut to: Vec<String> = mails(&w)[2..].iter().map(|m| m.0.clone()).collect();
    to.sort();
    assert_eq!(to, ["req@example.com", "w@example.com"]);
}

#[sqlx::test]
async fn urlは1回しか使えない(pool: PgPool) {
    let w = world(pool, None).await;
    submit(&w).await;
    let token = token_in(&mails(&w)[0].2);
    workflow::approve(&w.cx, &token).await.unwrap();
    assert_eq!(rule_message(workflow::approve(&w.cx, &token).await), "この依頼は、承認済みです。");
}

#[sqlx::test]
async fn 同時に2回押しても1回だけ進む(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    let token = token_in(&mails(&w)[0].2);
    // 2 つの承認を、同時に走らせる。鍵（FOR UPDATE）のおかげで、あとの方は「承認済み」になる
    let (r1, r2) = tokio::join!(workflow::approve(&w.cx, &token), workflow::approve(&w.cx, &token));
```

```
    assert_eq!([r1.is_ok(), r2.is_ok()].iter().filter(|ok| **ok).count(), 1);
    let approved: i64 = sqlx::query_scalar("SELECT count(*) FROM steps WHERE request_id = $1 AND decision = 'approved'")
        .bind(id)
        .fetch_one(&w.pool)
        .await
        .unwrap();
    assert_eq!(approved, 1);
    assert_eq!(mails(&w).len(), 2); // 1 人目へ 1 通・2 人目へ 1 通（2 人目へ 2 通にはならない）
}

#[sqlx::test]
async fn 期限が切れたurlは使えない(pool: PgPool) {
    let w = world(pool, None).await;
    submit(&w).await;
    sqlx::query("UPDATE steps SET token_expires_at = now() - interval '1 second'").execute(&w.pool).await.unwrap();
    let m = rule_message(workflow::approve(&w.cx, &token_in(&mails(&w)[0].2)).await);
    assert!(m.contains("期限が切れています"), "{m}");
}

#[sqlx::test]
async fn 差し戻すとコメントが追記され依頼者にメールが届く(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    workflow::return_request(&w.cx, &token_in(&mails(&w)[0].2), "寸法を追記してください").await.unwrap();
    assert_eq!(status(&w, id).await, Status::Returned);
    let comments: String =
        sqlx::query_scalar("SELECT return_comments FROM requests WHERE id = $1").bind(id).fetch_one(&w.pool).await.unwrap();
    //  【11/02 09:00 回覧者1 承認 一子】寸法を追記してください（日時の所は、動かした時刻になる）
    assert!(comments.starts_with('【'), "{comments}");
    assert!(comments.ends_with(" 回覧者1 承認 一子】寸法を追記してください"), "{comments}");
    assert_eq!(mails(&w).last().unwrap().0, "req@example.com");
}

#[sqlx::test]
async fn 理由の無い差し戻しは断る(pool: PgPool) {
    let w = world(pool, None).await;
    submit(&w).await;
    let m = rule_message(workflow::return_request(&w.cx, &token_in(&mails(&w)[0].2), "  ").await);
    assert_eq!(m, "差し戻すときは、理由を書いてください。");
}

#[sqlx::test]
async fn 引き戻すと承認待ちの人のurlが使えなくなる(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    let token = token_in(&mails(&w)[0].2);
    workflow::pull_back(&w.cx, &w.requester, id).await.unwrap();
    let mut conn = w.pool.acquire().await.unwrap();
    assert_eq!(rule_message(workflow::check_token(&mut conn, &token, false).await), "依頼者により引き戻されました。");
}

#[sqlx::test]
async fn 出し直すと最初の回覧者からやり直す(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    workflow::approve(&w.cx, &token_in(&mails(&w)[0].2)).await.unwrap(); // 1 人目は承認済み
    workflow::return_request(&w.cx, &token_in(&mails(&w)[1].2), "図面を直して").await.unwrap();
    let fixed = RequestInput { title: "治具の製作（直し）".into(), due_date: date!(2026 - 12 - 05), ..input(&[&w.a1, &w.a2]) };
    workflow::resubmit(&w.cx, &w.requester, id, fixed).await.unwrap();
    let (s, round): (String, i16) =
        sqlx::query_as("SELECT status, round FROM requests WHERE id = $1").bind(id).fetch_one(&w.pool).await.unwrap();
    assert_eq!((s.as_str(), round), ("circulating", 2));
    assert_eq!(mails(&w).last().unwrap().0, "a1@example.com");
}

#[sqlx::test]
async fn ほかの人の依頼は引き戻せない(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    assert_eq!(rule_message(workflow::pull_back(&w.cx, &w.a1, id).await), "自分の依頼ではありません。");
}

#[sqlx::test]
async fn 回覧者の決まり(pool: PgPool) {
    let w = world(pool, None).await;
    let twice = workflow::submit(&w.cx, &w.requester, input(&[&w.a1, &w.a1])).await;
    assert_eq!(rule_message(twice), "同じ人を 2 回は選べません。");
    let me = workflow::submit(&w.cx, &w.requester, input(&[&w.requester])).await;
    assert_eq!(rule_message(me), "自分自身は回覧者にできません。");
    let four = workflow::submit(&w.cx, &w.requester, input(&[&w.a1, &w.a2, &w.worker, &w.worker])).await;
    assert_eq!(rule_message(four), "回覧者は 1〜3 人にしてください。");
    assert!(mails(&w).is_empty()); // 断ったときは、メールも送らない
}
```

```
#[sqlx::test]
async fn 検証ではメールの宛先を書きかえる(pool: PgPool) {
    let w = world(pool, Some("it@example.com")).await;
    submit(&w).await;
    let m = &mails(&w)[0];
    assert_eq!(m.0, "it@example.com");
    assert!(m.1.starts_with("【検証】"));
    assert!(m.2.contains("本来の宛先: a1@example.com"));
}

#[sqlx::test]
async fn 担当を決めて完了にすると依頼者に知らせる(pool: PgPool) {
    let w = world(pool, None).await;
    let id = submit(&w).await;
    workflow::approve(&w.cx, &token_in(&mails(&w)[0].2)).await.unwrap();
    workflow::approve(&w.cx, &token_in(&mails(&w)[1].2)).await.unwrap();
    // 一般の人は担当になれない
    assert_eq!(rule_message(workflow::assign(&w.cx, &w.a1, id).await), "担当者だけができます。");
    workflow::assign(&w.cx, &w.worker, id).await.unwrap();
    workflow::complete(&w.cx, &w.worker, id).await.unwrap();
    assert_eq!(status(&w, id).await, Status::Done);
    let last = mails(&w).last().unwrap().clone();
    assert_eq!((last.0.as_str(), last.1.as_str()), ("req@example.com", "【依頼・承認】完了: 治具の製作"));
}

#[sqlx::test]
async fn 添付ファイルはフォルダに置き_失敗したら消す(pool: PgPool) {
    let w = world(pool, None).await;
    let png = UploadedFile { name: "写真.png".into(), bytes: b"\x89PNG\r\n\x1a\n....".to_vec() };
    let ok = RequestInput { files: vec![png.clone()], ..input(&[&w.a1]) };
    let id = workflow::submit(&w.cx, &w.requester, ok).await.unwrap();
    let stored: String =
        sqlx::query_scalar("SELECT stored_name FROM attachments WHERE request_id = $1").bind(id).fetch_one(&w.pool).await.unwrap();
    let dir = w.file_dir.join("attachments");
    assert!(dir.join(&stored).exists());

    // 回覧者の決まりで断られたら、書いたファイルも残さない
    let bad = RequestInput { files: vec![png], ..input(&[&w.requester]) };
    assert!(workflow::submit(&w.cx, &w.requester, bad).await.is_err());
    assert_eq!(std::fs::read_dir(&dir).unwrap().count(), 1);
    let _ = std::fs::remove_dir_all(&w.file_dir);
}

#[sqlx::test]
async fn 毎朝の通知は止まっている承認を送り直す(pool: PgPool) {
    let w = world(pool, None).await;
    submit(&w).await;
    let old = token_in(&mails(&w)[0].2);
    // 4 日前に送ったことにする
    sqlx::query("UPDATE steps SET sent_at = now() - interval '4 days' WHERE sent_at IS NOT NULL").execute(&w.pool).await.unwrap();
    let report = jobs::daily_reminders(&w.cx).await.unwrap();
    assert_eq!(report, jobs::Report { resent: 1, due_soon: 0 });
    let m = mails(&w);
    assert_eq!(m[1].0, "a1@example.com");
    // 前の URL は使えなくなり、新しい URL で承認できる
    assert_eq!(rule_message(workflow::approve(&w.cx, &old).await), "この URL は使えません。");
    workflow::approve(&w.cx, &token_in(&m[1].2)).await.unwrap();
}

#[sqlx::test]
async fn 毎朝の通知は8時を過ぎてから1日1回だけ動く(pool: PgPool) {
    let w = world(pool, None).await;
    assert!(jobs::run_daily_if_due(&w.cx, datetime!(2026-11-02 07:59 +9)).await.unwrap().is_none());
    assert!(jobs::run_daily_if_due(&w.cx, datetime!(2026-11-02 08:00 +9)).await.unwrap().is_some());
    assert!(jobs::run_daily_if_due(&w.cx, datetime!(2026-11-02 13:00 +9)).await.unwrap().is_none()); // 同じ日の 2 回目
    assert!(jobs::run_daily_if_due(&w.cx, datetime!(2026-11-03 08:01 +9)).await.unwrap().is_some()); // 次の日
}
```

▶ ここで初めて出てくるコード

- **`date!`**（時間）：`time` の部品で、日付（`Date`）を書くマクロ。`date!(2026 - 11 - 30)` は 2026 年 11 月 30 日。テストで、決まった日付を作る時に使います。
- **`panic!`**（エラー処理）：その場でプログラム（今のスレッド）を止めるマクロ。`panic!("起きるはずのないことが起きました")`。直しようのない、プログラムのまちがいを知らせる時に使います。ふつうの失敗には `Result` を使います。
- **`join!`**（非同期）：`tokio::join!(a, b)` は、いくつかの非同期の仕事を同時に進めて、全部が終わるのを待つマクロ。かかる時間は、いちばん長い物の分だけです。
- **`datetime!`**（時間）：`time` の部品で、時差付きの日時を書くマクロ。`datetime!(2026-11-02 07:59 +9)` は「日本時間の 2026 年 11 月 2 日 7 時 59 分」。テストで「8 時より前」などの時刻を作るのに使います。
- **`.sent()`**（メール）：`MemoryTransport` にたまった、送ったメールの一覧を返すメソッド。
- **`.text()`**（メール）：メールの本文（文字の方）を返すメソッド（`TextBody` で返る）。後編のアップロードでは、フォームの項目（`field.text()`）の中身を文字列で読むのにも使います。
- **`.to()`**（メール）：メールの宛先の一覧を返すメソッド。`m.to()[0].address()` で、1 人目の宛先のメールアドレスを取り出せます。
- **`.address()`**（メール）：宛先から、メールアドレスの部分を取り出すメソッド。
- **`.subject()`**（メール）：メールの件名を返すメソッド。
- **`.split()`**（文字列）：文字列を、区切り文字のたびに分けるメソッド。`"a,b,c".split(',')` は `a`・`b`・`c` の 3 つ。結果は 1 つずつ取り出せる形（イテレーター）なので、`Vec` にしたい時は `.collect()` を付けます。
- **`.nth()`**（イテレーター）：並びの中から「n 番目」を 1 つ取り出すメソッド。番号は 0 から数えます。`env::args().nth(1)` は、コマンドの引数の 1 番目（0 番目はプログラム自身の場所）。無いかもしれないので、結果は `Some(値)` か `None`（`Option`）で返ります。
- **`.lines()`**（文字列）：文字列を、1 行ずつに分けて取り出せる形にするメソッド。行の終わりの改行（`\n` や `\r\n`）は含みません。`text.lines().count()` で行の数が分かります。
- **`.next()`**（イテレーター）：イテレーター（1 つずつ取り出せる物）から、次の 1 つを取り出すメソッド。無くなると `None` を返すので、結果は `Option` です。
- **`.last()`**（Vec（並び））：並びの最後の 1 つを `Option` で返すメソッド（空なら `None`）。最初の 1 つは `.first()`。
- **`.exists()`**（ファイルとフォルダ）：その場所に、ファイルかフォルダがあれば `true` を返すメソッド。`Path::new("memo.txt").exists()`。
- **`std::env::temp_dir()`**（ファイルとフォルダ）：OS の、一時的なファイルを置くフォルダの場所を返す関数（Windows なら `C:\Users\you\AppData\Local\Temp`）。
- **`MemoryTransport::new()`**（メール）：空の `MemoryTransport` を作る関数。複製（`.clone()`）しても同じ入れ物を指すので、片方をアプリに渡し、もう片方で中身を見られます。

> - **`CxTestBuilder::new()`**（Topcoat）：`CxTestBuilder` を作る関数。`.app_context(物)` で入れたい物を足し、`.build()` で `Cx` にします。サーバーを動かさずに、アプリの関数を直接試せます。
> - **`std::fs::read_dir()`**（ファイルとフォルダ）：フォルダの中の一覧を返す関数（`fs::read_dir` と同じ）。
> - **`std::fs::remove_dir_all()`**（ファイルとフォルダ）：フォルダを、中身ごと全部消す関数。ごみ箱を通さずに消えるので、場所をよく確かめて使います。
> - **`CxTestBuilder`**（Topcoat）：テストで、`Cx`（共有の物の入れ物）を組み立てる型。
> - **`MemoryTransport`**（メール）：メールを送らずに、メモリーの中にためておく係。テストで、送ったはずのメールの宛先や本文を確かめるのに使います。
> - **`TextBody`**（メール）：メールの文字の本文を表す型。`TextBody::Text(文字列)` で中身を取り出せます。
> - **`Text`**（データベース）：`Value::Text(s)` は、データベースの値が文字列であることを表す種類です。

- `#[sqlx::test]` は、**テストごとに空のデータベースを作り**、マイグレーションを流してから、`pool` を渡してくれます。テストどうしが、データを取り合いません。
- メールは `MemoryTransport` でメモリーにためて、宛先・件名・本文を確かめます（06 の `mail.outbox`）。
- 06 の 11 個のテストと同じ決まりに、**同時に 2 回押す・担当と完了・添付の後片付け・毎朝の通知**（2 つ）を足しました。
- このほかに、`src/` の中に小さなテストが 12 個（添付の中身・図面の JSON・CSV・ダウンロードの名前・入力フォーム）、`tests/security.rs` に安全のテストが 3 個あります（入力フォームと `security.rs` は 10 の巻で足した）。

実際に動かした結果（`cargo test`）：

コピー

```
     Running unittests src\lib.rs (target\debug\deps\webapp-8c86094e59d42b38.exe)
test drawing::tests::空なら図面なし ... ok
test drawing::tests::四角と線を読める ... ok
test drawing::tests::知らない図形はエラー ... ok
test employees::tests::shift_jisは断る ... ok
test files::tests::日本語の名前は_xxの形にする ... ok
test upload::tests::中身がpdfならpdfとして通る ... ok
test request_form::tests::正しい入力は通る ... ok
test request_form::tests::件名に改行は入れられない ... ok
test employees::tests::間違いのあるcsvは全部の間違いを返す ... ok
test employees::tests::正しいcsvを読める ... ok
test upload::tests::場所ごとの名前は最後だけにする ... ok
test upload::tests::拡張子をpngに変えたexeは断る ... ok
test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
     Running unittests src\main.rs (target\debug\deps\webapp-729cf1be2339cc29.exe)
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
     Running tests\security.rs (target\debug\deps\security-eafc82bc57e25215.exe)
test 図面の文字にscriptを入れても_ただの文字 ... ok
test 件名に改行を入れても_ヘッダーは増えない ... ok
test ファイル名で別のフォルダに書かせない ... ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
     Running tests\workflow.rs (target\debug\deps\workflow-e585d0fc1d21207c.exe)
test ほかの人の依頼は引き戻せない ... ok
test urlは1回しか使えない ... ok
test 回覧者の決まり ... ok
test 出し直すと最初の回覧者からやり直す ... ok
test 引き戻すと承認待ちの人のurlが使えなくなる ... ok
test 全員が承認すると回覧完了になり担当者に知らせる ... ok
test 出すと1人目にメールが届き回覧中になる ... ok
test 同時に2回押しても1回だけ進む ... ok
test 検証ではメールの宛先を書きかえる ... ok
test 担当を決めて完了にすると依頼者に知らせる ... ok
test 差し戻すとコメントが追記され依頼者にメールが届く ... ok
test 期限が切れたurlは使えない ... ok
test 添付ファイルはフォルダに置き_失敗したら消す ... ok
test 理由の無い差し戻しは断る ... ok
test 毎朝の通知は8時を過ぎてから1日1回だけ動く ... ok
test 毎朝の通知は止まっている承認を送り直す ... ok
test result: ok. 16 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 2.65s
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

14. 練習問題

1. `return_request` の中の `UPDATE steps SET decision = 'cancelled' …` を消して、テストを動かしてください。今のテストは失敗するでしょうか。失敗しないなら、「回覧者 3 人で、2 人目が差し戻したら、3 人目の URL は使えない」を確かめるテストを書いてください。この 1 行は、どんな事故を防いでいたのでしょう。
2. `finish` の中の `outbox.send_all(cx).await;` を、`commit` の前に動かすと、何が困りますか（ヒント: 6.2 の図）。

3. `jobs.rs` の `RUN_AT` を 7:30 に変えるには、どこを直せばよいですか。テストのどこも直す必要がありますか。
4. 社員の CSV に「内線番号」の列を足すとします。どのファイルを直しますか（表・データの形・CSV の読み込み）。
5. 06 の `services.py` と、この巻の `workflow.rs` を並べて、`approve` を見比べてください。同じ所と、違う所を 3 つずつ書き出してください。

用語の索引

用語	意味	出てくる所
マイグレーション	表の形を変える手順を、ファイルにして順に流す仕組み	4.1
機能（feature）	部品の中の、使うか選べる部分	2.2
セッション	ログインしている間、「この人はだれか」を覚えておく仕組み	5
トークン	推測できない、長い乱数の文字列	5・7
ハッシュ・指紋	元に戻せない形に変えた値（SHA-256・Argon2）	5・7
トランザクション	まとめて確定するか、まとめて取り消す、ひとまとまりの処理	6.1
送る物の箱（Outbox）	確定するまでメールをためておく入れ物	6.2
行の鍵（FOR UPDATE）	読んだ行を、ほかの人が同時に書きかえられないようにする	6.3
SQL インジェクション	外から来た文字が、SQL の一部として動いてしまう攻撃	6.5・10
BOM	ファイルの先頭の、目に見えない印（UTF-8 の BOM は 3 バイト）	8.2
`#[sqlx::test]`	テストごとに空のデータベースを作ってくれる印	13